Verifying Agent Repairs: Evidence Quality, Deployment Gains, and Cost

Anonymous ACL submission

Abstract

Self-improving agent loops change an agent's harness (its prompts, tool descriptions and control code) and keep a change only when verification evidence says it helps. We ask whether that evidence selects changes that help on unseen tasks, and what it costs, by deploying every candidate change, accepted or rejected, on held-out tasks. Inside RRSI, a self-improvement loop that edits harness code, run with DeepSeek models on τ^2-bench and AppWorld, most of 183 measured candidates did not help held out, and evidence that never ran the changed agent on solved tasks accepted changes that crashed the harness or blocked legitimate actions. A rule that stops evaluating a candidate once RRSI's rule is predicted to reject it was non-inferior to full re-evaluation on recorded decisions at 57–62% of the episodes, but against a reference that gained almost nothing, and it dropped a quarter of the reference's acceptances through stops that were overconfident about the cost change. Loops it drove gained 4.4 points less held out (90% interval -9.4 to -0.1) when each loop calibrated its own noise band, and 0.8 $[-2.9, +1.2]$ less when blocks of loops shared one, at 12% lower total loop cost. An ablation detected an adverse effect of the dropped admissions (-1.6 $[-3.0, -0.2]$, suggestive after adjustment for multiple contrasts) and left the effect of the proposer's history unresolved. Cheaper verification should be reported with its recall of the reference's acceptances and the calibration of its stops beside its decision value.

1 Introduction

A growing family of systems improves LLM agents from their own failures. The agent runs, an analyst model reads the failed trajectories, a proposer changes the agent's *harness*, the prompts, tool descriptions and code around a frozen model, and the loop keeps the change or discards it (Shinn et al., 2023; Zhao et al., 2024; Hu et al., 2025; Zhang et al., 2025a; Agrawal et al., 2026; Chen et al., 2026; Zhang et al., 2026b; Xia et al., 2026). The check that decides is where these systems differ most: some replay the failure with the change applied (Ma et al., 2025; Lin et al., 2026; Shah, 2026), some ask an LLM judge, some re-run tasks that used to pass (Chen et al., 2026), and RRSI re-evaluates every candidate twice on its whole training set (Xia et al., 2026). What every check is meant to estimate is a change's *deployment gain*, how much it raises success on tasks the loop has not seen, and that gain can differ from what the loop observes on its own tasks: RRSI reports in-distribution gains that shrink out of distribution (Xia et al., 2026).

This paper asks two questions about verification evidence. *Which evidence predicts a change's deployment gain?* And *is the evidence worth what it costs?* Answering the first needs ground truth that verification does not provide: deploying the changes, from the first step, on independent tasks. Answering the second needs an account of what verification costs and what the change it selects is worth over a deployment horizon, which we write as $\mathrm{Net}(N)$ (§3). We price every piece of evidence in agent episodes and answer both questions in two studies.

Study 1 (§4) is one round of attribution-driven repair of a DeepSeek-V4-Flash agent on the retail and airline domains of τ^2-bench (Barres et al., 2025), mostly on injected faults. No patch set chosen by ten verification policies showed a reliable improvement over applying the attributed patches unverified, which raised held-out success by 11.7 (retail) and 6.7 (airline) points, and replays could not resolve the differences between sets.

Study 2 (§5, registered) tests evidence with the power and the modification space Study 1 lacked. We build on RRSI's open-source loop (Xia et al., 2026), whose proposer edits harness code over many rounds of natural failures, port it to DeepSeek

models and τ^2-bench with replays of code-level changes, and replace its selection step with each kind of evidence, deploying every candidate and incumbent on held-out tasks under a registered analysis. Evidence that does not run the changed agent on solved tasks accepted a change that crashed the harness and one that blocked legitimate actions, and no rule was shown to beat keeping the incumbent. Sequential evaluation, which stops a candidate's evaluation once RRSI's rule is predicted to reject it, was non-inferior to full re-evaluation on the loop's recorded decisions at 57% of its episodes, but against a reference that itself gained almost nothing, and it dropped a quarter of the reference's acceptances: a rule can pass such a test while losing acceptances. On airline's first ten rounds, where full re-evaluation did gain, those decisions predicted a loss of 1.0 point. Six independent airline loops per arm driven by it gained 4.4 points less held out than full evaluation's, but their separately calibrated noise bands differed by chance; with one calibration shared by five loops in each of four blocks, sequential loops gained 0.8 points less $[-2.9, +1.2]$, for 37% fewer candidate-evaluation episodes but only 12% lower loop cost, which the proposer dominates. An ablation detected an adverse effect of the admissions the rule's stops dropped, suggestive once the registered contrasts are adjusted for multiplicity, and left the effect of the proposer's history unresolved. The stops were overconfident where RRSI admits a change for being cheaper or new, a defect visible offline in the rule's recall of full evaluation's acceptances, though not in its decision value.

Our contributions are a candidate-level evaluation protocol that deploys every candidate, accepted or rejected, with an explicit cost account; a diagnosis of how decision-value comparisons against a reference that gains little can hide lost acceptances, and of the miscalibrated stops that lose them; registered loop comparisons that examine admission, the proposer's history, calibration and total cost; and the environments, banks and evidence of both studies. The loop, its regularised edits and its selection rule are RRSI's (Xia et al., 2026); we change only the evidence its selection step uses.

2 Related Work

Self-improving agents and how they verify. Reflexion, Self-Refine and ExpeL turn feedback into verbal lessons (Shinn et al., 2023; Madaan et al., 2023; Zhao et al., 2024); ProTeGi, OPRO, TextGrad and DSPy rewrite prompts (Pryzant et al., 2023; Yang et al., 2024; Yuksekgonul et al., 2025; Khattab et al., 2024); ADAS, the Darwin Gödel Machine and GEPA search over designs (Hu et al., 2025; Zhang et al., 2025a; Agrawal et al., 2026); ACE evolves a context playbook (Zhang et al., 2026b). HarnessFix repairs harness code and promotes a bundle of repairs only if it solves more validation tasks while breaking at most two (Chen et al., 2026); Bayesian-Agent applies threshold rules to posteriors over skills (Wu et al., 2026). RRSI regularises harness self-improvement with annealed edit budgets, a critic for benchmark-specific edits, pruning, and selection by full re-evaluation against a calibrated noise floor (Xia et al., 2026). These systems report the success of the final agent; we evaluate the evidence behind each decision.

Failure attribution. Locating the responsible agent and step is far from solved (Zhang et al., 2025b; Liu et al., 2026); attribution models, taxonomies and dependency tracing address it (Zhang et al., 2026a; Zhu et al., 2025; Li et al., 2026; Cemri et al., 2025, 2026).

Verification by intervention. DoVer validates hypotheses by editing a message or plan and rerunning (Ma et al., 2025); REFLECT tests patches by targeted replay (Lin et al., 2026); CausalFlow and Causal Agent Replay score step interventions causally (Bonagiri et al., 2026; Shah, 2026); AgentDebugX gates reruns by policy (Zhu et al., 2026). Their evaluations report whether an intervention repairs the failure it targets, how accurately steps are attributed, or how the final agent performs. Closest to our question, a failure critic with an AUROC of 0.94 lowered one agent's success by 26 points when its predictions triggered interventions within runs, because interventions also derail runs that would have succeeded (Vasudev et al., 2026). We study the analogous gap for harness changes selected over many rounds, between a rule's recorded decisions and the loops it drives. Program repair has long warned that a patch passing the failing test may not fix the fault (Smith et al., 2015).

Sequential evaluation and racing. Stopping the evaluation of candidates that are unlikely to win is an old idea. Hoeffding races and F-race discard configurations once a bound or a Friedman test separates them from the best (Maron and Moore,

1993; Birattari et al., 2002); successive halving and Hyperband spread a budget over rounds of elimination (Jamieson and Talwalkar, 2016; Li et al., 2018); fast cross-validation drops parameter settings by sequential tests across growing subsamples (Krueger et al., 2015); and clinical trials stop for futility when the predictive probability of a conclusive result is small (Spiegelhalter et al., 1986). Our sequential rule is a futility stop whose target is a self-improvement loop's admission rule, with its noise band, cost budget and novelty branch.

3 Verification Evidence and Deployment Gain

Changes and their deployment gain. A loop holds an incumbent harness h and proposes a change c: a rewritten prompt section or tool description, a code edit, or a set of these. Its *deployment gain* $\Delta p(c)$ is the difference in success rate between h with c and h alone, measured from the first step on tasks independent of everything the loop saw. A verification rule spends C_{verify} episodes on evidence, forms an estimate $\widehat{\Delta}$ of the change's effect and accepts the change if the estimate clears a threshold.

Evidence. *None* accepts the proposed change. An *LLM judge* reads the change and the failures it targets and predicts its effect, at a cost in tokens but no episodes; a *single-step check* regenerates the failing step with the change and asks a judge whether the error is gone. A *targeted replay* resumes a failed trajectory from the failing step with the change and succeeds with probability $b + (1 - b)q$, where q is the share of such failures the change repairs; a *null replay* does the same without the change and measures the *spurious recovery* rate b, without which replay success cannot be interpreted. A *regression run* is a previously solved task, from the first step, with the change; a *sample* or *full evaluation* runs candidate and incumbent on a random sample of, or all, training tasks. Replays and checks see only the failures a change targets; only regression runs and evaluations see harm to tasks that already succeed.

Is the evidence worth its cost? Compare a rule E with a reference rule, here applying changes without verification. Suppose E's choices change the deployed agent's success by Δp and its running cost by Δc_{run} per episode, that E spends C_{verify} episodes, and that it changes the one-off cost of reviewing and maintaining the applied changes by ΔC_{change}. Over the next N deployed episodes E is worth

$$\text{Net}(N) = N\,(v\,\Delta p - \Delta c_{\text{run}}) - C_{\text{verify}} - \Delta C_{\text{change}}, \quad (1)$$

with every cost in episodes of the unmodified agent's running cost and v the value of one success in the same unit. We count three things: *episodes*, runs of the agent on a task (a replay from a failing step counts as one); *episode equivalents*, which price replays (and, in Study 1, checks) by their tokens relative to an episode and are the unit of C_{verify}; and *dollars* at list prices, used only for a whole loop's cost, which also includes the analyst's and proposer's model calls. The one-off costs do not grow with N, so once Nv is large compared with C_{verify} the sign of $\text{Net}(N)$ is the sign of $v\Delta p - \Delta c_{\text{run}}$: evidence pays if the changes it selects are better, or cheaper to run. Its worth then rests on its *quality*, which we measure in two ways: how well its estimates correlate with, and are calibrated against, measured deployment gains, and the *decision value* of a rule that uses it, the deployment gain of its choice averaged over rounds, zero when it keeps the incumbent. Appendix A splits a patch's net effect into repair, spurious recovery and regressions.

4 Study 1: One Round of Attribution-Driven Repair

Setup. The agent is DeepSeek-V4-Flash, with the provider's default sampling, on the retail and airline domains of τ^2-bench (Yao et al., 2024; Barres et al., 2025), which have an LLM user simulator and database-state rewards; attribution, taxonomy induction (Wan et al., 2024), patch generation and judging use DeepSeek-V4-Pro. Most training failures come from one injected fault per episode (natural failures: 22 of 137 on retail, 28 of 112 on airline). For 36 (category, component) cells per domain the analyst writes three patches, and for every patch we record 20 targeted replays with single-step checks and 4 to 20 regression runs, and for every category 20 null replays. Eleven policies spend a budget of episodes from these *intervention banks* and output a patch set: *Apply attributed* applies the most-blamed components' first patches without evidence, and the others accept patches on replays, regression gates in the style of HarnessFix (Chen et al., 2026), uncertainty sampling or multi-fidelity allocation. Each policy's budget-10 set is deployed on 40 retail and 20 airline unseen tasks, 3

episodes each; retail was the development domain (Appendices B and M).

Results. Applying the attributed patches without verification raised held-out success from 75.8% to 87.5% on retail ($+11.7$ points, 90% interval $[4.5, 18.8]$) and from 68.3% to 75.0% on airline ($+6.7$, $[-3.3, 16.6]$; Table 5). No verified set showed a reliable improvement over it: the differences range from -4.3 to $+0.3$ points on retail and from -4.0 to -0.3 on airline, with intervals reaching $+6.0$ and $+8.5$, so this is not equivalence. Most of retail's gain comes from fault-injected episodes (Appendix K). The sets' gains predicted from replays differed by only 1.0–1.7 points, below 2.6–3.8 points of deployment noise; 20–23% of unpatched replays succeed anyway (Table 6). Whether verification pays thus turns on whether it selects better changes (Table 7), which Study 1 could not resolve; it does not show that verification brings no benefit.

5 Study 2: Verification Evidence Inside RRSI

Study 1 patches only text, in one round, mostly on injected faults. Study 2 builds on RRSI (Xia et al., 2026), whose proposer edits harness code over many rounds of natural failures. We run RRSI's loop, collect every kind of evidence for every candidate it measures, deploy every candidate, accepted or rejected, and ask what each kind of evidence would have decided. Table 1 lists the experiments, and Appendix N describes the port, the registrations and every change made after them.

Loop. We vendor RRSI's code (Apache 2.0) and change only its model backend, its cost accounting and one rule for empty proposals. Analyst, proposer, critic and judge are DeepSeek-V4-Pro with thinking disabled; the agent and user simulator are DeepSeek-V4-Flash at temperature 0; the starting harness is τ^2's reference agent. Each round the proposer drafts $m = 2$ candidates under an edit budget that anneals from 4 to 1, a critic rejects edits that hard-code task data or touch the grader, and RRSI evaluates the candidates that pass a smoke test twice on every evolve task. It accepts a candidate that beats the incumbent by more than a noise band δ, calibrated from three evaluations of the base harness, or that is within the band and cheaper to run or of an untried kind, and it rejects a harness that raises errors in more than 2% of trials (Appendix N). Evolve tasks are τ^2's training split (74 retail, 30 airline) and held-out tasks its test split (40 and 20). The loop ran $T = 20$ rounds per domain on three independent trajectories, r1–r3.

Evidence and decisions. For each measured candidate we collect the evidence of §3 as estimates of its change ΔS in evolve success (Table 22, Appendix N): an LLM judge's prediction from the diff and the failures it targets (no episodes); *sample*@b, the paired difference on b evolve episodes; *replay*@b, the failure share times the success rate of b targeted replays of failed evolve trials; *replay*$-$*null*@b, which subtracts the success rate of null replays with the incumbent; and *net*@b, which splits b between both kinds of replay and regression runs on solved tasks, with or without a Beta prior on the repair rate. Budgets of 10, 40 and 80 episodes per candidate are subsamples of one collection (four replays with each candidate and four null replays of each failed evolve task, at most 20 per round); a replay re-executes the recorded prefix through the new harness and goes live where it would first act differently. Every rule decides with RRSI's rule, with ΔS replaced by its estimate and δ by twice the estimate's standard error; full evaluation keeps RRSI's calibrated δ. Appendix Q decides every rule with one criterion.

Sequential full evaluation. RRSI evaluates a candidate on N paired episodes, two per evolve task. The sequential rule runs them in batches of 10 in random order. After n episodes with mean paired difference m it stops the candidate if its harness errors already exceed 2% of N, or if the predictive probability that RRSI's rule will admit it after all N episodes is below $\gamma = 0.05$. That probability treats the final ΔS as normal with mean m and variance $\left(\frac{N-n}{N}\right)^2 s^2 \left(\frac{1}{n} + \frac{1}{N-n}\right)$, where s^2 is the larger of the running variance of the paired differences and $S_t(1 - S_t)$; it holds the cost change at its running estimate, knows the novelty count, and adds RRSI's two ways to admit: above the band within the cost budget, and inside the band with a positive score for cost and novelty. A candidate that is not stopped completes its evaluation and is decided as under full evaluation. Stopping thus changes only admission: it can lose an acceptance but never add one. A stopped candidate enters RRSI's history with its early-stopped estimate. *Sequential sampling* caps N at 80 sampled episodes and decides with the sample's own band.

Experiment	Question	Independent units	Endpoint
Study 1 (§4)	Which verified patch set deploys best after one round?	2 domains, one bank each; 40 and 20 held-out tasks	held-out success of the budget-10 set minus the unverified set
r1 (R)	Which evidence decides best inside RRSI?	1 loop per domain, 40 rounds in 7 incumbent blocks; 40 and 20 test tasks	(P1) transfer of the final harness; (P2) D at $b = 40$
r2+r3 (R, primary)	Are cheaper rules non-inferior to full evaluation?	2 loops per domain, 80 rounds in 23 blocks; same tasks	D, non-inferiority at 0.75 points per round, Holm
r4 (D); E3 (R)	Does a thinking proposer, or a code agent, change this?	1 airline loop, 10 rounds; 1 AppWorld loop, 20 rounds, 168 test tasks	as r2+r3
CL (D); IL (R)	Do loops driven by sequential evaluation transfer as well?	6 + 6 airline loops (4 + 4 concurrent), 10 rounds; 20 test tasks	transfer at round 10, non-inferiority at 5 points; cost
BK (R)	Does the loss replicate under shared calibration, and does it come from admission or history?	4 blocks of 5 airline loops sharing a calibration, 10 rounds; 20 test tasks	transfer at round 10: rule differences, admission and history effects
Post hoc	Where does the sequential rule err?	as above	recall, calibration, noise-band adjustment, cost-aware rule

Table 1: The experiments. R: registered; D: registered as descriptive; Study 1 used retail for development. Rounds sharing an incumbent form a block, the unit of the block bootstrap; trials within units are repetitions (Table 37). D: decision value, the mean held-out gain of a rule's choice per round.

Ground truth and estimand. Every incumbent and measured candidate is deployed from the first step on the held-out tasks, 240 episodes each (retail 40 tasks $\times$ 6, airline 20 $\times$ 12), and the base and final harnesses on 400 and 240. A candidate's deployment gain $\Delta p(c \mid H_t)$ is its held-out success minus that of the incumbent H_t it was proposed against, paired by task. A rule E's *decision value* on a set $\mathcal{T}$ of rounds of both domains is $D(E) = \frac{1}{|\mathcal{T}|} \sum_{t \in \mathcal{T}} \sum_c P_E(c \mid t)\, \Delta p(c \mid H_t)$, the mean expected deployment gain of its choice per round (zero when it keeps H_t), with P_E taken over 400 subsamples of the evidence. For r1 the registered endpoints are (P1) *transfer*, the held-out gain of the final harness over the base, and (P2) D at $b = 40$ with three Holm-adjusted contrasts (Appendix O).

Further trajectories. Because r1's values rested on one trajectory per domain and its registered analysis gave RRSI's error check to full evaluation only, we registered r2 and r3 before any of their episodes ran, with this study's primary analysis: D over their 80 rounds with the error check applied to every rule's own episodes (the *common error check*), and one-sided non-inferiority tests of sample@40, net@40 and replay−null@40 against full evaluation at a margin of 0.75 points per round, Holm-adjusted at $\alpha = 0.05$; the two sequential rules form a second family. A bootstrap resamples blocks of rounds that share an incumbent, and held-out tasks, within domain and trajectory, so its intervals generalise to new held-out tasks and to rounds like the observed ones, not to new improvement runs, which only the loop comparisons below resample. E3 runs the same loop and analysis on AppWorld (Trivedi et al., 2024) (Appendix P).

5.1 Results

The loop. Over r1–r3 RRSI drafted 240 candidates and measured 154, 106 of which changed harness code. It accepted 5 candidates on r1, all in the first five rounds, and 21 on r2 and r3, as late as round 19; 19 of the 26 were within the noise band. Candidates' mean deployment gain was -1.5, -1.25 and $+0.14$ points on r1–r3, and 63 of 154 were positive.

Transfer (P1). Airline's evolve gains transferred on r1 and r2, by $+8.75$ $[+2.5, +15.8]$ and $+7.9$ $[+2.5, +14.2]$ points against $+13.3$ on evolve tasks. On r3 its six acceptances, all within the band, left evolve success unchanged ($+1.7$ $[-2.5, +5.8]$ held out). Retail's evolve gains of 4.1, 4.7 and 7.4 points never transferred: -0.25 $[-3.0, +2.75]$, 0.0 $[-2.0, +1.75]$ and -4.25 $[-7.5, -1.0]$.

First trajectory. On r1, one candidate (retail, round 6) added a check that reads an attribute the harness never sets: the harness raised errors in half of its evolve episodes and held-out success fell by 51.7 points, yet the critic passed it and the judge predicted a gain. It causes most of the losses of no verification, the judge and replay without null replays. In r1's registered analysis full evaluation led with $+0.22$ points per round $[-0.06, +0.62]$ and net@40 was not shown non-inferior (Holm $p = 0.17$; Appendix O).

Rule	Ep.	D [90%]	$p_{0.75}$	Agree	Recall
Full (RRSI)	141	+0.04 [−0.35, +0.43]	–	1.00	1.00
Keep incumbent	0	0	0.004†	0.74	0.00
None	0	−0.47 [−1.52, +0.62]	–	0.25	0.62
LLM judge	0	−0.40 [−1.41, +0.67]	–	0.30	0.48
Sample@40	54	−0.06 [−0.58, +0.45]	0.045	0.67	0.55
Net@40	49	−0.10 [−0.62, +0.39]	0.045	0.63	0.54
Replay−null@40	45	−0.39 [−0.79, −0.03]	0.14	0.68	0.23
Seq. full	81	+0.02 [−0.34, +0.35]	<0.001	0.93	0.74
Seq. sample	53	+0.07 [−0.26, +0.41]	<0.001	0.78	0.48

Table 2: Primary analysis, r2+r3 (80 rounds; from Tables 21 and 26). Ep.: evidence episodes per round; D in points per round; $p_{0.75}$: non-inferiority to full evaluation at the registered margin, Holm-adjusted (†post hoc); Agree: share of rounds with full evaluation's choice; Recall: share of its 21 acceptances the rule makes.

Primary analysis (r2 and r3). On their 80 rounds, full evaluation's decision value was $+0.04$ points per round $[-0.35, +0.43]$ at 141 episodes (Table 2; every rule in Table 21). Sample@40 and net@40, at 54 and 49 episodes, were non-inferior to it (Holm $p = 0.045$ each); replay−null@40 was not ($p = 0.14$), and its own value, -0.39 $[-0.79, -0.03]$, lies below zero, 40% of it from one candidate (r3 retail, round 11) that held every write whose arguments it could not trace to earlier tool results. It raised no harness errors but blocked legitimate writes, lowering held-out success by 12.5 points; full evaluation, sample and net, which run solved tasks, rejected it, while replay−null and the judge accepted it. No rule was shown to beat keeping the incumbent, and because full evaluation gained so little, keeping the incumbent is itself non-inferior to it at the registered margin ($p = 0.004$; Table 25). These tests show that the rules lose little against a rule that gained little, not that they preserve a useful improvement process; this makes the comparison informative about the criterion itself, as a rule can pass it while dropping many of the reference's acceptances.

Sequential evaluation on the recorded rounds. Sequential full evaluation was non-inferior to full evaluation, -0.02 $[-0.12, +0.07]$ at 81 episodes per round (57%), as was sequential sampling, $+0.03$ $[-0.26, +0.35]$ at 53 (both $p < 0.001$). Post hoc, sequential full evaluation stays non-inferior at a margin of 0.25 points ($p < 0.001$), which keeping the incumbent fails, and when any one trajectory and domain is left out (Table 36). It made full evaluation's choice in 93% of rounds but only 74% [67, 81] of its 21 acceptances (Table 2); those it would drop gained no less held out ($+0.30$ against $+0.11$ points), so it lost little only because full evaluation's acceptances were worth little (Table 26). Recall is not a goal in itself, since full evaluation also accepts changes that do not help, which is why we report the held-out gain of the acceptances kept and dropped.

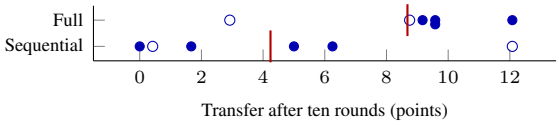


Figure 1: Transfer at round 10 of the twelve airline loops of the registered comparison (held-out success after round 9 minus the base, 240 episodes each). Filled: the eight loops run concurrently; open: cl1, cl2, r2, r3. Bars: arm means.

Is the stopping rule calibrated? No (post hoc; Table 32, Appendix R). Candidates stopped at a mean predicted probability of 0.02 would have been admitted by full evaluation in 8% of cases on r2+r3, 3% on AppWorld and 13% [6, 20] in the six live loops below. Holding the cost change at its running estimate causes this: with the complete evaluation's cost change in the same formula, looks below γ were followed by admission in at most 2% of cases. The rule stopped about a third of the candidates RRSI admits inside its band for being cheaper or new, where admission turns on the sign of a small cost change, in recorded and live rounds alike, against 4% of those admitted above the band. A post hoc correction, designed on these rounds, integrates the predictive probability over the uncertainty of the final cost change: it halves the stops inside the band and keeps 88% of full evaluation's acceptances on r2+r3 at 64% of the episodes (89% instead of 69% on airline, at 77%; Table 31). BK below runs it in registered loops.

Independent loops. The analyses above score decisions in the states full evaluation's loop visited. To compare loops, we registered two airline loops driven by sequential full evaluation (cl1, cl2), then eight more run concurrently, four with each selection step, from one commit (IL; Appendix S). With r2 and r3, six loops per arm ran rounds 0–9. Transfer after ten rounds averaged $+4.2$ points with sequential and $+8.7$ with full evaluation, a difference of -4.4 $[-9.4, -0.1]$ (Figure 1; -6.9 among the concurrent loops); non-inferiority at 5 points was not shown ($p = 0.38$). The recorded de-

Arm	Admission	History	B1	B2	B3	B4	Mean	Acc.	Cand. ep.	Cost ($)
Full	full	full	$+6.7$	$+9.6$	$+11.2$	$+7.5$	$+8.8$	12	855	3.40
History only	full	early	$+4.6$	$+10.4$	$+14.2$	$+10.8$	$+10.0$	12	900 ($+5\%$)	3.60 ($+6.0\%$)
Cost-aware seq.	cost-aware	early	$+7.1$	$+8.3$	$+13.3$	$+10.0$	$+9.7$	11	685 (-20%)	3.31 (-2.5%)
Admission only	seq.	full	$+7.1$	$+5.0$	$+11.2$	$+7.5$	$+7.7$	11	900 ($+5\%$)	3.49 ($+2.7\%$)
Sequential	seq.	early	$+4.6$	$+8.3$	$+10.8$	$+7.9$	$+7.9$	10	535 (-37%)	3.00 (-11.8%)

Contrast	90% / 95% interval	Est.	Holm p (one-sided)
Sequential − full	$[-2.9, +1.2]$ / $[-3.4, +1.7]$	-0.8	0.68
Cost-aware − full	$[-1.1, +3.0]$ / $[-1.6, +3.5]$	$+0.9$	1.00
Cost-aware − sequential	$[-0.2, +3.8]$ / $[-0.8, +4.3]$	$+1.8$	0.28
Admission (main effect)	$[-3.0, -0.2]$ / $[-3.4, +0.2]$	-1.6	0.19
History (main effect)	$[-2.2, +3.7]$ / $[-4.2, +5.6]$	$+0.7$	1.00
Interaction	$[-3.9, +1.8]$ / $[-4.6, +2.5]$	-1.0	–

Table 3: Experiment BK (registered): transfer after ten rounds on airline (held-out success after round 9 minus the block's base, points, 20 tasks × 12 trials) of five loops in each of four blocks that share one calibrated noise band. Admission: the rule that decides acceptance; history: the estimates stopped candidates enter the proposer's history with. Acc.: changes accepted over the four blocks; Cand. ep. and Cost: candidate-evaluation episodes and dollar cost per loop, as if run alone, and their change against full evaluation. Contrasts: means over blocks from the registered lineage model (t, Satterthwaite df), with the registered (unadjusted) 90% and post hoc 95% intervals and a post hoc Holm adjustment over the five registered contrasts, one-sided in the direction of each registered reading; main effects $(s + a - f - h)/2$ (admission) and $(s + h - f - a)/2$ (history).

cisions of airline rounds 0–9 of r1–r3 predict -1.0 $[-3.7, +0.8]$ (Table 40). But each loop calibrated its own noise band, and by chance the sequential loops' bands were wider (9.8 against 6.7 points); a wider band leaves more candidates to RRSI's in-band branch, where the rule's stops are least reliable, and adjusting for it post hoc halves the pooled difference, to -2.1 $[-6.2, +1.9]$, though the concurrent loops keep -5.8 $[-9.4, -2.2]$.

Blocked loops (BK). To separate the rule from its calibration and locate the loss, we registered a third comparison after IL's results (Appendix T). Four calibration blocks each gave one noise band to five airline loops that differ only in selection: full evaluation (f), sequential full evaluation (s), the cost-aware rule (c) and two ablations. Loop h admits as f does but enters stopped candidates into the proposer's history with their early-stopped estimates, as s does; loop a admits as s does but shows the proposer every complete evaluation. Loops of a block share computation only while their states are identical, so each has the distribution of an independent loop of its rule; the four blocks, not the twenty loops, are the independent units, and a contrast's variance combines a block-plus-rule residual from f, h and c with the spread over blocks of the coupled differences a−f and s−h. Sequential loops gained 0.8 points less than full evaluation's $[-2.9, +1.2]$ (Table 3), close to the recorded decisions' prediction: IL's loss did not replicate under shared calibration, though a loss of nearly three points is not excluded (pooled with IL and adjusted for the band, -1.7 $[-4.1, +0.8]$). The admission main effect, -1.6 $[-3.0, -0.2]$, met its registered reading, which uses unadjusted 90% intervals; at 95% its interval includes zero and Holm's adjustment over the five registered contrasts gives $p = 0.19$, so it is suggestive evidence that the dropped admissions cost transfer: in 7 of the 16 rounds in which full evaluation would have accepted in s's states, the rule had stopped its choice, and IL's four lost acceptances, deployed, gained $+3.3$ points on average. The history effect, $+0.7$ $[-2.2, +3.7]$ on 1.4 Satterthwaite df, does not exclude harm and is unresolved. The cost-aware rule made full evaluation's choice in all 40 of its rounds at 80% of the candidate-evaluation episodes; its gains over the registered rule, $+1.8$ $[-0.2, +3.8]$, and over full evaluation, $+0.9$ $[-1.1, +3.0]$, are not resolved.

Cost. What sequential evaluation saves is candidate-evaluation episodes, not the loop's cost: its IL loops ran 39% fewer such episodes at 10% lower cost ($-\$0.35$ $[-0.57, -0.10]$), and its BK loops 37% fewer at 12% lower cost (Table 3), because the proposal side's model calls dominate; the cost-aware rule saved 20% of the episodes and 2.5% of the cost. These are the costs of operating a rule in a loop; Table 2 counts the episodes of recorded decisions, and its fixed-budget rules would also need a full evaluation of each accepted candidate to continue a loop (Appendix Q). No operating point is clearly better: the registered rule

saves 12% of the cost at an unresolved loss, the cost-aware rule 2.5% at an unresolved gain. On r2 and r3, $\mathrm{Net}(10^4)$ at $v = 10$ against keeping the incumbent was -108 episodes for full evaluation and positive only for sequential sampling, $+10$ $[-315, +350]$.

Other trajectories. Pooled over r1–r3, full evaluation gained $+0.10$ $[-0.18, +0.41]$ per round, with the same non-inferiority results; accepting every candidate gained -1.87, -1.08 and $+0.14$ on r1–r3, so trajectories differed more than rules. With a thinking proposer, one airline trajectory (r4, descriptive) measured six candidates, all helpful. On AppWorld (E3) the 29 measured candidates barely changed held-out success and the evolve gain of 4.2 points did not transfer (0.0 $[-2.1, +2.1]$); the @40 rules missed non-inferiority (Holm $p = 0.053$), while sequential full evaluation was non-inferior at 62% of its episodes ($p < 0.001$; Table 24).

6 Discussion

What the evidence shows. Our results identify failure modes of verification, not a ranking of evidence by predictive quality. Most changes did not help held out, so verification earned its place by rejecting harmful ones, and only evidence that runs the changed agent on solved tasks rejected both kinds we saw.

Recorded decisions and loops. Scoring each decision once, in the states full evaluation chose, ignores that an accepted change becomes the base of later rounds and that every evaluated candidate enters the proposer's history; BK detected an adverse effect of the first, suggestive after adjustment, and left the second unresolved. Calibration noise is large: bands ranged from 2.7 to 12.5 points across IL's loops, and loops sharing a band varied by 1.5 points about the block and rule means, against 3.1–4.6 between IL's loops. The smaller loss under shared calibration is consistent with a calibration imbalance contributing to IL's result, but the experiments do not isolate it: their trajectories and band ranges also differ, and the adjusted concurrent contrast of IL remains negative. Loops compared with each other should share or balance their calibration.

The defect was visible offline. The offline analysis did detect this rule's problem, and its decision value hid it: the rule dropped a quarter of full evaluation's acceptances through stops that were overconfident on RRSI's cost branch, in recorded rounds as in the loops (Tables 2 and 32), but those acceptances were worth little in the primary analysis. In BK's loops the dropped admissions were the detected source of loss, and the rule that models the cost change made every one of full evaluation's choices, though over only 11 acceptances. Fidelity to the reference and value are different objectives: keeping acceptances that do not help is worth nothing. But a decision value averaged over rounds in which the reference gains little cannot tell a rule that would keep a loop's improvements from one that would drop them, and the acceptances this rule dropped gained no less than those it kept. A cheaper rule's recall of the reference's acceptances, with their held-out gain, and the calibration of its stops on each branch of the acceptance rule should be reported beside its decision value; here they would have flagged the rule before any loop ran.

Resolution limited these studies more than price. A round of full evaluation costs about 25 cents of τ^2 compute, but comparing evidence needs per-round differences of about 0.7 points (Appendix O); more trajectories and held-out tasks would help more than more trials (Table 37).

Implications for self-improvement loops. Loops that report only the final agent's held-out success (Chen et al., 2026; Zhang et al., 2026b; Xia et al., 2026) measure all their decisions at once; on retail and AppWorld, evolve gains of 4–7 points never transferred. Replays need null replays and regression runs. A stopping rule must model the uncertainty of every condition in the acceptance rule it predicts.

7 Conclusion

Inside RRSI's loop most changes did not help held out, and evidence that never runs the changed agent on solved tasks accepted harmful ones. Sequential evaluation matched full re-evaluation's decision value on recorded rounds only because the reference gained little, and it dropped a quarter of the reference's acceptances. Its loops gained 4.4 points less held out under separate calibration and 0.8 under shared calibration, with 12% lower total loop cost. Cheaper verification should be reported with its recall of the reference's acceptances and the calibration of its stops beside its decision value, and loops compared under shared calibration.

Limitations

Both studies use one model family, with DeepSeek-V4-Flash as agent and DeepSeek-V4-Pro as analyst, proposer and judge, and two customer-service domains of τ^2-bench; Study 1 adds ShopDesk and Study 2 one AppWorld trajectory. Study 1's patches rewrite only policy sections, tool descriptions and a step limit; it runs a single round, mostly on injected faults (natural failures are 16–25% of training failures), and deploys patch sets only at budget 10, on 120 (retail) and 60 (airline) held-out episodes, where most of retail's gain comes from fault-injected episodes. Its bank analyses rest on a harm model fitted to at most 20 regression runs per patch, and its edge priors use an analyst accuracy of 0.7 unless stated. Study 1's agent used the provider's default sampling and Study 2's uses temperature 0. Study 2's proposer is likely weaker than the model RRSI was developed with; a fourth trajectory with a thinking proposer was cut short because its drafts took over an hour and mostly ended without a proposal; on airline all six of its measured candidates helped, which suggests that a stronger proposer leaves less for verification to decide, but rests on one trajectory (Appendix N). Its held-out sets are τ^2's test splits of 40 and 20 tasks, on which most candidates' gains were not resolved. Its decision values are conditional on RRSI's trajectories: they score each rule's decisions in the states RRSI visited. The loop comparisons use one domain and ten rounds, at equal rounds rather than equal cost. IL's arms mix loops run at different times (cl1, cl2, r2 and r3, whose outcomes were known when it was registered, and eight concurrent loops), with arm labels fixed rather than randomised and noise bands that came out wider in the sequential arm. BK has four blocks, its independent units; its bands (5.4 to 8.2 points) were narrower than those of IL's sequential loops, and a wider band may enlarge the rule's loss. Its registration read each of five contrasts by an unadjusted 90% interval; the admission effect does not survive a 95% interval or Holm's adjustment, its history effect is imprecise (1.4 Satterthwaite df), its loops share computation while their states are identical, and its cost-aware loops' gain over the registered rule was not resolved. Whether calibration explains the difference between IL and BK is not identified: the two experiments also differ in their trajectories and band ranges. Our diagnosis concerns RRSI's acceptance rule, under which 19 of the 26 acceptances of r1–r3 fell inside the noise band, where cost and novelty decide, and one stopping rule; it does not establish general properties of replay, judging or sequential evaluation, and a loop with a different acceptance criterion may behave differently. The cost-aware rule's fidelity in loops rests on 11 acceptances in bands of 5.4 to 8.2 points. BK was registered after IL's results, and the cost-aware rule was designed post hoc on the recorded rounds before it was registered for BK. $\mathrm{Net}(N)$ sets the change cost to zero; charging each accepted change a fixed review cost favours rules that accept less and leaves the order of the sequential rules and full evaluation unchanged (Appendix Q). Three trajectories per domain show that between-trajectory variation is as large as the differences between rules, and the pooled analysis treats trajectories as fixed. The non-inferiority margin of 0.75 points per round was set before any result, but it is large relative to the decision values we observed; the smaller margins, the agreement rates, their split by full evaluation's decision, the decision-rule ablation, the coverage analyses, the calibration, threshold, matched-cost and leave-one-out analyses, the matched offline comparison, the noise-band adjustment, the accounting of IL's loops and the offline evaluation of the cost-aware stopping rule are post hoc. On r1, the registered analysis gave RRSI's error check to full evaluation only, so the comparison with a common check is post hoc there; its held-out count was raised after six rounds, when some held-out episodes had run and a few results had been seen (Appendix N), and the registered count gives the same conclusions. While r2 and r3 ran, a few of their held-out success rates appeared in progress logs before analysis; the analysis code had been fixed and tested on r1 only. The sequential rules were designed on r1's rounds and confirmed on r2 and r3. r1's component labels did not recognise our harness's files, which may have changed what its proposer was told to explore; r2 and r3 use corrected labels, so the trajectories differ in that respect as well as in sampling. Token prices for the models we used change; we report costs in episodes, and weighting tokens by list prices changes no ranking.

Ethics Statement

Self-improving agents change their own behaviour without a person reviewing each change. In Study 2 most proposed changes did not raise held-out success; one that crashed the harness in half of

its episodes, and another that blocked legitimate writes, passed both an LLM critic and an LLM judge. That Study 1 detected no gain from verification does not show that verification is unnecessary; our measurements cannot exclude small harms per patch and should not be read as licence to skip review. On ShopDesk a single patch silently broke a quarter of tasks (Appendix M). Our environments use synthetic customer data only.

References

Lakshya A. Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziems, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J. Ryan, Meng Jiang, Christopher Potts, Koushik Sen, Alexandros G. Dimakis, Ion Stoica, Dan Klein, Matei Zaharia, and Omar Khattab. 2026. GEPA: Reflective prompt evolution can outperform reinforcement learning. In *International Conference on Learning Representations*.

Victor Barres, Honghua Dong, Soham Ray, Xujie Si, and Karthik Narasimhan. 2025. τ^2-bench: Evaluating conversational agents in a dual-control environment. *arXiv preprint arXiv:2506.07982*.

Mauro Birattari, Thomas Stützle, Luís Paquete, and Klaus Varrentrapp. 2002. A racing algorithm for configuring metaheuristics. In *Proceedings of the Genetic and Evolutionary Computation Conference (GECCO 2002)*, pages 11–18.

Akash Bonagiri, Devang Borkar, Gerard Janno Anderias, Setareh Rafatirad, and Houman Homayoun. 2026. CausalFlow: Causal attribution and counterfactual repair for LLM agent failures. *arXiv preprint arXiv:2605.25338*.

Mert Cemri, Andrei Cojocaru, Melissa Pan, Shu Liu, Shubham Agarwal, Alexander Krentsel, Jay Tang, Kannan Ramchandran, Joseph E. Gonzalez, Matei Zaharia, Alexandros G. Dimakis, and Ion Stoica. 2026. Fantastic adaptive taxonomies and how to use them. *arXiv preprint arXiv:2607.16387*.

Mert Cemri, Melissa Z. Pan, Shuyi Yang, Lakshya A. Agrawal, Bhavya Chopra, Rishabh Tiwari, Kurt Keutzer, Aditya Parameswaran, Dan Klein, Kannan Ramchandran, Matei Zaharia, Joseph E. Gonzalez, and Ion Stoica. 2025. Why do multi-agent LLM systems fail? *arXiv preprint arXiv:2503.13657*.

Kathryn Chaloner and Isabella Verdinelli. 1995. Bayesian experimental design: A review. *Statistical Science*, 10(3):273–304.

Mengzhuo Chen, Junjie Wang, Zhe Liu, Yawen Wang, Haiming Zheng, and Qing Wang. 2026. From failed trajectories to reliable LLM agents: Diagnosing and repairing harness flaws. *arXiv preprint arXiv:2606.06324*.

Peter I. Frazier, Warren B. Powell, and Savas Dayanik. 2008. A knowledge-gradient policy for sequential information collection. *SIAM Journal on Control and Optimization*, 47(5):2410–2439.

Shengran Hu, Cong Lu, and Jeff Clune. 2025. Automated design of agentic systems. In *International Conference on Learning Representations*.

Kevin Jamieson and Ameet Talwalkar. 2016. Non-stochastic best arm identification and hyperparameter optimization. In *Proceedings of the 19th International Conference on Artificial Intelligence and Statistics*, volume 51 of *PMLR*, pages 240–248.

Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. 2024. DSPy: Compiling declarative language model calls into self-improving pipelines. In *International Conference on Learning Representations*.

Tammo Krueger, Danny Panknin, and Mikio Braun. 2015. Fast cross-validation via sequential testing. *Journal of Machine Learning Research*, 16(33):1103–1155.

Jiazheng Li, Emine Yilmaz, Bei Chen, and Dieu-Thu Le. 2026. Towards self-improving error diagnosis in multi-agent systems. *arXiv preprint arXiv:2604.17658*.

Lisha Li, Kevin Jamieson, Giulia DeSalvo, Afshin Rostamizadeh, and Ameet Talwalkar. 2018. Hyperband: A novel bandit-based approach to hyperparameter optimization. *Journal of Machine Learning Research*, 18(185):1–52.

Xiaofeng Lin, Yingxu Wang, Tung Sum Thomas Kwok, Daniel Guo, Sahil Arun Nale, Charles Fleming, and Guang Cheng. 2026. REFLECT: Intervention-supported error attribution for silent failures in LLM agent traces. *arXiv preprint arXiv:2606.09071*.

Jiale Liu, Huajun Xi, Shaokun Zhang, Yifan Zeng, Tianwei Yue, Chi Wang, Jian Kang, Qingyun Wu, and Huazheng Wang. 2026. Who&when pro: Can LLMs really attribute failures in AI agents? *arXiv preprint arXiv:2607.09996*.

Ming Ma, Jue Zhang, Fangkai Yang, Yu Kang, Qingwei Lin, Saravan Rajmohan, and Dongmei Zhang. 2025. DoVer: Intervention-driven auto debugging for LLM multi-agent systems. *arXiv preprint arXiv:2512.06749*.

Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. 2023. Self-refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems*.

Oded Maron and Andrew W. Moore. 1993. Hoeffding races: Accelerating model selection search for classification and function approximation. In *Advances in Neural Information Processing Systems 6*, pages 59–66.

Reid Pryzant, Dan Iter, Jerry Li, Yin Tat Lee, Chenguang Zhu, and Michael Zeng. 2023. Automatic prompt optimization with "gradient descent" and beam search. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*.

Jaineet Shah. 2026. Causal agent replay: Counterfactual attribution for LLM-agent failures. *arXiv preprint arXiv:2606.08275*.

Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*.

Edward K. Smith, Earl T. Barr, Claire Le Goues, and Yuriy Brun. 2015. Is the cure worse than the disease? overfitting in automated program repair. In *Proceedings of the 10th Joint Meeting on Foundations of Software Engineering*.

David J. Spiegelhalter, Laurence S. Freedman, and Patrick R. Blackburn. 1986. Monitoring clinical trials: Conditional or predictive power? *Controlled Clinical Trials*, 7(1):8–17.

Harsh Trivedi, Tushar Khot, Mareike Hartmann, Ruskin Manku, Vinty Dong, Edward Li, Shashank Gupta, Ashish Sabharwal, and Niranjan Balasubramanian. 2024. AppWorld: A controllable world of apps and people for benchmarking interactive coding agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*.

Rakshith Vasudev, Melisa Russak, Dan Bikel, and Waseem Alshikh. 2026. Accurate failure prediction in agents does not imply effective failure prevention. *arXiv preprint arXiv:2602.03338*.

Abraham Wald. 1945. Sequential tests of statistical hypotheses. *The Annals of Mathematical Statistics*, 16(2):117–186.

Mengting Wan, Tara Safavi, Sujay Kumar Jauhar, Yujin Kim, Scott Counts, Jennifer Neville, Siddharth Suri, Chirag Shah, Ryen W. White, Longqi Yang, Reid Andersen, Georg Buscher, Dhruv Joshi, and Nagu Rangan. 2024. TnT-LLM: Text mining at scale with large language models. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*.

Xiaojun Wu, Cehao Yang, Honghao Liu, Xueyuan Lin, Wenjie Zhang, Zhichao Shi, Xuhui Jiang, Chengjin Xu, Jia Li, and Jian Guo. 2026. Bayesian-Agent: Posterior-guided skill evolution across LLM agent harnesses. *arXiv preprint arXiv:2606.08348*.

Peng Xia, Rujun Han, Zifeng Wang, Yanfei Chen, Yufan Zhuang, Yoonho Lee, Chengsong Huang, Han Yu, Zhongying CuiZhu, Yifei Ming, Huaxiu Yao, Burak Gokturk, Tomas Pfister, and Chen-Yu Lee. 2026. RRSI: Regularized recursive self-improvement of agent harnesses. *arXiv preprint arXiv:2609.24972*.

Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc V. Le, Denny Zhou, and Xinyun Chen. 2024. Large language models as optimizers. In *International Conference on Learning Representations*.

Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. 2024. τ-bench: A benchmark for tool-agent-user interaction in real-world domains. *arXiv preprint arXiv:2406.12045*.

Mert Yuksekgonul, Federico Bianchi, Joseph Boen, Sheng Liu, Pan Lu, Zhi Huang, Carlos Guestrin, and James Zou. 2025. Optimizing generative AI by backpropagating language model feedback. *Nature*, 639.

Guibin Zhang, Junhao Wang, Junjie Chen, Wangchunshu Zhou, Kun Wang, and Shuicheng Yan. 2026a. AgenTracer: Who is inducing failure in the LLM agentic systems? In *International Conference on Learning Representations*. ArXiv:2509.03312.

Jenny Zhang, Shengran Hu, Cong Lu, Robert Lange, and Jeff Clune. 2025a. Darwin Gödel machine: Open-ended evolution of self-improving agents. *arXiv preprint arXiv:2505.22954*.

Qizheng Zhang, Changran Hu, Shubhangi Upasani, Boyuan Ma, Fenglu Hong, Vamsidhar Kamanuru, Jay Rainton, Chen Wu, Mengmeng Ji, Hanchen Li, Urmish Thakker, James Zou, and Kunle Olukotun. 2026b. Agentic context engineering: Evolving contexts for self-improving language models. In *International Conference on Learning Representations*.

Shaokun Zhang, Ming Yin, Jieyu Zhang, Jiale Liu, Zhiguang Han, Jingyang Zhang, Beibin Li, Chi Wang, Huazheng Wang, Yiran Chen, and Qingyun Wu. 2025b. Which agent causes task failures and when? on automated failure attribution of LLM multi-agent systems. In *Proceedings of the 42nd International Conference on Machine Learning*. ArXiv:2505.00212.

Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. 2024. ExpeL: LLM agents are experiential learners. In *Proceedings of the AAAI Conference on Artificial Intelligence*.

Kunlun Zhu, Zijia Liu, Bingxuan Li, Muxin Tian, Yingxuan Yang, Jiaxun Zhang, Pengrui Han, Qipeng Xie, Fuyang Cui, Weijia Zhang, Xiaoteng Ma, Xiaodong Yu, Gowtham Ramesh, Jialian Wu, Zicheng Liu, Pan Lu, James Zou, and Jiaxuan You. 2025. Where LLM agents fail and how they can learn from failures. *arXiv preprint arXiv:2509.25370*.

Kunlun Zhu, Xuyan Ye, Zhiguang Han, Yuchen Zhao, Bingxuan Li, Weijia Zhang, Muxin Tian, Xiangru Tang, Pan Lu, James Zou, Jiaxuan You, and Heng Ji.

2026. AgentDebugX: An open-source toolkit for failure observability, attribution, and recovery in LLM agents. *arXiv preprint arXiv:2607.18754*.

A An Accounting Framework for Single-Round Repair

Setting. An agent is assembled from *components* j (prompt sections, tool descriptions, configuration). On a task distribution it fails with rate F; an LLM groups its failures into categories i with shares f_i. For each *cell* (i, j) that the analyst blames, it proposes K candidate patches k that rewrite component j. Let $e_{ij} \in \{0, 1\}$ indicate whether changing j can repair category i at all, and q_k the fraction of category-i failures that patch k repairs if so. Separating e from q matters because a patch that fails to help may be aimed at the wrong component or merely be a bad patch. We call a cell with $e_{ij} = 1$ an *effective edge*. This is an interventional notion: some change to j repairs failures of category i. It need not mean that j was the original cause of those failures.

Net effect. Applying patch k of cell (i, j) changes the success rate by

$$\Delta_k = F f_i e_{ij} q_k - (1-F)\rho_k - c_{\mathrm{a}} - c_{\mathrm{w}}(1-e_{ij}), \quad (2)$$

where ρ_k is the patch's absolute excess failure rate on tasks that used to succeed (regressions, which may hit any category): such a task fails a re-run with probability r_0 unpatched and $r_0 + \rho_k$ patched, so $\rho_k < 0$ is possible; $c_{\mathrm{a}} \geq 0$ is a cost charged for every applied change (review, maintenance) and $c_{\mathrm{w}} \geq 0$ a penalty charged only for a change aimed at a wrong component (risk not captured by ρ_k). Costs are fractions of the failure mass F. The goal is to choose a set of patches, at most one per cell, that maximises $\sum_k \Delta_k$ on held-out tasks; effects of patches to different components are treated as additive, and held-out success of whole patch sets (Table 5) and a leave-one-out ablation (Appendix K) are the only checks. A targeted replay of a category-i failure with patch k succeeds with probability $b_i + (1 - b_i)e_{ij}q_k$, a null replay with probability b_i, and a regression run fails with probability $r_0 + \rho_k$.

Value of verification. Before any evidence, the net effects $\Delta = (\Delta_k)$ of the candidate patches are uncertain, with a prior that reflects the analyst's accuracy and what is known about patch quality and regressions. The *default* applies, in each cell, the patch with the largest $\mathbb{E}[\Delta_k]$ if that is positive. Verification observes outcomes Y and then chooses; its value is $V(Y) = \mathbb{E}\big[\max_S \mathbb{E}[\sum_{k\in S} \Delta_k \mid Y]\big] - \max_S \mathbb{E}[\sum_{k\in S} \Delta_k]$, before the cost of the experiments. Write $x^+ = \max(x, 0)$ and $x^- = \max(-x, 0)$.

Proposition 1 (Value of verification)**.** *For any experiments Y, $V(Y)$ is at most the value of learning Δ exactly, which is the sum over cells of*

$$V_{ij} = \mathbb{E}\big[\max_k \Delta_k^+\big] - \max_k \big(\mathbb{E}\Delta_k\big)^+. \quad (3)$$

With $k^\star$ the default patch of the cell,

$$V_{ij} \;\leq\; \underbrace{\mathbb{E}\big[\max_k \Delta_k^+ - \Delta_{k^\star}^+\big]}_{\text{choosing among patches}} \;+\; \underbrace{\mathbb{E}\big[\Delta_{k^\star}^-\big]}_{\text{avoiding harm}}, \quad (4)$$

with equality when the default applies $k^\star$, and $\mathbb{E}[\Delta_{k^\star}^-] \leq (1 - F)\,\mathbb{E}[\rho_{k^\star}^+] + c_{\mathrm{a}} + c_{\mathrm{w}} P(e_{ij}{=}0)$.

Corollary 1 (Edge-only verification)**.** *Suppose a cell has a single candidate patch whose net effect depends only on e_{ij}: $\Delta = G$ if $e_{ij} = 1$ and $\Delta = -H$ if $e_{ij} = 0$, with $G, H \geq 0$ known. With $\pi = P(e_{ij}{=}1)$, $V_{ij} = \min\big(\pi G, (1 - \pi)H\big)$.*

What survives deployment constraints. Deployment may use a heuristic default D, applies at most one patch per component, which couples cells, combines repairs of one category as $1 - \prod_k (1 - q_k)$ (overstated by at most $f_i \sum_{k<l} q_k q_l$ when added), and cannot push the failure rate on solved tasks below zero ($-\sum_k \rho_k \leq r_0$). Three statements survive. (i) The first sentence of the proposition holds for any set of admissible actions, so $V(Y)$ is at most the value of perfect information over deployable sets. (ii) Over a heuristic default that value is $V + G_D$, where $G_D = \max_S \mathbb{E}[\sum_{k\in S} \Delta_k] - \mathbb{E}[\sum_{k\in D} \Delta_k] \geq 0$ is the *decision-rule gap* that better use of the prior alone would close. (iii) The value of dropping harmful default patches is at most their expected harm, $\sum_{k\in D} \mathbb{E}[(1 - F)\rho_k^+ + c_{\mathrm{a}} + c_{\mathrm{w}}(1 - e_{ij})]$, because dropping a patch from an overlapping set loses at most its marginal repair. What does not survive is the separation over cells: with one patch per component, the choice and discovery terms have no closed form, and we compute them in sequence (harm, then choice on the default's components, then any component) with a greedy optimiser, which gives a lower bound on the constrained value of perfect information under a given harm model. The three sources of value are *avoiding harm*, which needs

regressions or change costs because replay success alone cannot create it; *choosing among patches* of different quality; and *discovery* of edges the analyst missed, the whole term $\mathbb{E}[\max_k \Delta_k^+]$ in cells the default leaves alone. The corollary fails when quality or regressions are uncertain given the edge ($\pi = 1$ with $\Delta = +0.10$ or -0.05 equally likely gives $V = 0.025$).

Proposition 2 (Cost of evidence)**.** *Under Wald's approximations, a sequential test that decides e_{ij} from targeted replays with error δ needs on average about* $\log \frac{1-\delta}{\delta} \,/\, \mathrm{KL}\big(b+(1-b)q \,\|\, b\big)$ *replays when the edge is real. Detecting a regression of size ρ against a base rate r_0 with a one-sided level-α test and power $1-\beta$ needs about $(z_\alpha + z_\beta)^2 r_0(1-r_0)/\rho^2$ regression runs.*

Two consequences follow. First, targeted replays are uninformative about the harm term, because they only see the target category: a patch can pass every replay and still break other tasks. Second, small regressions are expensive to see. With $r_0 = 0.1$, a regression of $\rho = 0.25$ is detected with 90% power by 20 runs, whereas $\rho = 0.03$ needs about 850. Proposition 1 is the standard value-of-information argument (Chaloner and Verdinelli, 1995); what is new is its decomposition for patches and its evaluation on measured banks.

Where could verification add value? Table 4 evaluates Proposition 1 on deployable sets. Every value in it is conditional on a model of the harm ρ_k that the regression runs cannot pin down per patch, so we read only its qualitative pattern. *Defaults.* The Bayes default, built from the net-effect method's priors, scores 1.49 oracles on retail against 1.43 for applying the attributed patches, and 0.53 against 0.66 on airline; $G_D \geq 0$ holds only under the prior, so the negative realised gap on airline means the edge prior is too generous there. *Value.* Under the binomial model, perfect information is worth 1.18 oracles over the heuristic on retail (90% interval $[0.10, 2.99]$, about 8 points of absolute success) and 0.86 on airline ($[0.24, 1.55]$, about 10 points), split into 0.41, 0.57 and 0.20 (retail) and 0.20, 0.36 and 0.29 (airline) for harm, choice and discovery. Under the earlier normal models the retail value ranges from 0.87 (complete pooling) to 4.65 (no pooling) and airline's from 0.69 to 1.15; dropping the support constraint, so that the uncertain benefits of every unblamed patch add up without limit, gives 9.5 on retail (Table 16). Only coarse features are stable: all three terms contribute, and most of the value needs evidence about patches the default applies (harm and choice) rather than discovery of missed edges. Proposition 1 names the quantities that could make verification pay, but it cannot decide whether to verify a new diagnosis: its most influential input, each patch's harm, takes hundreds of regression runs to measure (Proposition 2), and its values move several-fold with the harm model.

	Default rule	Value	Gap	Harm	Choice	Discov.	Total
Retail	Apply, $c = 0$	1.43	0.06	0.41	0.57	0.20	1.18
	Bayes, $c = 0$	1.49	–	0.80	0.32	0.00	1.12
	Apply, $c_w = 0.02$	1.10	−0.19	0.54	0.56	0.03	1.13
	normal, partial pooling	0.77		0.87	0.98	0.17	2.02
	normal, τ=0	1.12		0.23	0.22	0.42	0.87
	no pooling	−1.82		3.30	1.21	0.13	4.65
	tasks resampled	−0.52		2.28	0.90	0.13	3.31
	per cell	0.64		1.39	0.60	0.16	2.15
Airline	Apply, $c = 0$	0.66	−0.13	0.20	0.36	0.29	0.86
	Bayes, $c = 0$	0.53	–	0.55	0.44	0.00	0.98
	Apply, $c_w = 0.02$	0.54	0.02	0.27	0.36	0.11	0.74
	normal, partial pooling	0.73		0.04	0.25	0.40	0.69
	normal, τ=0	0.73		0.04	0.25	0.40	0.69
	no pooling	0.39		0.91	0.10	0.15	1.15
	tasks resampled	0.63		0.39	0.25	0.23	0.87
	per cell	0.86		0.15	0.11	0.30	0.56

Table 4: Value of perfect verification over two no-evidence defaults, deployable sets, posterior means under the task-level binomial harm model (Appendix J) as fractions of the repair-only oracle. *Gap*: Bayes default minus Apply attributed (G_D with the sign reversed; negative when the heuristic does better). Harm, choice and discovery are added in that order: dropping default patches with negative net effect, then the best cell and patch on the default's components, then any other component. Indented rows give the value over Apply at $c = 0$ under the normal-approximation harm models of an earlier analysis (partial, complete and no pooling, tasks resampled before fitting) and with the additive per-cell form of Proposition 1 instead of deployable sets. A value of 1 is 0.070 (retail) or 0.114 (airline) in absolute success.

B Study 1: Measurements and Net Value

Setup details. Each training episode runs clean or with one injected fault (retail: a policy section allowing gift-card refunds, a cancellation rule without the reason check, a product-tool description that confuses item and product ids, a tight step limit; airline: a wrong bag allowance, a 7-day instead of 24-hour cancellation window, a missing basic-economy restriction, proactive compensation, a tight step limit). The 36 cells per domain are the blamed cells in order of blame plus random unblamed ones; regression runs were extended from 4 to 20 for the 121 patches that any policy accepted, and an unpatched baseline of 10 runs on each of the 111 regression tasks gives task-matched estimates

of harm. Every experiment is priced in measured tokens relative to a full replay. *Apply attributed* applies, for each category, the first patch of the most-blamed component and of any component with at least 40% of the blame; *Replay-each* accepts a hypothesis on two successes in three replays, as do-then-verify methods do (Ma et al., 2025); two HarnessFix-style gates require regression runs to break nothing, per patch or on the whole bundle; uncertainty sampling runs with and without cheap checks; four variants of CARVE (Appendix V) and a net-effect allocation that charges regressions complete the list (Appendix D). Deployed sets come from five seeds, reduced to one patch per component; the retail held-out episodes are 38 clean and 82 with an injected fault, the airline ones 25 clean, and five policies were deployed on airline. On clean episodes an exploratory comparison puts the bundle gate 4 points ahead, from seed sets identical to the unverified one; on retail's 38 clean episodes the unverified set gains $+5.3$ points $[-5.1, 15.6]$, and on 158 clean episodes, 120 of them new, $+4$ $[-3, 10]$ (Appendix K). Predicted from their replays net of spurious recovery, the deployed sets' gains correlate weakly with the observed ones ($r = 0.23$ over 47 retail sets, -0.49 over 14 airline sets).

Tables 6 and 7 give the measurements and the $\mathrm{Net}(N)$ values discussed in §4.

Does the evidence predict deployment? Replays measure repair from the failing step, whereas deployed patches act from the first step and on every task. For every distinct deployed set we predict its gain over the unpatched agent from its patches' replays as $\hat{F} \sum_i f_i \big[1 - \prod_k (1 - \hat{q}_k)\big]$, with $\hat{F}$ the held-out failure rate, f_i the category shares and $\hat{q}_k = (p_k - b_i)/(1 - b_i)$ the replay success p_k net of the category's spurious recovery b_i. Figure 2 compares the predictions with the observed gains. They correlate weakly, at $r = 0.23$ over 47 distinct retail sets and -0.49 over 14 airline sets; predictions from the judge checks, calibrated to replay success, do no better ($r = 0.14$ and -0.05).

The ranking test has little power, for a reason that matters beyond this study. Replay predictions differ between sets by a standard deviation of 1.0 point on retail and 1.7 on airline, whereas a set's held-out success measured on 120 (60) episodes has a run-to-run standard error of 2.6 (3.8) points; the observed gains spread by 2.8 (3.7), no more than that noise. A perfect predictor of differences this small would be indistinguishable from a useless one. Testing whether evidence predicts deployment needs candidates that differ by more than the deployment noise and more precise deployment measurements; Study 2 is designed for both.

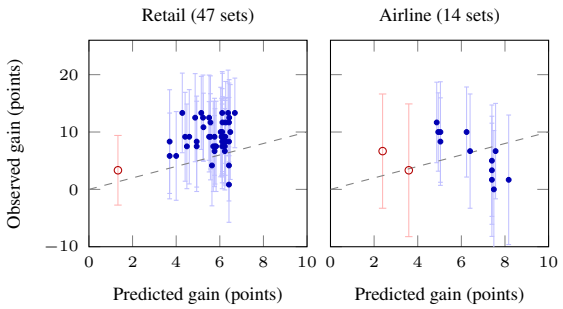


Figure 2: Each distinct deployed patch set's gain over the unpatched agent as predicted from its bank replays (net of spurious recovery) against its observed held-out gain, with 90% intervals. Filled: sets containing the step-limit patch; open: sets without it. Dashed: $y = x$. Correlations are 0.23 (retail) and -0.49 (airline).

C Proofs

Proposition 1. For any signal Y and any action set, $\max_S \mathbb{E}[U(S) \mid Y] \le \mathbb{E}[\max_S U(S) \mid Y]$, so taking expectations gives $V(Y) \le \mathbb{E}[\max_S U(S)] - \max_S \mathbb{E}[U(S)]$, the value of perfect information; this is the standard argument and needs no assumption on how Y relates to Δ. With $U(S) = \sum_{k \in S} \Delta_k$ and at most one patch per cell, both maxima separate over cells: $\max_S U(S) = \sum_{ij} \max_k \Delta_k^+$ and $\max_S \mathbb{E}[U(S)] = \sum_{ij} \max_k (\mathbb{E}\Delta_k)^+$, which gives V_{ij}; no independence between cells is needed. For Eq. 4, let $k^\star$ maximise $\mathbb{E}\Delta_k$. If $\mathbb{E}\Delta_{k^\star} \ge 0$, then $V_{ij} = \mathbb{E}[\max_k \Delta_k^+] - \mathbb{E}[\Delta_{k^\star}^+] + \mathbb{E}[\Delta_{k^\star}^-]$, which is Eq. 4 with equality. Otherwise $V_{ij} = \mathbb{E}[\max_k \Delta_k^+] = \mathbb{E}[\max_k \Delta_k^+ - \Delta_{k^\star}^+] + \mathbb{E}[\Delta_{k^\star}^+]$, and $\mathbb{E}[\Delta_{k^\star}^+] < \mathbb{E}[\Delta_{k^\star}^-]$ because $\mathbb{E}\Delta_{k^\star} < 0$. Finally, the repair term $F f_i e_{ij} q_k$ is non-negative, so $\Delta_k^- \le (1 - F)\rho_k^+ + c_\mathrm{a} + c_\mathrm{w}(1 - e_{ij})$; take expectations.

Heuristic defaults and constraints. For a default D, $\mathbb{E}[\max_S U(S)] - \mathbb{E}[U(D)] = \big(\mathbb{E}[\max_S U(S)] - \max_S \mathbb{E}[U(S)]\big) + \big(\max_S \mathbb{E}[U(S)] - \mathbb{E}[U(D)]\big)$, and the second bracket is $G_D \ge 0$ by definition of the maximum. Restricting S to deployable sets leaves the argument unchanged. For repairs in one category, the Bonferroni inequality gives $1 - \prod_k (1 - q_k) \ge \sum_k q_k - \sum_{k<l} q_k q_l$, which bounds the overstatement of the additive form. The

Patch set (budget 10)	Retail (40 tasks × 3) All	Clean	−Apply [90%]	#p	C	Airline (20 tasks × 3) All	Clean	−Apply [90%]	#p	C
Unpatched	75.8	81.6		0.0		68.3	76.0		0.0	
Apply attributed	87.5	86.8	–	7.0	0	75.0	76.0	–	5.0	0
Replay-each	83.5	86.8	−4.0 [−9.8, 1.8]	6.0	10					
HarnessFix-style, per patch	83.2	90.0	−4.3 [−12.8, 4.2]	0.8	9	71.0	76.8	−4.0 [−11.9, 3.9]	1.0	8
HarnessFix-style, bundle	84.8	86.8	−2.7 [−5.0, −0.4]	5.6	6	71.7	76.8	−3.3 [−9.3, 2.6]	3.0	6
Uncertainty sampling	85.8	86.3	−1.7 [−6.9, 3.5]	5.8	10					
Uncertainty + checks	84.7	82.1	−2.8 [−8.4, 2.7]	5.8	10					
CARVE, replay only	87.8	86.8	0.3 [−5.3, 6.0]	6.0	10					
CARVE	83.8	80.5	−3.7 [−9.9, 2.6]	5.2	10	74.3	72.0	−0.7 [−8.8, 7.5]	4.2	10
CARVE, measured check	84.8	86.3	−2.7 [−7.6, 2.3]	6.0	10					
CARVE, robust check	86.3	83.7	−1.2 [−6.3, 4.0]	6.0	9					
Net-effect	83.2	83.2	−4.3 [−9.9, 1.3]	3.4	10	74.7	76.0	−0.3 [−9.1, 8.5]	3.6	10

Table 5: Held-out success (%) of the patch set each policy selects with a budget of 10 episodes, mean over five policy seeds. *All*: every held-out episode (retail: 82 of 120 with an injected fault; airline: 35 of 60). *Clean*: episodes without an injected fault (38 and 25). −Apply: paired difference from Apply attributed over all episodes, in points, with a task-clustered 90% interval. #p: patches deployed; C: verification episodes spent.

	ShopDesk	Retail	Airline
Analyst accuracy (audited)	0.72	0.41	0.29
Effective edges among blamed cells	–	0.18	0.33
Judge check: cost / replay	0.72	0.28	0.23
sens. / FPR, vs. single replay	0.57 / 0.21	0.37 / 0.11	0.41 / 0.13
sens. / FPR, vs. patch replay rate	–	0.49 / 0.05	0.55 / 0.07
efficiency ratio vs. replays	–	0.88 [0.46, 1.53]	1.25 [0.19, 3.95]
Spurious recovery b (range)	–	0.23 (0–0.50)	0.20 (0–0.50)
Repair q of effective edges (median)	–	0.50	0.41
Replays to decide an edge	–	5	7
Unpatched failure on solved tasks	–	0.15	0.12
Mean harm $\bar{\rho}$ (points)	–	−1.0 [−3.4, 1.5]	−0.6 [−3.3, 1.9]
Spread of ρ_k (points)	–	2.6 [0.2, 5.3]	2.1 [0.2, 4.5]
Patches harmful / helpful (95%)	–	2 / 5	3 / 4

Table 6: What the evidence costs and shows. The efficiency ratio is the judge's break-even cost ratio divided by its actual cost (Proposition 4); values above 1 favour judge checks (Appendix I). Harm rows: excess failure rate of a patch on previously solved tasks under the task-level binomial model (Appendix J), posterior mean and 90% interval; the last row counts patches whose 20 regression runs differ significantly from the task-matched baseline.

Verified set	Δp [90%]	Δc_{run}	C	Net(1000) [90%], episodes $v = 1$	$v = 10$
Retail					
Replay-each	−4.0 [−9.8, 1.8]	−0.06	10	12 [−217, 242]	−348 [−966, 271]
HarnessFix-style, per patch	−4.3 [−12.8, 4.2]	−0.03	9	−18 [−204, 167]	−408 [−1273, 456]
HarnessFix-style, bundle	−2.7 [−5.0, −0.4]	−0.00	6	−31 [−244, 183]	−271 [−583, 42]
Uncertainty sampling	−1.7 [−6.9, 3.5]	−0.07	10	41 [−189, 271]	−109 [−676, 458]
Uncertainty + checks	−2.8 [−8.4, 2.7]	0.02	10	−56 [−285, 173]	−311 [−907, 286]
CARVE, replay only	0.3 [−5.3, 6.0]	−0.02	10	10 [−220, 240]	40 [−568, 648]
CARVE	−3.7 [−9.9, 2.6]	−0.04	10	−8 [−227, 210]	−338 [−999, 322]
CARVE, measured check	−2.7 [−7.6, 2.3]	−0.01	10	−24 [−254, 206]	−264 [−810, 282]
CARVE, robust check	−1.2 [−6.3, 4.0]	0.02	9	−37 [−263, 188]	−142 [−704, 420]
Net-effect	−4.3 [−9.9, 1.3]	0.05	10	−100 [−307, 106]	−490 [−1083, 103]
Airline					
HarnessFix-style, per patch	−4.0 [−11.9, 3.9]	−0.04	8	−7 [−226, 212]	−367 [−1185, 451]
HarnessFix-style, bundle	−3.3 [−9.3, 2.6]	−0.06	6	23 [−209, 254]	−277 [−911, 356]
CARVE	−0.7 [−8.8, 7.5]	−0.11	10	96 [−156, 349]	36 [−814, 887]
Net-effect	−0.3 [−9.1, 8.5]	−0.08	10	68 [−194, 330]	38 [−875, 951]

Table 7: $\mathrm{Net}(N)$ of each verified set against Apply attributed (Eq. 1), at $N = 1000$ deployed episodes, with no change cost and a success worth v episodes of running cost. Δp: held-out difference in points (Table 5); Δc_{run}: estimated change in running cost per deployed episode, from the tokens of patched agents on solved tasks and of truncated step-limit episodes; C: verification episodes. Intervals combine the uncertainty of Δp and of Δc_{run} (standard error about 0.1), as normal.

bound on $-\sum_k \rho_k$ holds because the patched failure rate $r_0 + \sum_k \rho_k$ on solved tasks is non-negative under additivity. For (iii), removing patch k from a set S changes its value by $-\big[\mathrm{rep}(S) - \mathrm{rep}(S \setminus k)\big] + (1-F)\big[\bar{\rho}(S) - \bar{\rho}(S \setminus k)\big] + c_{\mathrm{a}} + c_{\mathrm{w}}(1 - e_{ij})$, where rep is the product-form repair and $\bar{\rho}$ the capped sum of harms. Repair is monotone in S, so the first bracket is non-negative, and capping $\sum \rho$ from below changes it by at most ρ_k^+ when k is removed; summing over the dropped patches and taking expectations gives the bound. The greedy optimiser keeps, per component, the cell and patch with the largest positive net effect, which is a feasible deployable set, so the value it attains is a lower bound on the constrained optimum.

Corollary 1. With one patch, $V = \mathbb{E}[\Delta^+] - (\mathbb{E}\Delta)^+ = \pi G - (\pi G - (1-\pi)H)^+ = \min(\pi G, (1-\pi)H)$. The assumption that Δ is a function of e_{ij} is what makes the edge state sufficient: if q_k or ρ_k is uncertain given e_{ij}, learning only e_{ij} leaves value on the table and the general expression applies.

Proposition 2. The first part is Wald's approximation for a sequential probability ratio test between $p_1 = b + (1-b)q$ and $p_0 = b$: with thresholds $\pm \log \frac{1-\delta}{\delta}$, neglecting overshoot, the expected sample size under p_1 is the threshold divided by the per-observation drift $\mathrm{KL}(p_1 \| p_0)$. The second is the normal-approximation sample size of a one-sided test of a binomial proportion, evaluated with the

variance at r_0.

D Verification Policies on the Intervention Banks

Summary. The banks give the same answer as the deployment comparison without new runs. In that comparison the policy seed added no detectable variance: the variance of the seed means of the verified sets' differences from Apply attributed is 0.0007, against 0.0019 expected from run-to-run noise alone. Under the task-level harm model, applying the attributed patches is worth 1.43 (retail) and 0.66 (airline) of the repair-only oracle, and the verifying policies 0.39–1.56 at budget 40, beating it in 19–57% of posterior draws; scored on bank halves that share no trajectory with the policies' evidence, they beat it in 22–47%. Policies that use the analyst's blame as an edge prior used an analyst accuracy of 0.7; with the audited 0.41 and 0.29, or half of these, every such policy is worth less than with 0.7, on both domains (Table 10).

Policies. *Net-effect allocation.* A spike-and-slab model per cell takes its edge prior from the attribution counts (Eq. 7, Appendix V), with grid posteriors for q_k and Beta posteriors for each b_i and each patch's regression failure rate r_k. It estimates Eq. 2 as $P(e_{ij}{=}1 \mid \mathcal{D})\, F f_i\, \mathbb{E}[q_k \mid \mathcal{D}] - (1 - F)\max(0, \mathbb{E}[r_k \mid \mathcal{D}] - r_0)/(1 - r_0) - c$, a stricter objective than Eq. 2: it charges c on every applied change, never credits a patch for lowering failures on solved tasks, and scales harm by $1/(1 - r_0)$; Table 14 ablates the last two. It repeatedly runs the targeted replay, null replay or regression run with the largest n-step knowledge gradient per unit cost, $n \in \{1, 2, 4, 8\}$ (Frazier et al., 2008), and applies every patch with positive posterior net effect. *Apply attributed* uses no evidence: for each category it applies the first patch of the most-blamed component and of any component with at least 40% of the blame. *Replay-each* tests hypotheses in order of blame with three replays and accepts on two successes, as in do-then-verify methods (Ma et al., 2025). *HarnessFix-style, per patch* adds two regression runs that must break nothing; these counts are ours. *HarnessFix-style, bundle gate* is closer to the original, which validates a whole bundle of repairs on held-out tasks and promotes it if it solves at least one more task and breaks at most two (Chen et al., 2026); we apply the *Apply attributed* bundle, draw validation episodes from the bank, and allow three attempts. *Uncertainty sampling* replays the cell with the largest frequency-weighted posterior uncertainty, with or without three cheap checks per patch first. CARVE and its variants are described in Appendix V.

Scoring. Policies run offline by drawing recorded outcomes from the bank, with replacement (without, as a check), with costs in measured tokens and a full replay as the unit; resampling seeds measure policy variability given the bank, not new evidence. *Adjudication*: a cell is an effective edge if its best patch's replay success exceeds the category's null rate by at least 0.2 (one-sided Fisher $p < 0.05$), giving 4 effective edges on retail and 7 on airline; choosing the best of three patches on the same replays makes this label optimistic, so we also score policies on bank halves they did not see. *Harm*: every patch's ρ_k is drawn from the task-level binomial model of Appendix J, and edges and repair rates from a bootstrap of the replays. *Deployable sets*: every accepted set is reduced to one patch per component, keeping the cell of the most frequent category, as on held-out tasks. Values are fractions of the *repair-only oracle*, the failure mass repaired by the effective edges' best patches with no cost, a fixed yardstick worth 0.070 (retail) and 0.114 (airline) in absolute success. The unpatched baseline (10 runs on each of 111 regression tasks) was collected about 12 hours after the regression runs; unpatched held-out success did not change over that time (0.816 vs. 0.817 on clean retail episodes). *Protocol*: retail was the development domain for every method, including the variants in Appendix L; airline was evaluated once for the original policies and once, later, for the net-effect method, which was frozen on retail beforehand.

Results. Table 8 reports each policy's posterior expected value. On retail, applying the attributed patches is worth 1.43 oracles (10.1 points of success) and is the best policy at budget 10; at 40 and 80, CARVE with a robust check is ahead (1.56 and 1.59) but beats applying everything in only 57% of draws at budget 40; across budgets, the other verifying policies are worth 0.55–1.49. On airline, applying everything scores 0.66 and no verifying policy exceeds 0.73. The 90% intervals span about three oracles on retail (applying everything: $[-0.52, 2.67]$) and one and a half on airline. On source-split halves every value falls, because each half supports fewer effective edges: applying everything scores 0.96 (retail) and 0.23 (airline), and verifying policies beat it in 22–42% (retail)

	τ^2 retail (development)						τ^2 airline					
	full bank			source-split halves			full bank			source-split halves		
Budget (episodes)	10	40	80	10	40	80	10	40	80	10	40	80
Apply attributed	**1.43**	1.43	1.43	**0.96**	0.96	0.96	0.66	**0.66**	0.66	0.23	0.23	0.23
Bayes default (Prop. 1)	1.49	1.49	1.49	–	–	–	0.53	0.53	0.53	–	–	–
Replay-each	1.37	1.26	1.35	0.85	0.81	0.83	0.60	0.53	0.54	0.18	0.15	0.11
HarnessFix-style, per patch	0.55	1.01	1.28	0.55	0.87	0.97	0.19	0.45	0.53	0.09	**0.24**	0.22
HarnessFix-style, bundle	0.68	1.16	1.31	0.38	0.56	0.67	0.36	0.57	0.60	0.12	0.17	0.17
Uncertainty sampling	1.27	1.39	1.32	0.84	0.94	0.94	0.59	0.65	0.67	0.10	0.22	**0.24**
Uncertainty + checks	1.36	1.33	1.35	0.73	0.87	0.98	0.54	0.58	0.65	0.04	0.14	0.18
CARVE, replay only	1.23	1.41	1.49	0.87	0.91	0.98	**0.71**	0.65	0.70	0.24	0.24	0.24
CARVE	1.30	1.12	0.93	0.61	0.54	0.63	0.63	0.39	0.25	0.11	0.08	0.09
CARVE, measured check	1.23	1.41	1.49	0.82	0.93	1.01	0.71	0.65	0.71	0.24	0.23	0.23
CARVE, robust check	1.35	**1.56**	**1.59**	0.84	**0.96**	**1.02**	0.68	0.64	**0.73**	**0.26**	0.24	0.23
Net-effect (ours)	1.02	1.09	1.29	0.82	0.87	0.97	0.54	0.60	0.64	0.14	0.18	0.19

Table 8: Posterior expected value of each policy's deployable patch set on the τ^2 banks at $c = 0$, under the task-level binomial harm model, as a fraction of the repair-only oracle; bold is the best per column. *Full bank*: policies and scoring use the same bank; 300 posterior draws, each also bootstrapping the replays, averaged over 40 policy seeds. *Source-split halves*: the bank is split into halves that share no source trajectory and no regression task; policies run on one half, with their inputs (experiment costs, spurious recovery, judge calibration) from that half and the audited analyst accuracy, and are scored on the other (5 halvings $\times$ 100 draws, 20 seeds). The Bayes default is Proposition 1's no-evidence rule under the net-effect method's priors. Values above 1 arise when patches are estimated to reduce failures on previously solved tasks.

and 35–47% (airline) of draws at budget 40. With $c_w = 0.02$ per wrong change (Table 9) applying everything falls to 1.10 and 0.54 on the full banks; the leaders change with budget and split, and none leads everywhere. On the banks, as on held-out tasks, no verification policy reliably beats applying the attributed patches.

Priors. Every policy with an edge prior used an analyst accuracy of $\hat{r} = 0.7$, well above the audited 0.41 (retail) and 0.29 (airline). Table 10 reruns their budget-40 decisions with the audited accuracy and with half of it. Every such policy is worth less than with 0.7 on both domains, by 0.11 to 0.65 oracles on retail and 0.04 to 0.25 on airline at the audited accuracy, and halving it again changes values by at most 0.30 more. With the audited or the weaker accuracy, no verifying policy is worth more than applying the attributed patches.

E Significance-Thresholded Scoring

Charging only regressions significant against the paired-trial r_0 reproduces the picture of a scoring that sets uncertain harm to zero: applying everything recovers 84% and 60% of the oracle and no verifying policy reaches it. With $c_w = 0.02$ per wrong change, the per-patch HarnessFix-style rule (retail, budgets 10–40; airline, 40) and the net-effect method (retail, 80; airline, 80, tied with uncertainty sampling with checks) lead, and CARVE declines with budget on airline (0.30→0.19). Appendix G varies c for both cost types; the winner changes with c.

F Robustness at Budget 40

G Change-Cost Sweep

Table 13 varies the change cost. The best policy changes with c: applying the attributed patches is best on retail at $c = 0$ (on airline, CARVE told $c = 0$ edges it, 0.71 against 0.60), replay-each at $c = 0.01$ on retail, the per-patch HarnessFix-style rule from $c = 0.01$ to 0.02 on airline and at 0.02 on retail, and the net-effect method at $c = 0.05$ on retail under both cost types and on airline per applied change. Charging every applied change rather than only wrong ones lowers every policy but changes the winner in only one setting (airline, $c = 0.05$). Told $c = 0$, the policies that use a cost internally accept patches with significant regressions, which is why some of them are negative in the first column.

H Policy Inputs and Regression Sampling

Attribution, taxonomy induction, labelling and candidate patches are shared by all policies and not charged to any budget. Every policy that uses an edge prior uses Eq. 7 with analyst accuracy $\hat{r} = 0.7$, not the audited 0.41 and 0.29 of Table 6; Table 10

	τ^2 retail (development)						τ^2 airline					
	full bank			source-split halves			full bank			source-split halves		
Budget (episodes)	10	40	80	10	40	80	10	40	80	10	40	80
Apply attributed	**1.10**	1.10	1.10	0.61	0.61	0.61	0.54	0.54	0.54	0.06	0.06	0.06
Bayes default (Prop. 1)	0.91	0.91	0.91	–	–	–	0.56	0.56	0.56	–	–	–
Replay-each	1.10	1.03	1.13	0.54	0.52	0.51	0.49	0.45	0.44	0.02	0.01	−0.06
HarnessFix-style, per patch	0.55	0.93	1.10	0.55	0.80	0.78	0.18	0.42	0.48	0.06	**0.18**	0.10
HarnessFix-style, bundle	0.52	0.89	1.02	0.22	0.30	0.40	0.29	0.46	0.48	0.02	0.01	0.02
Uncertainty sampling	1.00	1.20	1.13	**0.79**	**0.82**	0.74	0.53	**0.59**	0.61	0.03	0.13	**0.12**
Uncertainty + checks	1.10	1.19	1.21	0.69	0.79	**0.85**	0.47	0.53	0.60	−0.02	0.06	0.07
CARVE, replay only	0.92	1.14	1.15	0.75	0.71	0.67	0.62	0.56	0.60	0.13	0.11	0.07
CARVE	1.08	1.02	0.89	0.56	0.44	0.49	0.55	0.33	0.21	0.04	−0.02	−0.03
CARVE, measured check	0.92	1.14	1.15	0.70	0.73	0.69	**0.63**	0.56	0.62	0.14	0.10	0.06
CARVE, robust check	1.04	**1.27**	**1.25**	0.71	0.75	0.70	0.61	0.56	**0.64**	**0.16**	0.11	0.06
Net-effect (ours)	0.87	0.98	1.15	0.77	0.78	0.83	0.49	0.55	0.58	0.08	0.08	0.06

Table 9: As Table 8, with a cost of $c_{\mathrm{w}} = 0.02$ of the failure mass charged per patch aimed at a wrong edge.

	Retail			Airline		
Analyst accuracy $\hat{r}$	0.7	0.41	0.20	0.7	0.29	0.15
Apply attributed	1.43	1.43	1.43	0.66	0.66	0.66
Uncertainty sampling	1.39	1.02	0.95	0.65	0.40	0.35
Uncertainty + checks	1.33	0.99	1.00	0.58	0.35	0.26
CARVE, replay only	1.41	1.22	1.10	0.65	0.57	0.49
CARVE	1.12	0.47	0.17	0.39	0.17	0.12
CARVE, measured check	1.41	1.22	1.10	0.65	0.61	0.50
CARVE, robust check	1.56	1.27	1.14	0.64	0.53	0.47
Net-effect	1.09	0.98	0.99	0.60	0.42	0.40

Table 10: Policies that use the edge prior of Eq. 7, with the analyst accuracy used in the main analysis (0.7), the audited accuracy and half of it; posterior value of deployable sets at budget 40 and $c = 0$ on the full bank (binomial harm model). Apply attributed uses no prior.

rescores them with the audited accuracy and half of it. Three further inputs come from the bank itself and are also uncharged: the cost of each kind of experiment (mean tokens), the mean spurious recovery $\hat{b}$, which CARVE and uncertainty sampling plug in and the net-effect method uses only as a prior mean for its per-category b_i, and, for the two CARVE variants that use them, the judge's measured sensitivity and false-positive rate. The re-run failure rate r_0 comes from paired re-runs of training tasks. *Regression tasks* are drawn uniformly, with replacement, from the clean training episodes the agent solved, so they follow the training-task distribution of the same domain, not the held-out tasks, which also include fault-injected episodes. The extension to 20 runs covered every patch an original policy accepted; 98–100% of the patches that any policy, including the later net-effect method, deploys at budget 40 have 20 runs, and a policy asking for more runs of a 4-run patch resamples those 4.

I Judge Configurations

The three new configurations regenerate the failing step once per pair and judge it with each prompt; the short prompt shows the task, the diagnosed root cause, the tool results or user message just before the failing step, and the old and new actions. Pairs are stratified on the paired replay's outcome; because a check and its paired replay are separate regenerations, the check is independent of that outcome given the patch, so the stratification does not bias the patch-level fit. The short-prompt judges agree with the bank judge on 64–76% of pairs. The experiment used about 310 episode-equivalents of tokens. *Why the two definitions differ.* Conditioning on a single replay's outcome attenuates both quantities towards the judge's overall pass rate, because many failed replays come from patches that do repair some failures and many successful ones from spurious recovery; the patch-level fit removes this noise, which raises the bank judge's break-even ratio from about 0.1 to 0.25 (retail) and 0.29 (airline). Across categories, using each category's own spurious recovery and pairs (≥ 20), the single-replay ratio of the bank judge ranges from 0 to 2.0 on retail and 0.04 to 1.5 on airline. *Latency* was not measured; counted in sequential model calls, a check is two (regeneration and judge) and a replay from the failing step a median of 6 (retail) or 7 (airline), interquartile range 4–11. *Money*: we cannot state prices for the models we used that will remain accurate, so the table reports token cost with the analyst model priced at one, two and four times the agent model.

	τ^2 retail (development)						τ^2 airline (held out)					
	significant harm			+ c_w=0.02			significant harm			+ c_w=0.02		
Budget (episodes)	10	40	80	10	40	80	10	40	80	10	40	80
Apply attributed	**0.84**	**0.84**	**0.84**	0.16	0.16	0.16	**0.60**	**0.60**	**0.60**	0.15	0.15	0.15
Replay-each	0.78	0.74	0.71	0.14	0.33	0.36	0.43	0.09	0.10	0.07	−0.12	−0.08
HarnessFix-style, per patch	0.52	0.51	0.52	**0.52**	**0.42**	0.30	0.18	0.40	0.43	0.18	**0.37**	0.31
HarnessFix-style, bundle	0.11	0.21	0.26	−0.21	−0.31	−0.31	0.34	0.53	0.52	0.07	0.12	0.11
Uncertainty sampling	0.42	0.50	0.31	0.01	0.23	0.01	0.56	0.50	0.51	0.28	0.25	0.25
Uncertainty + checks	0.14	0.47	0.48	−0.25	0.25	0.24	0.55	0.50	0.53	0.27	0.29	0.36
CARVE, replay only	0.01	0.18	0.14	−0.48	−0.26	−0.39	0.22	0.35	0.48	−0.16	−0.02	0.12
CARVE	0.33	0.32	0.52	−0.01	0.19	0.47	0.55	0.34	0.25	**0.30**	0.23	0.19
CARVE, measured check	0.01	0.18	0.14	−0.48	−0.26	−0.39	0.25	0.43	0.54	−0.09	0.07	0.20
CARVE, robust check	0.34	−0.03	0.02	−0.15	−0.49	−0.48	0.22	0.34	0.48	−0.12	−0.01	0.14
Net-effect (ours)	0.24	0.52	0.66	0.09	0.41	**0.52**	0.26	0.39	0.46	0.14	0.31	**0.36**

Table 11: Sensitivity analysis: as Table 8, but accepted patches are charged only their significant measured regressions (one-sided binomial $p < 0.05$ against r_0), mean over 200 resampling seeds of the original bank. This scoring sets uncertain harm to zero and so favours policies that apply many patches.

		Posterior, c_w=0.02		Sensitivity: significant harm only						
	Policy (budget 40)	Value [90%]	$P(>$ apply)	c_w	c_a	Split	Natural	No step limit	One per comp.	No repl.
Retail	Apply attributed	0.44 [−1.68, 2.40]	–	0.16	0.01	−0.46	−1.25	−0.61	0.47	0.16
	HarnessFix-style, per patch	0.70 [0.04, 1.38]	0.57	0.42	0.34	0.12	−0.82	−0.27	0.42	0.54
	HarnessFix-style, bundle	0.39 [−1.11, 1.89]	0.44	−0.31	−0.42	−0.80	−1.99	−0.83	−0.08	−0.45
	Uncertainty sampling	0.87 [−0.50, 2.15]	0.69	0.23	0.09	−0.13	−1.10	−0.64	0.29	0.28
	CARVE	0.75 [−0.09, 1.65]	0.59	0.19	0.07	0.04	−0.89	−0.25	0.20	0.51
	Net-effect (ours)	0.80 [0.06, 1.52]	0.60	0.41	0.32	0.23	−0.86	−0.35	0.41	0.52
Airline	Apply attributed	0.61 [−0.13, 1.38]	–	0.15	−0.08	−0.41	−0.07	−0.03	0.46	0.15
	HarnessFix-style, per patch	0.44 [0.06, 0.86]	0.26	0.37	0.25	0.17	0.28	0.20	0.38	0.38
	HarnessFix-style, bundle	0.55 [−0.12, 1.25]	0.15	0.12	−0.09	−0.30	−0.09	−0.04	0.39	0.10
	Uncertainty sampling	0.66 [0.06, 1.32]	0.66	0.25	0.01	0.04	0.05	0.05	0.41	0.26
	CARVE	0.36 [−0.09, 0.83]	0.16	0.23	0.08	−0.03	0.24	0.22	0.25	0.23
	Net-effect (ours)	0.65 [0.06, 1.26]	0.58	0.31	0.14	0.17	0.23	0.13	0.35	0.26

Table 12: Uncertainty and robustness at budget 40 (fractions of the repair-only oracle). *Posterior*: deployable sets as in Table 8, mean and 90% interval over the posterior of the earlier normal harm model (Appendix J) and the replay bootstrap, and the probability of beating Apply attributed in the same draw. Sensitivity columns charge significant regressions and $c_w = 0.02$ per wrong change unless stated: c_a, 0.02 per applied change instead; *Split*, policies see a random half of every outcome list and adjudication uses the other half (5 splits); *Natural*, values reweighted to the category shares of natural failures; *No step limit*, the step-limit component removed from the bank; *One per comp.*, the accepted set reduced to one patch per component as deployed; *No repl.*, policies draw bank outcomes without replacement.

J Harm Model, Checks and Sensitivity

Model. Regression outcomes are modelled per patch and task, together with the unpatched baseline runs on the same tasks. With n_{0t} unpatched runs of regression task t and f_{0t} failures, and n_{kt} runs of patch k on task t with f_{kt} failures,

$$f_{0t} \sim \mathrm{Bin}\big(n_{0t}, \sigma(\alpha_t)\big),$$
$$f_{kt} \sim \mathrm{Bin}\big(n_{kt}, \sigma(\alpha_t + \beta(\alpha_t - \bar{\alpha}) + \delta_k)\big),$$

with $\alpha_t \sim N(a, s_a^2)$, $\delta_k \sim N(\mu, s_d^2)$, $a, \mu \sim N(0, 3^2)$, $s_a \sim \mathrm{HalfNormal}(2)$, $s_d \sim \mathrm{HalfNormal}(1)$, $\beta \sim N(0, 1)$, and $\bar{\alpha}$ the logit of the mean unpatched failure rate. Task effects are shared by the baseline and every patch, so tasks that often fail unpatched do not make patches look harmful, and the common slope β lets patches move every task's failure rate towards or away from the middle. A patch's harm is its excess failure probability averaged over the regression tasks, weighted by how often each task was solved among clean training episodes,

$$\rho_k = \textstyle\sum_t w_t \big[\sigma(\alpha_t + \beta(\alpha_t - \bar{\alpha}) + \delta_k) - \sigma(\alpha_t)\big],$$

which respects the support $\rho_k \geq -r_0$ by construction; patches without runs draw δ_k from the population. We sample by Metropolis within Gibbs, four chains, 2,400 retained draws; $\hat{R} \leq 1.006$ for μ, s_d, β and the pooled harm.

		per wrong change c_w				per applied change c_a			
	Policy (budget 40)	0	0.01	0.02	0.05	0	0.01	0.02	0.05
Retail	Apply attributed	0.84	0.50	0.16	−0.85	0.84	0.43	0.01	−1.23
	Replay-each	0.74	0.53	0.33	−0.30	0.74	0.48	0.22	−0.57
	HarnessFix-style, per patch	0.51	0.46	0.42	0.28	0.51	0.42	0.34	0.09
	HarnessFix-style, bundle	0.21	−0.05	−0.31	−1.09	0.21	−0.11	−0.42	−1.38
	Uncertainty sampling	−1.90	−2.46	0.23	0.05	−1.90	−2.57	0.09	−0.17
	Uncertainty + checks	−1.57	−1.95	0.25	0.25	−1.57	−2.06	0.12	0.01
	CARVE, replay only	−0.07	−0.49	−0.26	0.00	−0.07	−0.60	−0.44	−0.25
	CARVE	−0.19	−0.19	0.19	0.28	−0.19	−0.27	0.07	0.14
	CARVE, measured check	−0.07	−0.49	−0.26	0.00	−0.07	−0.60	−0.44	−0.25
	CARVE, robust check	−0.05	−0.70	−0.49	0.01	−0.05	−0.81	−0.66	−0.26
	Net-effect (ours)	−1.02	−0.58	0.41	0.63	−1.02	−0.67	0.32	0.44
Airline	Apply attributed	0.60	0.37	0.15	−0.53	0.60	0.26	−0.08	−1.09
	Replay-each	0.09	−0.02	−0.12	−0.43	0.09	−0.08	−0.25	−0.76
	HarnessFix-style, per patch	0.40	0.39	0.37	0.32	0.40	0.33	0.25	0.02
	HarnessFix-style, bundle	0.53	0.32	0.12	−0.50	0.53	0.22	−0.09	−1.02
	Uncertainty sampling	0.49	0.09	0.25	0.19	0.49	−0.04	0.01	−0.25
	Uncertainty + checks	0.44	0.01	0.29	0.21	0.44	−0.12	0.08	−0.13
	CARVE, replay only	0.48	0.08	−0.02	0.06	0.48	−0.07	−0.26	−0.44
	CARVE	0.71	0.33	0.23	0.10	0.71	0.22	0.08	−0.07
	CARVE, measured check	0.56	0.10	0.07	0.11	0.56	−0.04	−0.17	−0.38
	CARVE, robust check	0.63	0.21	−0.01	0.08	0.63	0.06	−0.26	−0.42
	Net-effect (ours)	0.60	0.10	0.31	0.29	0.60	−0.04	0.14	0.05

Table 13: Value at budget 40 as a fraction of the repair-only oracle, with significant regressions charged, as the change cost varies. Policies that use a cost internally are told the same c (the net-effect method always charges it per applied change, the others per wrong change). At $c = 0$ the uncertainty-based policies accept almost every cell with a positive posterior and pick up patches with significant regressions.

	Retail			Airline		
Regression term	10	40	80	10	40	80
As used (clip, rescale)	0.70	0.90	1.08	0.63	0.70	0.77
No clip	0.65	0.38	0.52	0.78	0.82	0.81
No rescale	0.70	0.90	1.08	0.63	0.70	0.77
Neither (Eq. 2)	0.63	0.40	0.56	0.79	0.82	0.81

Table 14: Net-effect method with its regression term as used (clipped at zero, scaled by $1/(1-r_0)$) and without either, posterior value of deployable sets at $c = 0$ over 40 seeds.

Estimates. The unpatched failure rate on solved tasks is 0.146 (retail) and 0.124 (airline). The pooled excess failure rate of the patches is -1.0 points, 90% interval $[-3.4, 1.5]$, on retail and -0.6 $[-3.3, 1.9]$ on airline, and the spread of ρ_k between patches is 2.6 $[0.2, 5.3]$ and 2.1 $[0.2, 4.5]$ points. On retail the slope is negative, $\beta = -0.36$ $[-0.51, -0.17]$: patches tend to raise failures on tasks that rarely fail and lower them on tasks that often fail; on airline it is 0.03 $[-0.21, 0.31]$.

Posterior predictive check. On the patches with 20 runs, the observed standard deviation of failure rates is 0.076 (retail) and 0.066 (airline); data replicated from the fitted model give 0.079 and 0.072, with $P(\text{replicated} \geq \text{observed}) = 0.62$ and 0.79. The number of patches without a failure (4 and 7 observed, 4.2 and 6.2 replicated) and the largest failure rate (0.30 and 0.30 observed, 0.35 and 0.31 replicated) are also reproduced. An earlier analysis used a normal approximation, $y_k = m_k/n_k - r_{0,k} \sim N(\rho_k, v_k)$ with $\rho_k \sim N(\mu, \tau^2)$, which predicted more spread between patches than observed (0.092 and 0.081, $P = 0.95$ and 0.97); its posterior columns in Table 12 and its variants in Tables 4 and 16 are kept as sensitivity analyses.

Sensitivity. Table 16 shows why the harm model matters on retail and hardly on airline. The binomial model values the attributed set at 1.43 oracles on retail; the normal model, which over-predicts the spread between patches and therefore the harm a seven-patch set may carry, values it at 0.77, and its variants range from -1.82 (no pooling) to 1.12 (complete pooling). On airline all models but no pooling give 0.63–0.86. Across models, the best of the five verifying policies beats applying everything in 33–82% of draws on retail and 43–47% on airline.

K Held-Out Ablations

With a margin of 5 points, chosen after the fact, applying the attributed patches is non-inferior to 9

	Cost per check		vs. single replay	
Judge	Tokens	Pro 2× / 4×	Break-even	Ratio [90
Retail				
Bank judge (analyst, full trajectory)	0.29	0.43 / 0.70	0.14	0.47 [0.25,
Analyst, failing step only	0.30	0.45 / 0.76	0.10	0.32 [0.13,
Agent model, full trajectory†	0.67	0.67 / 0.67	0.11	0.16 [0.01,
Agent model, failing step only	0.28	0.28 / 0.28	0.10	0.35 [0.13,
Airline				
Bank judge (analyst, full trajectory)	0.22	0.33 / 0.54	0.14	0.62 [0.33,
Analyst, failing step only	0.23	0.34 / 0.57	0.10	0.43 [0.17,
Agent model, full trajectory†	0.58	0.58 / 0.58	0.24	0.40 [0.10,
Agent model, failing step only	0.22	0.22 / 0.22	0.09	0.39 [0.16,

Table 15: Single-step checks with four judges. Cost is per check as a fraction of a full replay in uncached input plus output tokens, including regenerating the failing step (0.15 of a replay on retail, 0.12 on airline); "Pro 2× / 4×" prices the analyst model's tokens at twice or four times the agent model's, which affects only analyst-model judges (replays use the agent model). Break-even is the cost ratio below which checks decide an edge at lower expected cost than replays (Proposition 4, with the domain's mean spurious recovery and median repair rate of effective edges, averaged over true and false edges); Ratio is break-even divided by cost, so values above 1 favour checks. *vs. single replay*: sensitivity and false-positive rate conditional on the paired replay's outcome (360 stratified pairs; Table 6 gives the bank judge's on all pairs). *vs. patch replay rate*: $z = \epsilon + (s - \epsilon)p_k$ fitted by moments with the binomial noise of each patch's 20-replay rate removed, which is the observation model of Proposition 4; the bank judge row uses all 2,160 pairs, the others their 360 pairs. Intervals bootstrap source trajectories (single replay) or patches (patch rate) together with null replays. †40 random pairs per domain.

	Retail					Airline				
Harm model	μ	Apply	Best	P	VPI	μ	Apply	Best	P	VPI
Task-level binomial (primary)	−0.01	1.43	1.39	0.33	1.18	−0.01	0.66	0.65	0.45	0.86
Hierarchical normal	−0.00	0.77	1.05	0.63	2.02	−0.01	0.73	0.71	0.46	0.69
Complete pooling ($\tau = 0$)	−0.00	1.12	1.02	0.36	0.87	−0.01	0.73	0.71	0.46	0.69
Spread doubled (2τ)	−0.00	0.27	0.96	0.70	2.51	−0.01	0.73	0.71	0.46	0.69
No pooling	0.04	−1.82	0.20	0.82	4.65	0.04	0.39	0.37	0.43	1.15
Tasks resampled	−0.00	−0.52	0.58	0.65	3.31	−0.00	0.63	0.62	0.45	0.87
No support cap	−0.00	0.82	1.09	0.64	9.48	−0.01	0.74	0.71	0.45	1.06
Not deployable	−0.00	0.64	1.14	0.61	2.15	−0.01	0.86	0.86	0.47	0.56

Table 16: Sensitivity of the posterior to the harm model, $c = 0$, budget 40: mean harm μ (for the binomial model, the pooled excess failure rate), value of Apply attributed and of the best of five verifying policies (both HarnessFix-style gates, uncertainty sampling, CARVE, net-effect), the probability that it beats Apply attributed, and the total value of perfect information over Apply attributed, as fractions of the oracle. Rows below the first use the earlier normal approximation; airline has $\tau = 0$ there, so pooling and doubling τ change nothing.

of the 10 verified sets on retail (all but CARVE with replays only) and to the two HarnessFix-style sets on airline. Single patches show the same problem: leaving the step-limit patch out of the attributed set lowers retail success by 3.3 points (s.e. 4.2) where its replays predict 4.9, and leaving out each policy patch lowers it by 0.0 to 3.3 points where replays predict 0.0 to 0.5, each within about one standard error. The shortfall in retail's predicted level is therefore spread over the policy patches rather than located in one, and no single patch's deployed effect is resolved. These runs were made on retail, on the same held-out episodes as Table 5 (Tables 17 and 18) or on new clean episodes (Table 19). Differences are paired and task-clustered, with 90% intervals. Leaving any one of the seven attributed patches out lowers success by 0.00 to 0.03, and the sum of these contributions is not distinguishable from the whole set's effect, so the runs neither show nor exclude interactions. On clean episodes, applying everything gains $+0.04$ $[-0.03, 0.10]$ over the unpatched agent. The bundle gate is ahead of it by $+0.04$ $[0.01, 0.08]$, but three of its five seed sets are identical to the attributed set and one is empty, so the difference is not an effect of verification; on the 72 slots where it ran the identical set the difference is $+0.03$ $[-0.04, 0.09]$, which indicates the run-to-run noise of these comparisons. Unpatched clean success was 0.82 both before and after the new runs.

	With − without step patch			Without − unpatched
Set (seed 0)	All	No step fault	Step fault	No step fault
Apply attributed	0.03 [−0.04, 0.10]	−0.03 [−0.10, 0.04]	0.58 [0.37, 0.80]	0.09 [0.01, 0.18]
HarnessFix-style, bundle	0.03 [−0.04, 0.10]	−0.03 [−0.10, 0.04]	0.58 [0.37, 0.80]	0.09 [0.01, 0.18]
Uncertainty sampling	0.01 [−0.07, 0.08]	−0.02 [−0.09, 0.06]	0.25 [−0.07, 0.57]	0.06 [−0.04, 0.15]
CARVE	0.03 [−0.04, 0.11]	0.00 [−0.07, 0.07]	0.33 [0.07, 0.59]	0.05 [−0.03, 0.13]
Net-effect (ours)	0.05 [−0.02, 0.12]	−0.01 [−0.08, 0.06]	0.58 [0.34, 0.83]	0.02 [−0.07, 0.10]

Table 17: Removing the step-limit patch from each policy's seed-0 set (the bundle gate's seed-0 set equals the attributed set). 108 episodes without and 12 with the step-limit fault.

Patch left out of Apply attributed	All	No step fault
`CFG.max_steps`	0.03 [−0.04, 0.10]	−0.03 [−0.10, 0.04]
`POL.cancel`	0.03 [−0.04, 0.10]	0.04 [−0.04, 0.12]
`POL.exchange`	0.03 [−0.04, 0.09]	0.03 [−0.05, 0.11]
`POL.generic`	0.03 [−0.04, 0.09]	0.01 [−0.05, 0.07]
`POL.intro`	0.02 [−0.05, 0.08]	0.02 [−0.05, 0.08]
`POL.modify`	0.03 [−0.05, 0.10]	0.04 [−0.05, 0.12]
`POL.return`	0.00 [−0.06, 0.06]	−0.01 [−0.08, 0.06]
Whole set vs unpatched	0.12 [0.05, 0.19]	0.06 [−0.01, 0.14]
Whole set − sum of parts	−0.04 [−0.35, 0.27]	−0.03 [−0.38, 0.33]

Table 18: Leave-one-out on the attributed set: success of the whole set minus the set without the named patch.

L Development Variants

On the retail bank we tried, before any airline evaluation of them:

- CARVE with two null replays per category before trusting its replays (per-category b);
- CARVE with a regression gate of two runs per accepted patch;
- both combined;
- an evidence rule that requires a positive likelihood ratio from data;
- tempered and flat priors;
- a Bayesian regression gate.

With control replays and the gate combined, CARVE gained 0.68 and 0.60 of the oracle at budgets 10 and 20, above plain CARVE (0.42, 0.48), but fell below it at 40 and 80 (0.47 vs. 0.57 and 0.56). It never beat the per-patch HarnessFix-style rule (0.72, 0.73). Post hoc on airline it was worse than plain CARVE (0.19 vs. 0.31 at budget 10). The other variants were worse everywhere; the evidence rule and the Bayesian gate accepted almost nothing (gain ≤ 0.12). These numbers use the fixed change cost c without regressions and the bank before the regression runs were extended. The net-effect method of Appendix D was developed afterwards, on retail, from the diagnosis that CARVE lacked per-category spurious recovery and evidence for regressions.

	Old	New	Pooled	− Unpatched	− Apply
Unpatched	0.82	0.82	0.82	–	−0.04 [−0.10, 0.03]
Apply attributed	0.87	0.85	0.85	0.04 [−0.03, 0.10]	–
Net-effect (ours)	0.83	0.87	0.86	0.04 [−0.01, 0.09]	0.00 [−0.05, 0.06]
HarnessFix-style, bundle	0.87	0.91	0.90	0.08 [0.03, 0.14]	0.04 [0.01, 0.08]
HarnessFix-style, per patch	0.90	0.85	0.86	0.05 [−0.01, 0.10]	0.01 [−0.06, 0.08]

Table 19: Clean held-out episodes: 38 old (Table 5) and 120 new (40 tasks × 3), where new episode e uses the policy's seed-$(e \bmod 5)$ set. Differences pool both, paired by episode or slot and clustered by task.

M ShopDesk Details

ShopDesk has six deterministic tools (orders, products, policies, refunds), answers checked exactly (refund amounts, counts, totals), and five injected faults: a tool description misstating price units, a list tool whose description omits pagination, a format instruction that rounds to whole dollars, a planning instruction that skips the policy lookup, and a tight step limit. Over three seeds, the analyst's attribution accuracy on injected failures was 0.75, 0.64 and 0.77. ShopDesk was evaluated online, without a bank: each policy ran against the live agent with fault-based ground truth (an edge is real when at least 30% of a category's failures come from that component's fault). Table 20 reports edge F1 and held-out success. Large harms do occur: a formatting patch that passed its target-category replays made 26% of held-out tasks answer N/A (success 0.89→0.73).

N Study 2: RRSI Port and Registered Analysis

Code. RRSI's loop is vendored from its public repository under the Apache 2.0 licence, with the upstream copyright headers kept. We changed the model backend (calls go to DeepSeek's Anthropic-compatible endpoint, with thinking disabled and token usage recorded per role), and one rule: a proposal that ends with no file change and no declared edit is bounced back to the proposer like an aborted one, at most three times, instead of ending the round with no candidate. RRSI's selection rule is used unchanged: a candidate is admissible if its evolve success is at most δ below the best so far and either exceeds the incumbent's by more than δ at an increase in running cost within $\beta_0 + \beta_1 \Delta S$, or lies within the band with a positive weighted sum of gain, cost saving and novelty (with RRSI's weights, gain counts 0 there, so only a cheaper or untried kind of change passes); a harness with errors in more than 2% of trials is rejected; the admissible candidate with the highest success wins. RRSI's

Method	Edge F1 at budget 10	20	30	60	Held-out success @60	Single checks
LLM attribution only	0.61 ±0.04	0.61 ±0.04	0.61 ±0.04	0.61 ±0.04	0.98 ±0.00	0
Replay-each (do-then-verify)	0.62 ±0.03	0.63 ±0.03	0.67 ±0.05	0.75 ±0.03	0.99 ±0.00	0
Uncertainty sampling	0.55 ±0.04	0.57 ±0.04	0.58 ±0.05	0.66 ±0.04	0.99 ±0.00	0
Uncertainty + cheap checks	0.52 ±0.07	0.56 ±0.07	0.53 ±0.09	0.53 ±0.08	0.96 ±0.03	57
CARVE, full replay only	0.61 ±0.07	0.61 ±0.06	0.59 ±0.04	0.64 ±0.03	0.91 ±0.09	0
CARVE (default check model)	0.59 ±0.06	0.57 ±0.05	0.61 ±0.05	0.68 ±0.03	0.98 ±0.01	4
CARVE, calibrated check	0.59 ±0.06	0.58 ±0.05	0.60 ±0.05	0.66 ±0.04	0.89 ±0.08	0
CARVE, robust check model	0.58 ±0.07	0.61 ±0.06	0.61 ±0.04	0.69 ±0.01	0.99 ±0.01	0
Unpatched agent					0.59 ±0.04	

Table 20: ShopDesk (3 seeds, mean ± s.e.). Held-out success after applying the patches accepted at budget 60 to 120 held-out episodes.

own domains (coding, workspace, engineering design) are not used. Our τ^2 adapter implements RRSI's domain interface: a frozen episode driver that uses τ^2's databases, tools, policies, tasks and user-simulation guidelines, a starting harness with four hooks (choose the next action, check a tool call before it runs, render a tool result, rebuild per-episode state from a recorded prefix), and optional mechanisms the proposer may use: bounded extra model calls, harness-side tools and skill files. Memory across episodes is not allowed.

Harness check. At temperature 0 our episode loop matches τ^2's own orchestrator on 60 test tasks $\times$ 3: 90.0% against 90.8% success on retail (paired difference -0.8 points, $[-5.8, 3.3]$) and 71.7% against 71.7% on airline. Scoring τ^2's natural-language assertions as well changed the reward of 2 of about 1,000 episodes, so we keep Study 1's reward (database state and required information).

Leakage screening. Besides RRSI's LLM critic, a pattern check rejects any added line that contains a task's user id, order or reservation id, item or flight number or customer e-mail, refers to the grader (gold actions, database hashes, evaluation criteria) or to τ^2's task data, imports the benchmark or our experiment code, or writes files.

Evidence collection per round. *References*: for every evolve task the incumbent failed in RRSI's full evaluation, its worst trial, at most 20 tasks (a seeded random subset beyond that). The start step is the earliest step the round's failure digest cites; without a digest, the first state-changing call that is not a gold action; otherwise the first step. Each reference is replayed 4 times with each measured candidate and 4 times with the incumbent (null replays). *Judge*: DeepSeek-V4-Pro reads the candidate's diff and declared edits, up to 14 of the incumbent's failed trials of the round with their grading block, and one line per solved task, and returns a probability of repair for each failed task and an expected number of broken trials; its estimate is (expected repairs − expected breaks) / evolve trials. *Deployment*: every incumbent and measured candidate on the held-out tasks, retail 6 and airline 12 trials per task (4 and 8 as registered; see below), the base and final harnesses on 10 (retail) and 12 (airline) trials per task.

Estimators. With f the incumbent's evolve failure share, *sample@b* is the mean paired difference on b random (task, trial) pairs of the full evaluations; *replay@b* is $f \cdot \widehat{\text{fix}}_c$ from b candidate replays spread evenly over the references; *replay−null@b* is $f(\widehat{\text{fix}}_c - \widehat{\text{fix}}_{\text{null}})$ from $b/2$ replays of each kind; *net@b* uses $b/4$ candidate and $b/4$ null replays and $b/2$ regression pairs, on tasks the incumbent solved, and adds $(1-f)$ times their mean paired difference; *net@b with prior* shrinks the candidate's repair rate to a Beta prior with mean 0.41 or 0.29, 0.7 or 0.2 and two pseudo-trials; *full* is RRSI's own decision. Every estimate goes through RRSI's selection rule with the evidence's own noise band, twice its standard error, and the evidence's measured token cost; the judge has no noise band. Each rule's per-round score is the deployment gain of its choice, averaged over 400 random subsamples of the evidence. The registered analysis applies RRSI's error guard, which rejects a harness that raises errors in more than 2% of trials, only through RRSI's own decision, not to the other rules' evidence; §5 reports the alternative post hoc.

Registration of r1 and changes after it. The pre-registration (`verify/PREREGISTRATION.md`), the analysis code and the run scripts were com-

		Decision value D [90%], common error check			r2+r3			
Evidence	b	r1 (40 rounds)	r2+r3 (80, primary)	r1–r3 (120)	Accepts	Agree	Ep.	$\mathrm{Net}(10^4)$
Keep incumbent	0	+0.00 [+0.00, +0.00]	+0.00 [+0.00, +0.00]	+0.00 [+0.00, +0.00]	0.00	0.74	0	0
None	0	−1.87 [−3.52, +1.53]	−0.47 [−1.52, +0.62]	−0.94 [−1.94, +0.22]	0.91	0.25	0	−818
LLM judge	0	−1.72 [−3.30, +1.37]	−0.40 [−1.41, +0.67]	−0.84 [−1.75, +0.27]	0.81	0.30	0	−731
None + check	10	−0.58 [−1.88, +1.56]	−0.47 [−1.52, +0.62]	−0.51 [−1.39, +0.46]	0.91	0.25	13	−831
Judge + check	10	−0.43 [−1.53, +1.41]	−0.40 [−1.41, +0.67]	−0.41 [−1.20, +0.47]	0.81	0.30	13	−744
Full (RRSI)	all	**+0.22** [−0.04, +1.11]	+0.04 [−0.35, +0.43]	**+0.10** [−0.18, +0.41]	0.26	1.00	141	−108
Sequential full	≤all	+0.15 [−0.03, +0.79]	+0.02 [−0.34, +0.35]	+0.06 [−0.18, +0.32]	0.20	0.93	81	−71
Sequential sample	≤80	+0.12 [−0.15, +0.98]	**+0.07** [−0.26, +0.41]	+0.09 [−0.15, +0.38]	0.23	0.78	53	10
Sample	10	−0.04 [−0.42, +0.77]	−0.08 [−0.54, +0.37]	−0.07 [−0.39, +0.30]	0.39	0.60	13	−107
Replay	10	−0.37 [−1.26, +0.88]	−0.67 [−1.56, +0.17]	−0.57 [−1.20, +0.13]	0.63	0.44	13	−1,000
Replay−null	10	−0.05 [−0.25, +0.16]	−0.36 [−0.74, −0.01]	−0.25 [−0.53, −0.00]	0.18	0.69	11	−594
Net	10	−0.07 [−0.54, +0.93]	−0.17 [−0.73, +0.35]	−0.13 [−0.52, +0.30]	0.48	0.56	12	−217
Sample	40	+0.04 [−0.35, +0.93]	−0.06 [−0.58, +0.45]	−0.02 [−0.37, +0.38]	0.40	0.67	54	−119
Replay	40	−0.40 [−1.56, +1.34]	−0.74 [−1.83, +0.36]	−0.63 [−1.46, +0.30]	0.86	0.33	50	−1,126
Replay−null	40	−0.07 [−0.34, +0.21]	−0.39 [−0.79, −0.03]	−0.29 [−0.58, −0.01]	0.20	0.68	45	−674
Net	40	−0.04 [−0.53, +0.76]	−0.10 [−0.62, +0.39]	−0.08 [−0.45, +0.31]	0.44	0.63	49	−152
Sample	80	+0.08 [−0.30, +1.06]	+0.04 [−0.46, +0.57]	+0.06 [−0.30, +0.47]	0.38	0.78	94	−57
Replay	80	−0.40 [−1.55, +1.34]	−0.71 [−1.80, +0.42]	−0.60 [−1.45, +0.33]	0.88	0.31	56	−1,101
Replay−null	80	−0.12 [−0.60, +0.45]	−0.51 [−1.03, −0.05]	−0.38 [−0.74, −0.02]	0.25	0.65	84	−848
Net	80	−0.02 [−0.53, +0.81]	−0.04 [−0.55, +0.45]	−0.03 [−0.42, +0.36]	0.41	0.69	98	−123

Table 21: Study 2: decision value D of each rule with the error check applied to every rule's own episodes, the held-out gain of its choice over the incumbent in points per round (keeping the incumbent scores 0), with 90% intervals from a bootstrap of blocks of rounds that share an incumbent and of held-out tasks, within domain and trajectory. r1: the first trajectory per domain (its registered analysis is in Table 23); r2+r3: the registered primary analysis; r1–r3: pooled. + *check* rules run 10 episodes of each candidate only for the check; sequential rules stop a candidate's episodes early. Accepts (mean probability of accepting a candidate in a round), Agree (share of rounds in which the rule makes full evaluation's choice), evidence episodes per round, and $\mathrm{Net}(10^4)$ (Eq. 1 against keeping the incumbent, $N = 10^4$ deployed episodes, a success worth $v = 10$ episodes, in episodes) are for r2+r3. Full: RRSI's own decision. Best value per column in bold.

Evidence	Episodes	Estimate of ΔS
None	0	accept a candidate (at random if two)
LLM judge	0	predicted fixes − predicted breaks
Sample@b	b	paired difference on b evolve episodes
Replay@b	b	failure share × repair rate
Replay−null@b	b	failure share × (repair − null rate)
Net@b	b	replay−null − success share × regressions
Net@b, prior	b	net@b with a Beta prior on the repair rate
Full (RRSI)	148 / 60	paired difference on all evolve episodes

Table 22: Evidence compared in Study 2, per candidate, with $b \in \{10, 40, 80\}$; full evaluation costs 148 (retail) or 60 (airline) episodes. The priors have means 0.41 or 0.29 (audited analyst accuracy), 0.7 and 0.2. Every rule decides with RRSI's selection rule.

mitted before the first held-out episode; domains, splits, models, temperatures, T, m, k, budgets, evidence definitions, the decision rule, the margin and the endpoints did not change. The registration file logs four changes. (i) Before any held-out episode, a bug fix: the failure share f was computed from the replay references, which are capped at 20 tasks, instead of from all failed trials of the incumbent's evaluation. (ii) Also before any held-out episode, $\mathrm{Net}(N)$, a registered secondary outcome, was implemented as the mean over rounds of $N(v\Delta p - \Delta c_{\mathrm{run}}) - C_{\mathrm{verify}}$ against keeping the incumbent, with $\Delta C_{\mathrm{change}} = 0$ because every rule sees the same candidates. (iii) After six rounds, net@40 and full evaluation chose differently in 58% of them, above the 40% of the power analysis, and held-out trials per harness were raised from 160 to 240 (retail 4 to 6, airline 8 to 12 per task, airline endpoints 10 to 12); harnesses already deployed were topped up. The registration assumed this rate would be known before any held-out episode ran; in fact the first rounds had been deployed, and a progress log had shown a few held-out success rates while the run was monitored, although no held-out analysis was run. (iv) Over all 40 rounds the rate was 26.5%, for which the registered 160 episodes would have sufficed; we report 240 as primary and 160 as a sensitivity analysis, with the same conclusions. The analyses without the broken candidate,

with the error check applied to every rule, with the execution-check baselines, with blocks of rounds that share an incumbent and of detectable differences, and the rules that verify one candidate or verify sequentially, are post hoc.

Registration of r2 and r3. After r1's results, we registered r2 and r3 (`verify/PREREGISTRATION_E1R.md`) before any of their episodes ran, with the analysis of §5. The analysis code (`verify/e1r.py`) was committed while they ran and tested on r1 only. The sequential rules and their tests were added as an addendum before any decision value of r2 or r3 was computed. The operations log records the following. In r2's third retail round, RRSI's digester received a string where it expected an object and the driver stopped; the fix returns such a reply to the digester as an error and changes nothing else. The container restarted twice, and every run resumed from its last settled step, reusing the reports, drafts and evaluations it had written, so no completed step was sampled again. While the second restart was checked, progress logs displayed the held-out success rates of a few rounds' harnesses (r2 retail round 10, r2 airline 12, r3 retail 9, r3 airline 15). Nothing was decided from them, and the analysis was fixed before.

Thinking proposer (r4). A fourth trajectory per domain, registered as descriptive, runs the proposer, analyst, digester and critic with thinking enabled (8,000 tokens). With thinking, a draft took 1–1.6 hours and 4 of the first 5 drafts after round 0 ended at the proposer's turn limit without a proposal. Retail's noise band was calibrated at 17.7 points from base evaluations of 89.2%, 78.4% and 89.2% (r1–r3: 1.1–4.0), so its acceptance rule was not comparable; it was stopped after two rounds. Airline ran ten rounds, after which the base and final harnesses were deployed; we compare it with the first ten airline rounds of r1–r3. Of its 20 drafts, 14 ended without a proposal (r1–r3: 2–6), so it measured 6 candidates (13–15) and accepted 2 (3–4). All six measured candidates had a positive deployment gain, on average +3.5 points (r1–r3: +0.2 to +1.0, with 40–54% positive), although with a single-candidate MDE of 8.5 points none is resolved on its own. Because nearly every candidate helped, the rules hardly differed: decision values per round lay between +0.85 (sequential full) and +1.17 (no verification), with full evaluation at +0.92 (r1–r3: +0.29 to +0.87) and every 90% interval reaching from about 0 to +6 or more. After ten rounds the incumbent gained +9.2 [+3.8, +15.4] points held out over the base harness (r1–r3 after ten rounds: +8.8, +8.8, +2.9) and +20.0 on the evolve set. Thinking raised the loop's model cost per round by a factor of 3.4–3.9 over r2 and r3. On this one trajectory a stronger proposer drafted fewer but better candidates, which left little for verification to decide; one trajectory cannot separate that from the variation between trajectories.

Sequential rules. Designed post hoc on r1 and registered for r2 and r3, a sequential rule plans N paired episodes per candidate (RRSI's full evaluation, or 80) and runs them in batches of 10 in random order. After n episodes with mean paired difference m it drops the candidate if its harness errors exceed 2% of N, or if the predictive probability that RRSI's rule admits it after N episodes is below $\gamma = 0.05$. For that probability the final ΔS is normal with mean m and variance $\left(\frac{N-n}{N}\right)^2 s^2 \left(\frac{1}{n} + \frac{1}{N-n}\right)$, where s^2 is the larger of the running variance of the paired differences and $S_t(1 - S_t)$; the cost change is held at its running estimate and the novelty count is known. Candidates that are not dropped complete the N episodes and are decided as without stopping; without the predictive stop, sequential full evaluation makes every decision full evaluation makes. With $\gamma = 0.01$ and 0.10, sequential full evaluation gains $+0.17$ at 81 and $+0.13$ at 55 episodes per round on r1, and $+0.03$ at 100 and $+0.02$ at 68 on r2 and r3 ($+0.02$ at 81 with $\gamma = 0.05$). Fully evaluating only the candidate the judge prefers gains $+0.00$ on r2 and r3 at 97 episodes, and a random one $+0.03$.

Classifying candidates. We call a candidate a code change if it edits the harness's agent, tool or check modules, and a text change if it edits only the system prompt or skill files. Code changes rarely lengthened replays: 88 of 1,222 replays of code changes went live before the requested step, all for 4 of 31 candidates, and none of 640 replays of text changes did. RRSI's own component labels rely on file-name patterns that do not match our harness, and called most code changes prompt edits. The labels also feed RRSI's record of which kinds of change have been tried, which it uses to point the proposer at untried kinds and in the novelty term of its selection rule, so the loop explored as RRSI would on a harness whose files its patterns do not recognise; one candidate declared as a harness-side

		Registered analysis		Common error check			Cost per round	$\mathrm{Net}(10^4)$
Evidence	b	D	[90%]	D	[90%]	Accepts	episodes / equiv.	$v = 10$
None	0	−1.87	[−4.61, +0.25]	−1.87	[−4.61, +0.25]	0.90	0 / 0	−1,904
LLM judge	0	−1.72	[−4.42, +0.28]	−1.72	[−4.42, +0.28]	0.72	0 / 0	−1,763
None + check	10	–	–	−0.58	[−1.94, +0.79]	0.88	12 / 12	−652
Judge + check	10	–	–	−0.43	[−1.61, +0.72]	0.70	12 / 12	−512
Full (RRSI)	all	**+0.22**	[−0.06, +0.62]	**+0.22**	[−0.06, +0.62]	0.12	123 / 123	78
Sequential full	≤all	–	–	+0.15	[−0.05, +0.42]	0.10	65 / 65	75
Sequential sample	≤80	–	–	+0.12	[−0.19, +0.51]	0.17	43 / 43	77
Sample	10	−0.34	[−0.99, +0.21]	−0.04	[−0.44, +0.35]	0.30	12 / 12	−69
Replay	10	−0.43	[−1.27, +0.37]	−0.37	[−1.20, +0.45]	0.54	12 / 9	−423
Replay−null	10	−0.05	[−0.26, +0.13]	−0.05	[−0.26, +0.13]	0.08	10 / 8	−69
Net	10	−0.50	[−1.43, +0.23]	−0.07	[−0.57, +0.41]	0.37	11 / 10	−101
Sample	40	+0.04	[−0.38, +0.45]	+0.04	[−0.38, +0.45]	0.28	47 / 47	−6
Replay	40	−1.67	[−4.35, +0.26]	−0.40	[−1.60, +0.78]	0.80	42 / 33	−499
Replay−null	40	−0.07	[−0.35, +0.15]	−0.07	[−0.35, +0.15]	0.10	42 / 34	−125
Net	40	−0.07	[−0.52, +0.36]	−0.04	[−0.50, +0.38]	0.30	44 / 40	−101
Sample	80	+0.08	[−0.35, +0.54]	+0.08	[−0.35, +0.54]	0.26	82 / 82	3
Replay	80	−1.69	[−4.41, +0.27]	−0.40	[−1.59, +0.79]	0.80	47 / 37	−496
Replay−null	80	−0.12	[−0.59, +0.27]	−0.12	[−0.59, +0.27]	0.14	74 / 59	−210
Net	80	−0.02	[−0.49, +0.46]	−0.02	[−0.49, +0.46]	0.27	88 / 81	−119

Table 23: Study 2, first trajectory (r1): decision value D of each rule, the held-out gain of its choice over the incumbent in points per round (keeping the incumbent scores 0), over 40 rounds of both domains, with a 90% bootstrap interval over rounds and tasks (Table 21 gives intervals from blocks of rounds). Registered: RRSI's harness-error check (reject above 2% errors) only inside full evaluation. Common error check (post hoc): the check applied to every rule's own episodes; + *check* rules run 10 episodes of each candidate only for the check; sequential rules stop a candidate's episodes early (post hoc, Appendix N). Accepts, cost and $\mathrm{Net}(10^4)$ (Eq. 1 against keeping the incumbent, $N = 10^4$ deployed episodes, a success worth $v = 10$ episodes, in episodes) are for the common check. No candidate was measured in 4 of the 40 rounds. Full: RRSI's own decision. Best value in bold.

tool was labelled a configuration change. For r2 and r3 the domains' signals see git's python diff driver and label code in the agent module or in a new harness module as control flow; on r1's 47 candidates this gives 24 control-flow and 18 prompt labels instead of 2 and 38, and no code change is labelled prompt only. With these labels the 107 candidates of r2 and r3 were labelled control flow 44 times, prompt 32, rendering of tool results 26, skill 10, harness-side tool 4, extra model call 4, context 3 and configuration 2 (a candidate can carry several labels). As on r1, code changes rarely lengthened replays (316 of 3,236, for 12 of 75 candidates), and replays of text changes never did.

Planned and actual cost. The registered estimate was, per domain, the RRSI trajectory (three base evaluations and up to 40 candidate evaluations), about 2,000–2,700 targeted replays and about 48 harnesses deployed on 160 held-out episodes each: about 30,000 episodes in all, plus the search and judge models, about $70. The study used 21,151 episodes (retail 12,428, airline 8,723): 5,556 in RRSI's evaluations, 3,515 replays and 12,080 held-out episodes, and about $37 at third-party list prices, $10 of it for DeepSeek-V4-Pro. Only 47 of 80 drafts were measured, so fewer replays and deployments were needed than planned. r2 and r3 were quoted at about 42,000 episodes and $75, with a stop at $110. They used 46,584 episodes (11,480 in RRSI's evaluations, 7,824 replays and 27,280 held-out episodes) and about $83, $20 of it for DeepSeek-V4-Pro.

O Study 2: Further Results on the First Trajectory

Registered endpoints. For r1 the registered primary endpoints are (P1) transfer, the held-out gain of the final harness over the base, and (P2) D at $b = 40$, with three one-sided contrasts, Holm-adjusted at $\alpha = 0.05$: net@40 against full evaluation (non-inferiority, margin 0.75 points per round), net@40 against no verification, and full evaluation against no verification. Its bootstrap resamples rounds and held-out tasks within domain. The registered 160 held-out episodes per harness were raised

to 240 after six rounds, as the registration allowed (Appendix N); 160 is a sensitivity analysis.

P2, as registered. Full evaluation had the highest decision value, $+0.22$ points per round $[-0.06, +0.62]$ (Table 23, Appendix N); sample@40 and sample@80 also had small positive point estimates. Net@40, at a third of the cost, had -0.07; the non-inferiority difference was -0.28 $[-0.77, +0.17]$, one-sided $p = 0.058$ (Holm 0.17), so neither non-inferiority nor inferiority was shown. Neither rule was shown better than no verification (full $+2.09$, net@40 $+1.80$; Holm $p = 0.17$). With the registered 160 held-out episodes the non-inferiority difference was -0.48 $[-1.03, +0.01]$, $p = 0.185$. The priors moved net@40 by at most 0.01.

The broken candidate and a common error check. Most of the losses of no verification, the judge and replay without null replays come from one candidate (retail, round 6): a check before item-changing calls that reads an attribute the harness never sets, so the harness raised an error in half of its evolve episodes and held-out success fell by 51.7 points. RRSI's critic accepted it and the judge predicted a gain of 1.5 points; 6 of its 48 replays still succeeded, which replay@b counts as repairs, while 15 of 48 null replays succeeded. The registered analysis gave RRSI's error check to full evaluation only. To compare evidence rather than error handling, we apply the check to every rule's own episodes and add two baselines that run ten episodes of each candidate only for the check, then accept a passing candidate at random or by the judge (post hoc; Tables 23 and 21). Every rule that runs episodes then rejects the broken candidate. Full evaluation still has the highest value; its differences from sample@40, net@40 and replay$-$null@40 are $+0.18$ $[-0.22, +0.58]$, $+0.26$ $[-0.20, +0.75]$ (non-inferiority $p = 0.049$, Holm 0.15) and $+0.29$ $[-0.06, +0.72]$, and these three differ from each other by at most 0.11. Accepting what passes the execution check loses 0.58 points per round $[-1.94, +0.79]$ (0.43 with the judge), 0.80 $[-0.67, +2.19]$ less than full evaluation: beyond crashes, most candidates still hurt.

Dependence and resolution. Retail's 20 rounds share 3 incumbents and airline's 4. Resampling blocks of rounds that share an incumbent widens the standard errors of these contrasts by a factor of 1.3–1.9 (full minus net@40: $[-0.27, +1.00]$), and leaving out any one incumbent moves them by at most 0.20 points. With 80% power at a one-sided 5% level, the design detects per-round differences of 0.60–0.74 points between full evaluation and the @40 rules (0.78–1.11 by blocks), 0.29–0.52 among the @40 rules and about 2 against the execution-only baseline; a single candidate's gain is detected only above about 5.3 points (median standard error 2.1). Apart from the broken candidate, gains were too small to replicate: the split-half correlation of candidates' held-out gains was 0.76 with it and -0.30 without it. Correlations with deployment gain were 0.85 for RRSI's evolve difference (0.33 without that candidate), 0.03 (0.08) for the judge and 0.25 (0.26) for replays.

Spending fewer episodes (post hoc, exploratory on r1). Fully evaluating only the candidate the judge prefers gains $+0.14$ points per round at 94 episodes, as does a random one. Sequential rules run a candidate's evaluation in batches of ten in random order and drop it once its errors exceed the check or the predictive probability that RRSI's rule will admit it falls below 0.05 (Appendix N). Sequential full evaluation gains $+0.15$ $[-0.05, +0.42]$ at 65 episodes per round, 0.07 $[-0.03, +0.24]$ below full evaluation at about half its cost. Sequential sampling capped at 80 episodes gains $+0.12$ $[-0.19, +0.51]$ at 43, against $+0.04$ for fixed sample@40 at 47.

Cost. On r1, full evaluation cost 123 episodes per round and the @40 rules 33–47 episode equivalents, because a replay used about 0.8 of an episode's tokens. Weighting tokens by list prices changes no ranking; it adds the judge's calls, on the larger model, at about 7 episodes per round. Against keeping the incumbent, full evaluation's $\mathrm{Net}(10^4)$ at $v = 10$ is $+78$ episodes $[-183, +466]$, positive in 58% of resamples; its break-even horizon of about 6,000 episodes is a point estimate, and at $v = 1$ no rule repays its evidence within 10^4 episodes. r1 used about 21,200 episodes and $37 (Appendix N).

P Study 2 on AppWorld (E3)

Design. Registered before any episode of its loop, E3 runs the loop, evidence and analysis of r2 and r3 on AppWorld 0.1.3 (Trivedi et al., 2024). The agent is DeepSeek-V4-Flash at temperature 0 with AppWorld's ReAct prompt; each step is one assistant turn whose first Python block runs in a persistent interpreter that calls the apps' APIs. Evolve tasks

Evidence	D [90%]	Ep.	− full [90%]	p (Holm)
Full (RRSI)	+0.03 [−0.30, +0.71]	174	–	–
Sequential full	+0.08 [−0.19, +0.72]	108	+0.05 [−0.00, +0.14]	<0.001 (<0.001)
Sequential sample@80	−0.01 [−0.22, +0.31]	58	−0.04 [−0.47, +0.14]	0.019 (0.019)
Sample@40	−0.04 [−0.38, +0.41]	58	−0.07 [−0.49, +0.15]	0.018 (0.053)
Net@40	−0.08 [−0.46, +0.37]	53	−0.11 [−0.59, +0.18]	0.025 (0.053)
Replay−null@40	+0.22 [−0.20, +0.79]	48	+0.19 [−0.47, +0.67]	0.023 (0.053)
Sample@80	−0.05 [−0.38, +0.44]	116	–	–
None + check	−0.02 [−1.10, +1.35]	14	–	–
Judge + check	+0.18 [−0.86, +1.51]	14	–	–

Table 24: E3 (AppWorld, 20 rounds): decision value D under the common error check in points per round, episodes per round, and the registered non-inferiority contrasts with full evaluation (margin 0.75, one-sided p, Holm within each family), with 90% intervals from the block bootstrap.

are the first 20 training scenarios (60 tasks, two trials each) and held-out tasks are `test_normal` (168 tasks, scenarios disjoint from training); every candidate and incumbent is deployed on two trials per held-out task and the base and final harnesses on four. A task succeeds when all of its unit tests pass. The judge's prompt calls the agent a code agent; nothing else differs. Base evaluations gave 88.3%, 90.0% and 89.2%, a noise band of 2.4 points. The loop ran $T = 20$ rounds, drafted 40 candidates and measured 29 (16 changed code), and accepted three, two within the band. No candidate's harness errors exceeded the check. The study used 15,472 episodes and about $48.

Results. Table 24. Accepting every candidate gained -0.02 points per round and the judge's choice $+0.18$; candidates' gains exceeded the single-candidate detectable effect of 3.2 points for 1 of 29, and their split-half correlation was 0.36. Under r1's registered analysis, with its bootstrap over rounds, net@40 was non-inferior to full evaluation (Holm $p = 0.036$). $\mathrm{Net}(10^4)$ at $v = 10$ was negative for every rule (-160 episodes for full evaluation, -39 for sequential full evaluation).

Q Margins, Decision Rules, Calibration and Loop Cost

The analyses of this appendix are post hoc and use no new episodes: they recompute the registered decision values from the same evidence and deployments (`verify/posthoc_r7.py`, `verify/posthoc_r8.py`).

Smaller margins and agreement. Table 25 repeats the non-inferiority tests against full evaluation at margins of 0.1, 0.25 and 0.5 points per round, adds keeping the incumbent as a rule with zero cost and zero value, and reports how often each rule makes full evaluation's choice (for keeping the incumbent, how often full evaluation keeps it). At the registered margin every rule that runs solved tasks passes, and so does keeping the incumbent; at 0.25 points only sequential full evaluation passes on r2+r3 (sequential sampling: $p = 0.056$), and on r1, the design set, it passes from 0.5. On r4, where full evaluation gained, keeping the incumbent fails at every margin and the sequential rules pass at 0.25. r4's tests rest on ten rounds, two acceptances and three incumbent blocks and are not adjusted for the number of margins and rules; failing non-inferiority does not show that full evaluation beats keeping the incumbent there, and we treat r4 as descriptive.

Agreement on acceptances. Agreement counts every round, and full evaluation keeps the incumbent in 74% of the rounds of r2+r3, where sequential full evaluation, which only stops candidates early, cannot differ from it. Table 26 conditions on full evaluation's decision. Sequential full evaluation made 74% [67, 81] of full evaluation's 21 acceptances on r2+r3 (73% of 26 on r1–r3, 88% of 3 on E3) and never accepted when full evaluation kept the incumbent. It did not select out the unhelpful acceptances: the candidates it would drop gained $+0.30$ points held out on average, those it keeps $+0.11$. It lost little because full evaluation's acceptances were themselves worth little ($+0.16$ points on average), so the gains it forgoes and the harm it avoids nearly cancel. On r4, whose two acceptances gained $+4.6$ points, it would drop a quarter of them in expectation and forgo 0.09 points per round; where accepted changes help more, the gain it forgoes grows with them. The fixed-budget rules differ from full evaluation in both directions: they make about half of its acceptances (sample@80: 76%) and accept in 21–34% of the rounds in which it keeps the incumbent, which with what they accept instead costs 0.12–0.14 points per round (replay−null@40: 0.37).

Evidence or decision rule. In the registered analysis full evaluation is RRSI's own decision, with the noise band calibrated from three base evaluations, while every other rule uses twice its estimate's standard error, and RRSI admits a candidate within its band if it is cheaper or of an untried kind. Table 27 decides full evaluation (and sequential full evaluation) like the other rules, with its own band, and then decides every rule by one cri-

	r2+r3: non-inferiority to full evaluation					Agree with full evaluation				p at margin 0.25		
Rule	− full [90%]	0.1	0.25	0.5	0.75	r2+r3	r1	r4	E3	r1	r4	E3
Sequential full	−0.02 [−0.12, +0.07]	0.086	<0.001	<0.001	<0.001	0.93	0.96	0.95	0.98	0.111	0.023	<0.001
Sequential sample@80	+0.03 [−0.26, +0.35]	0.233	0.056	0.002	<0.001	0.78	0.88	0.96	0.89	0.166	0.005	0.136
Sample@80	+0.00 [−0.44, +0.44]	0.330	0.165	0.029	0.004	0.78	0.84	1.00	0.87	0.269	<0.001	0.131
Sample@40	−0.10 [−0.57, +0.35]	0.477	0.280	0.083	0.018	0.67	0.78	0.86	0.78	0.355	0.135	0.170
Net@40	−0.14 [−0.58, +0.27]	0.536	0.309	0.080	0.015	0.63	0.74	0.82	0.68	0.485	0.316	0.264
Replay−null@40	−0.43 [−0.94, +0.05]	0.868	0.728	0.392	0.141	0.68	0.81	0.77	0.69	0.565	0.182	0.113
Keep incumbent	−0.04 [−0.43, +0.35]	0.390	0.170	0.028	0.004	0.74	0.88	0.80	0.85	0.433	0.778	0.202

Table 25: Non-inferiority to full evaluation at smaller margins (one-sided bootstrap p, block bootstrap as in Table 21, not adjusted; 0.75 is the registered margin), and the expected share of rounds in which a rule makes full evaluation's choice. For keeping the incumbent, Agree is the share of rounds in which full evaluation keeps it. r1 is the exploratory set of the sequential rules; r4 has ten airline rounds and three blocks.

			Full evaluation's acceptances			D(rule) − D(full), points per round			
Rule	Recall [90%]	Acc.	Gain kept	Gain dropped	Dropped	Forgone	Avoided	Other	Total
Sequential full	0.74 [0.67, 0.81]	0.00	+0.11	+0.30	5.4	−0.06	+0.04	−0.00	−0.02
Sequential sample@80	0.48 [0.39, 0.56]	0.11	+1.06	−0.68	10.9	−0.10	+0.19	−0.07	+0.03
Sample@80	0.76 [0.66, 0.85]	0.21	+0.81	−1.94	5.0	−0.03	+0.15	−0.12	+0.00
Sample@40	0.55 [0.48, 0.62]	0.28	+0.60	−0.38	9.4	−0.11	+0.16	−0.14	−0.10
Net@40	0.54 [0.48, 0.61]	0.34	+0.27	+0.03	9.6	−0.12	+0.12	−0.13	−0.14
Replay−null@40	0.23 [0.10, 0.35]	0.15	−0.32	+0.30	16.3	−0.28	+0.21	−0.37	−0.43
LLM judge	0.48 [0.30, 0.65]	0.76	−0.12	+0.42	11.0	−0.21	+0.16	−0.38	−0.44
None	0.62 [0.55, 0.69]	0.88	+0.24	+0.03	8.0	−0.11	+0.11	−0.51	−0.51
Keep incumbent	0.00 [0.00, 0.00]	0.00	–	+0.16	21.0	−0.32	+0.28	+0.00	−0.04
Sequential full, r1 (5)	0.69 [0.49, 0.90]	0.00	+1.52	+2.27	1.5	−0.09	+0.00	+0.02	−0.07
Sequential full, r1–r3 (26)	0.73 [0.67, 0.80]	0.00	+0.37	+0.73	7.0	−0.07	+0.03	+0.01	−0.04
Sequential full, r4 (2)	0.74 [0.49, 1.00]	0.00	+5.57	+1.70	0.5	−0.09	−0.00	+0.02	−0.06
Sequential full, E3 (3)	0.88 [0.64, 1.00]	0.00	+0.59	−2.60	0.4	−0.00	+0.05	−0.00	+0.05

Table 26: Agreement with full evaluation's acceptances on r2+r3 (21 acceptances in 80 rounds; post hoc). Recall: expected share of full evaluation's acceptances the rule also makes, with a 90% block-bootstrap interval. Acc.: share of the rounds in which full evaluation keeps the incumbent where the rule accepts a candidate. Gain kept and dropped: mean held-out gain (points) of full evaluation's accepted candidates, weighted by the probability that the rule makes or does not make the same acceptance; Dropped: expected number it does not make. The last four columns split the difference in decision value: gains forgone by dropping helpful acceptances, harm avoided by dropping harmful ones, the value of what the rule accepts instead (another candidate, or one in a round where full evaluation keeps the incumbent), and the total. Lower block: sequential full evaluation on the other trajectories, with full evaluation's number of acceptances in parentheses.

terion: admit only if the estimate exceeds twice its standard error and the cost condition holds, with no within-band branch. The judge and no verification have no standard error and are unchanged. With their own band, full evaluation's value hardly changes. With one criterion every rule that runs episodes accepts a candidate in at most 3% of rounds (replay−null@40: 7%), and its value moves towards zero; replay−null@40 remains below full evaluation decided the same way (-0.21 [-0.47, -0.01]). The evidence's standard errors are wide relative to candidates' effects, so whether a rule gains or loses is decided mostly by its within-band acceptances.

Calibration of the bands. Table 28 gives, for each evidence type at 40 episodes, the share of evidence draws whose interval, the estimate ± twice its standard error, covers the candidate's full-evaluation ΔS on the evolve set, which the evidence estimates, and its held-out deployment gain. Neither is a nominal coverage of the true effect: full evaluation's ΔS is itself an estimate, with a standard error of 3–5 points on τ^2; the paired samples are subsamples of that same evaluation; and the deployment gain adds held-out noise and the shift from evolve to held-out tasks. We read the table as a consistency check against full evaluation. Paired samples and net evidence are close to 95% on τ^2 and below it on AppWorld (0.81 and 0.70); replay−null covers in 39–62% of draws. Table 29

	Registered rule		Full evaluation with own band		One criterion, no within-band branch	
Rule	D [90%]	Accepts	D [90%]	Accepts	D [90%]	Accepts
Full (RRSI) / own band	+0.04 [−0.35, +0.43]	0.26	+0.03 [−0.47, +0.54]	0.36	+0.09 [−0.06, +0.36]	0.03
Sequential full / own band	+0.02 [−0.34, +0.35]	0.20	+0.08 [−0.21, +0.41]	0.20	+0.09 [−0.05, +0.35]	0.02
Sequential sample@80	+0.07 [−0.26, +0.41]	0.23	+0.07 [−0.26, +0.41]	0.23	+0.10 [−0.02, +0.35]	0.02
Sample@80	+0.04 [−0.46, +0.57]	0.38	+0.04 [−0.46, +0.57]	0.38	+0.10 [−0.02, +0.36]	0.02
Sample@40	−0.06 [−0.58, +0.45]	0.40	−0.06 [−0.58, +0.45]	0.40	+0.04 [−0.04, +0.17]	0.02
Net@40	−0.10 [−0.62, +0.39]	0.44	−0.10 [−0.62, +0.39]	0.44	−0.00 [−0.04, +0.04]	0.02
Replay−null@40	−0.39 [−0.79, −0.03]	0.20	−0.39 [−0.79, −0.03]	0.20	−0.12 [−0.30, +0.03]	0.07
LLM judge	−0.40 [−1.41, +0.67]	0.81	−0.40 [−1.41, +0.67]	0.81	−0.40 [−1.41, +0.67]	0.81
None	−0.47 [−1.52, +0.62]	0.91	−0.47 [−1.52, +0.62]	0.91	−0.47 [−1.52, +0.62]	0.91

Table 27: Decision value on r2+r3 (common error check, points per round, block bootstrap) under three decision rules. "Own band": full and sequential full evaluation decided with twice their own standard error, as the other rules already are, so only their rows change. "One criterion": every rule admits a candidate only if its estimate exceeds twice its standard error and the cost condition holds.

Evidence (@40)	r1	r2+r3	r4	E3
Sample@40	0.98 / 0.94	0.98 / 0.92	1.00 / 0.96	0.81 / 0.80
Net@40	0.95 / 0.94	0.92 / 0.94	0.99 / 0.95	0.70 / 0.83
Replay−null@40	0.54 / 0.76	0.51 / 0.66	0.62 / 0.88	0.39 / 0.67
Replay@40	0.12 / 0.15	0.16 / 0.15	0.32 / 0.00	0.24 / 0.45
Full evaluation	1.00 / 0.94	1.00 / 0.90	1.00 / 1.00	1.00 / 0.97

Table 28: Coverage of ± 2 standard-error intervals: evolve ΔS / deployment gain, averaged over candidates (200 evidence draws each). Full evaluation covers its own ΔS by construction.

Evidence (@40)	Offset [90%]	> 0	SD / SE	Own
τ^2, *r1–r4* (159–160 candidates)				
Sample@40	−0.0 [−0.1, +0.0]	0.46	4.1 / 5.8	0.98
Net@40	+1.2 [+0.8, +1.7]	0.70	5.6 / 6.7	0.97
Replay−null@40	+3.1 [+2.5, +3.8]	0.78	1.1 / 2.1	1.00
Replay@40	+6.6 [+6.0, +7.3]	0.96	0.2 / 1.2	1.00
AppWorld (29 candidates)				
Sample@40	−0.0 [−0.0, +0.0]	0.48	2.5 / 2.8	0.80
Net@40	+0.8 [+0.4, +1.1]	0.72	3.0 / 2.8	0.81
Replay−null@40	+1.9 [+1.1, +2.8]	0.79	0.1 / 0.6	1.00
Replay@40	+2.4 [+1.7, +3.3]	0.93	0.0 / 0.5	1.00

Table 29: Offset of each evidence type at 40 episodes from full evaluation's evolve ΔS: the mean over candidates of the mean of 200 draws minus ΔS, in points, with a 90% bootstrap interval over candidates; the share of candidates with a positive offset; the draws' standard deviation against their mean reported standard error (points); and the share of draws whose ± 2 SE interval covers the draws' own mean (post hoc).

separates two causes of a shortfall. Replay−null's draws are always covered around their own mean, and its standard error is about twice their spread; its misses come from an offset: its mean exceeds full evaluation's ΔS by $+3.1$ points $[+2.5, +3.8]$ on τ^2 and $+1.9$ $[+1.1, +2.8]$ on AppWorld, for 78–79% of candidates. Sampling noise in either estimate averages out over candidates, so a mean offset means that replay−null is optimistic relative to full evaluation, as expected of evidence that cannot see harm to solved tasks; the draws resample one collection of replays, which understates that collection's own variance but does not create the offset. On AppWorld the shortfall of paired samples has the other cause: they have no offset, but their standard error understates the spread of their draws (coverage of their own mean 0.80). Full evaluation's own band covers the deployment gain in 90–100% of candidates.

The loop's whole cost. Table 30 prices the RRSI loop itself, not this study's extra evidence or deployments: the proposal side's model calls (analyst, digester, proposer, critic), the candidates' full evaluations, smoke tests and the base-harness calibration, at list prices. Candidate evaluation is 24–54% of it on τ^2 and 45% on AppWorld; the episodes sequential full evaluation would have saved are 19% of the loop's cost on r1–r3 and 17% on AppWorld. With a thinking proposer (r4) the model calls are 93% of the cost and sequential evaluation saves 1%. In the independent loops, where the proposer also saw early-stopped estimates and made different proposals, sequential evaluation ran 39% fewer candidate-evaluation episodes and the loops cost 10% less (Appendix S).

What each rule buys. Every rule uses what the loop already has: the incumbent's full evaluation, which it ran when it accepted the incumbent, and its failed trials. Sample and sequential rules buy only candidate episodes, paired with the incumbent's existing trials. Replay rules buy candidate and null replays from recorded failures; null replays are shared by a round's candidates. Net evidence buys

Trajectory	Loop $	Model calls %	Cand. eval. %	Smoke, calib. %	Sequential full saves episodes	Sequential full saves % of loop
r1 airline	6.52	69	27	4	793	15
r1 retail	10.73	47	46	6	1,536	20
r2 airline	6.92	64	32	4	437	8
r2 retail	11.92	41	53	6	2,361	32
r3 airline	7.10	72	24	3	392	7
r3 retail	11.83	40	54	6	1,604	21
r4 airline	9.39	93	5	2	93	1
E3 AppWorld	18.58	52	45	3	1,318	17

Table 30: Dollar cost of the RRSI loop per trajectory and domain (r4 retail, stopped after two rounds, omitted), its shares, and the evolve episodes sequential full evaluation would have saved, priced at the mean price of a candidate-evaluation episode.

replays and candidate runs on solved tasks. Under sequential evaluation accepted candidates complete their evaluation, which becomes the next incumbent's, so the next round analyses the same kind of traces as under full evaluation. The proposer's inputs still change: RRSI shows it the scores of rejected candidates, which become early-stopped estimates, so its later proposals may differ even after rounds whose decisions agree; BK's ablation h measures this, and the history main effect, $+0.7$ $[-2.2, +3.7]$, was unresolved (Appendix T). The fixed-budget rules would need an extra full evaluation of each accepted candidate to give the next round the same traces; our counts omit it, which favours them.

Change cost. $\mathrm{Net}(N)$ sets $\Delta C_{\text{change}} = 0$ because every rule sees the same candidates, a simplification: reviewing and maintaining an accepted change costs something, and rules accept different numbers of changes. Charging κ episodes per accepted change lowers a rule's $\mathrm{Net}(10^4)$ by κ times its acceptance rate. On r2+r3 at $v = 10$, sequential sampling, sequential full evaluation and full evaluation have $+10$ $[-315, +350]$, -71 $[-438, +254]$ and -108 $[-500, +274]$ episodes at $\kappa = 0$ and -215, -267 and -370 at $\kappa = 1{,}000$; the fixed-budget rules, which accept 38–44% of rounds, fall further.

Registration timeline. All times UTC, 2026, from the repository history. 7 Oct 11:26: r1's registration, analysis code and launch script committed; the loop and its held-out deployments started after it. 15:34–15:54: the registered power rule raised held-out trials (Appendix N). 20:22: r1 finished; 20:23 its final disagreement rate logged. 8 Oct 00:24: r2 and r3 registered, before any of their episodes; 00:45: addendum (thinking roles for r4, verify-one-candidate rules). 01:00: E3 registered, before any episode of its loop. 02:06: the sequential rules and their tests registered for r2, r3 and E3 (addendum 2); no decision value of r2, r3 or E3 had been computed. 08:01: r2 and r3's analysis code committed, tested on r1 only. 09:55: every round of r2 and r3 deployed; 10:00–10:15 their registered analyses run as committed. 16:32 and 18:02: E3's loop and deployments finished; 18:14 its registered analysis run. 9 Oct 08:32: the closed loops cl1 and cl2 registered, before any of their episodes; 08:38: their analysis code; 14:07: their deployments finished; 14:08 their registered analysis run. 14:43: IL registered with its analysis code, before any of its episodes, and its eight loops launched; 17:28: its last deployment and shadow evaluation finished; 18:01: its registered analysis run. The operations logs in the registration files record the restarts, the r4 truncation and the held-out rates seen in progress logs.

R Validating the Stopping Rule

The analyses of this appendix are post hoc and use no new episodes (`verify/posthoc_r9.py`).

Calibration. For every measured candidate of r1–r4 and E3 we replay sequential full evaluation ($\gamma = 0.05$, batches of 10) in 400 random orders of its full evaluation's episodes and record, at every interim look, the predicted probability p that RRSI's rule admits the candidate after all N episodes, and whether it stops there; the outcome is full evaluation's decision. In the six live loops (cl1, cl2, s1–s4) each candidate has the one order it ran, and the outcome is the loop's decision for candidates that completed and the shadow evaluation's for those it stopped. Table 32 bins the looks by p. The probabilities are too extreme at both ends: on r2+r3, looks with $p \geq 0.95$ were followed by admission in 78% of cases and looks with $p < 0.01$ in 8%. Candidates stopped at a mean p of 0.02 were admitted by full evaluation in 8% of cases on r2+r3, 12% on airline, 3% on AppWorld and 13% in the live loops, so γ does not bound the admissible candidates lost: the rule stopped 24% of the candidates full evaluation admitted on r2+r3 and 35% in the live loops. The second pair of columns recomputes p with the cost change of the complete evaluation in place of its running estimate, a diagnostic rather than a usable rule: looks below 0.05 are then followed by admission in at most 2% of cases, so most of the error at the low end comes from the

cost term. The error at the high end remains.

A stopping rule that models the cost change (post hoc). The diagnostic above uses the complete evaluation's cost change and cannot be run. A rule that can treats the final cost change as normal around its running estimate, with variance $\left(\frac{N-n}{N}\right)^2 s_c^2 \left(\frac{1}{n} + \frac{1}{N-n}\right)$ from the per-pair variance s_c^2 of the token difference relative to the incumbent's mean, independent of the final ΔS, and integrates RRSI's admission regions over both (`verify/posthoc_r10.py`). We replay it in the same 400 orders per candidate as the registered rule (Table 31). At $\gamma = 0.05$ it keeps 88% of the candidates full evaluation admits on r2+r3 instead of 75%, 91% instead of 71% on airline and 94% instead of 86% on AppWorld, for 64%, 77% and 67% of full evaluation's episodes instead of 57%, 65% and 62%. Its stopped candidates are admitted by full evaluation in 4%, 5% and 1% of cases, against 8%, 12% and 3%, and on r2+r3 it stops 14% and 15% of the candidates admitted inside the band for cost and for novelty, against 32% and 35%. Per round it makes 88% of full evaluation's acceptances on r2+r3 (registered rule 74%), 89% on airline (69%) and 96% on AppWorld (88%), and its decision value differs from full evaluation's by $+0.00$, -0.02 and $+0.02$ points per round. At $\gamma = 0.10$ it keeps 79% at 54% of the episodes on r2+r3, more than the registered rule at fewer episodes. Never stopping for cost, that is stopping only when the final ΔS is unlikely to reach RRSI's floor, keeps 96–100% but saves little: 77% of the episodes on r2+r3 and 97% on airline. The correction removes about half of the lost acceptances offline; BK then ran it in registered loops (Appendix T).

Dependence (`verify/posthoc_r12.py`). The rule treats the final changes in success and cost as independent. Per candidate, the correlation of the paired success difference with the paired token difference has median -0.07 on r2+r3 (10th to 90th percentile -0.22 to $+0.15$), -0.02 on airline and -0.34 on AppWorld, where successful episodes are shorter. Making the final cost change normal given the final success change, with correlation ρ, moves the share of full evaluation's acceptances made per round on r2+r3 from 90% ($\rho = -0.6$) to 82% ($\rho = +0.6$), at 67% to 59% of the episodes: a positive correlation, gains that cost tokens, makes the rule stop more. With ρ estimated from the candidate's pairs so far it makes 87% at 64% of the episodes (independent: 88% at 64%), 88% on airline (89%) and 98% on AppWorld at 70% (96% at 67%), and decision values change by at most 0.01 points per round. It also saves less: replayed in the rounds of the four concurrent full-evaluation loops of the registered comparison (`verify.bk replay`, no new episodes), the registered rule changes 3 of their 40 decisions and runs 73% of their candidate-evaluation episodes, the cost-aware rule 1 decision at 82%. With candidate evaluation about a third of a loop's cost, its saving is about 5% of the loop's cost, and part of the registered rule's saving came from the stops the correction removes.

By branch. Table 33 splits candidates by the branch of RRSI's rule that decided their complete evaluation. Sequential evaluation stops most rejected candidates early and rarely those admitted above the band, but about a third of those admitted inside it. Inside the band RRSI's shaped score is $w_s \Delta S - 15\,\Delta C + 0.5\,\nu$ with $w_s = 0$ in RRSI's configuration, so a candidate without a new component is admitted exactly when its token cost falls; the rule holds that cost change at its running estimate, whose sign after 10–30 episodes is often wrong.

Thresholds. Table 34 varies γ from 0.005 to 0.5. On r2+r3 the episodes rise from 22% to 74% of full evaluation's and the share of its acceptances kept from 26% to 86%; keeping 85% needs $\gamma = 0.01$ and 71% of the episodes. The decision values hardly move, because full evaluation's own is near zero; on r1 they rise with the share kept, toward full evaluation's $+0.22$.

Matched cost. Table 35 sets the rules on r2+r3 against their episodes. At sequential full evaluation's episodes, fixed sampling (sample@60) keeps as many of full evaluation's acceptances, 75% against 74%, but also accepts in 21% of the rounds in which full evaluation keeps the incumbent, which a rule that only stops evaluations never does. Fully evaluating one candidate drawn at random keeps 61% at 69% of the episodes. The decision values differ by less than 0.05 points per round, because full evaluation's acceptances were worth little. On AppWorld sequential full evaluation keeps 88% of full evaluation's acceptances at 62% of its episodes, fixed sampling 55–64% at 50–67%, and one random candidate 47% at 66%.

Leaving out trajectories. Table 36 repeats the non-inferiority tests with one trajectory and domain left out at a time. Sequential full evalu-

	τ^2 r2+r3			τ^2 airline r1–r4			AppWorld		
Stopping rule	Stop adm.	Kept	Ep.	Stop adm.	Kept	Ep.	Stop adm.	Kept	Ep.
Registered (cost held)	8%	75%	57%	12%	71%	65%	3%	86%	62%
Cost-aware, $\gamma = 0.05$	4%	88%	64%	5%	91%	77%	1%	94%	67%
Cost-aware, $\gamma = 0.10$	7%	79%	54%	8%	84%	69%	2%	89%	59%
Cost-aware, $\gamma = 0.20$	11%	62%	41%	12%	71%	58%	6%	71%	45%
Never stop for cost	2%	96%	77%	6%	99%	97%	0%	100%	94%

Table 31: Sequential full evaluation with the cost change held at its running estimate (registered) or with its uncertainty modelled (cost-aware), replayed in 400 random orders of each candidate's episodes (post hoc). Stop adm.: share of stopped candidates that full evaluation admits; Kept: share of the candidates full evaluation admits that are not stopped; Ep.: episodes as a share of full evaluation's. Never stop for cost: stop only when the final ΔS is predicted to fall below RRSI's floor.

	r2+r3		r2+r3, final ΔC		Airline r1–r4		AppWorld		Live loops	
Predicted p	mean p	admitted	mean p	admitted	mean p	admitted	mean p	admitted	mean p	admitted
$[0.00, 0.01)$	0.003	0.08	0.003	0.00	0.003	0.10	0.004	0.03	0.004	0.22
$[0.01, 0.05)$	0.028	0.08	0.028	0.02	0.026	0.14	0.030	0.03	0.027	0.08
$[0.05, 0.20)$	0.12	0.12	0.12	0.07	0.11	0.17	0.10	0.05	0.12	0.18
$[0.20, 0.50)$	0.34	0.21	0.34	0.19	0.33	0.25	0.32	0.10	0.35	0.16
$[0.50, 0.80)$	0.64	0.36	0.64	0.39	0.65	0.38	0.65	0.38	0.65	0.51
$[0.80, 0.95)$	0.88	0.52	0.88	0.62	0.88	0.52	0.89	0.66	0.87	0.64
$[0.95, 1]$	0.98	0.78	0.98	0.89	0.99	0.74	0.98	0.84	0.98	0.74
Stopped: mean p / admitted	0.021	0.08	–		0.014	0.12	0.025	0.03	0.018	0.13
Stopped: selected by full	0.07		–		0.10		0.02		0.10	
Admitted by full: stopped	24% of 24		–		28% of 23		15% of 5		35% of 23	

Table 32: Calibration of sequential full evaluation's predicted admission probabilities (post hoc). Rows bin the interim looks by p; "admitted" is the share of those looks whose candidate full evaluation admitted. Recorded trajectories: 400 orders per candidate. Live loops: cl1, cl2 and s1–s4, outcomes from the loop or its shadow evaluation. "Final ΔC": p recomputed with the complete evaluation's cost change (a diagnostic). Bottom rows: stopped candidates (mean p at the stop, share admitted and selected by full evaluation) and the share of full evaluation's admitted candidates that were stopped.

	τ^2 r1–r4		AppWorld		Live loops	
Full evaluation's branch	Cand.	Stopped	Cand.	Stopped	Cand.	Stopped
Above the band	11	0.04	2	0.01	2	0.00
In band, cheaper	20	0.33	2	0.19	16	0.38
In band, new component	5	0.38	1	0.33	5	0.40
In band, rejected	53	0.81	13	0.93	54	0.91
Below the floor	71	0.91	11	0.99	9	0.44

Table 33: Share of candidates sequential full evaluation stops (statistical stops; expected over 400 orders for recorded trajectories), by the branch of RRSI's rule that decided the complete evaluation.

ation stays non-inferior at 0.25 points in every case ($p \leq 0.006$). Sample@40 and net@40 keep $p \leq 0.063$ at the registered margin of 0.75 points, below 0.05 except when r3's airline rounds are left out; replay−null@40 fails in every case on r2+r3.

Independent evidence. Table 37 counts the units behind each analysis. The block bootstrap of the primary analysis resamples 23 incumbent blocks of r2+r3, which hold 21 acceptances, and held-out sets of 40 and 20 tasks; the 1,400 to 4,100 candidate-evaluation episodes per trajectory and domain are repetitions within these units. Narrower intervals need more trajectories and more held-out tasks rather than more trials per task.

S Loops Driven by Sequential Evaluation

Two registered experiments ran RRSI with sequential full evaluation as its selection step: CL (two loops, descriptive) and IL (eight concurrent loops, four with each selection step, and the registered comparison of the main text).

CL design. Registered after every analysis above and before any of its episodes (`verify/PREREGISTRATION_CL.md`), two new airline trajectories, cl1 and cl2, ran rounds 0–9 of RRSI with its selection step replaced by sequential full evaluation ($\gamma = 0.05$, batches of 10; Appendix N); everything else is as for r2 and r3, with $T = 20$ so that the edit-budget schedule matches r1–r3's first ten rounds. A dropped candidate enters RRSI's history as rejected with its

	r2+r3			r1			E3 (AppWorld)			r2+r3
γ	Ep. share	Kept	$D-$full	Ep. share	Kept	$D-$full	Ep. share	Kept	$D-$full	p at 0.25
0.005	0.74	0.86	−0.01	0.70	0.79	−0.05	0.79	0.95	+0.02	<0.001
0.01	0.71	0.85	−0.02	0.65	0.78	−0.05	0.74	0.93	+0.03	<0.001
0.02	0.65	0.80	−0.02	0.60	0.75	−0.06	0.68	0.90	+0.04	<0.001
0.05	0.57	0.74	−0.02	0.53	0.69	−0.07	0.62	0.88	+0.05	<0.001
0.1	0.48	0.65	−0.02	0.44	0.62	−0.09	0.49	0.80	+0.07	0.006
0.2	0.38	0.52	−0.02	0.34	0.53	−0.09	0.44	0.73	+0.06	0.022
0.3	0.32	0.43	−0.01	0.30	0.45	−0.10	0.34	0.62	+0.05	0.033
0.5	0.22	0.26	+0.01	0.22	0.33	−0.11	0.20	0.34	+0.01	0.045

Table 34: Sequential full evaluation across thresholds γ (post hoc): candidate-evaluation episodes as a share of full evaluation's, the expected share of full evaluation's acceptances it makes (Kept), its decision value minus full evaluation's (points per round), and the one-sided bootstrap p of non-inferiority at a margin of 0.25 points on r2+r3.

Rule (r2+r3)	Ep.	Kept	Acc. | keep	D	−full [90%]	$p_{0.25}$
Full evaluation	141	1.00	0.00	+0.04	–	–
Sequential full ($\gamma = 0.05$)	81	0.74	0.00	+0.02	−0.12, +0.07	<0.001
Full, one random candidate	97	0.61	0.00	+0.03	−0.21, +0.19	0.029
Sample@10	13	0.35	0.32	−0.08	−0.58, +0.34	0.299
Sample@20	27	0.47	0.33	−0.07	−0.61, +0.36	0.303
Sample@30	40	0.51	0.30	−0.06	−0.57, +0.35	0.279
Sample@40	54	0.55	0.28	−0.06	−0.57, +0.35	0.280
Sample@50	67	0.60	0.25	−0.01	−0.51, +0.38	0.225
Sample@60	80	0.75	0.21	+0.05	−0.43, +0.45	0.161
Sample@80	94	0.76	0.21	+0.04	−0.44, +0.44	0.165
Sample@100	108	0.79	0.19	+0.05	−0.40, +0.43	0.143
Sample@120	122	0.81	0.18	+0.05	−0.38, +0.43	0.137
Sample@150	141	0.86	0.17	+0.03	−0.41, +0.42	0.165

Table 35: Rules on r2+r3 by their candidate-evaluation episodes per round (post hoc). Kept: expected share of full evaluation's acceptances the rule makes; Acc. | keep: how often it accepts when full evaluation keeps the incumbent; D and its difference from full evaluation in points per round with a 90% block-bootstrap interval; $p_{0.25}$: non-inferiority at 0.25 points. Sample@b decides with twice the sample's standard error; at $b \geq 60$ on airline it uses every pair.

	Seq. full		Sample@40		Net@40		Replay−null@40	
Left out	−full	$p_{0.25}$	−full	$p_{0.75}$	−full	$p_{0.75}$	−full	$p_{0.75}$
r2+r3 without r2 retail	−0.02	0.001	+0.08	<0.001	+0.03	0.002	−0.42	0.171
r2+r3 without r2 airline	−0.01	<0.001	−0.13	0.035	−0.09	0.030	−0.27	0.051
r2+r3 without r3 retail	−0.04	0.005	−0.17	0.043	−0.25	0.049	−0.46	0.149
r2+r3 without r3 airline	−0.02	0.004	−0.18	0.060	−0.24	0.063	−0.58	0.289
r1–r3 without r1 retail	−0.05	0.004	−0.09	0.004	−0.16	0.009	−0.44	0.119
r1–r3 without r1 airline	−0.01	<0.001	−0.13	0.011	−0.16	0.010	−0.37	0.069
r1–r3 without r2 retail	−0.04	0.004	−0.03	<0.001	−0.08	<0.001	−0.37	0.092
r1–r3 without r2 airline	−0.04	0.002	−0.15	0.015	−0.16	0.016	−0.28	0.029
r1–r3 without r3 retail	−0.05	0.006	−0.17	0.013	−0.25	0.025	−0.39	0.081
r1–r3 without r3 airline	−0.04	0.004	−0.18	0.019	−0.25	0.028	−0.47	0.138

Table 36: Leave one trajectory and domain out (post hoc): difference from full evaluation in points per round and one-sided bootstrap p of non-inferiority (block bootstrap, not adjusted).

Trajectory	Rounds	Cand.	Full acc.	Blocks	Evolve	Held out	Cand. ep.
r1 retail	20	24	2	3	74 × 2	40 × 6	3,552
r1 airline	20	23	3	4	30 × 2	20 × 12	1,380
r2 retail	20	27	3	4	74 × 2	40 × 6	3,996
r2 airline	20	29	6	6	30 × 2	20 × 12	1,740
r3 retail	20	28	6	6	74 × 2	40 × 6	4,144
r3 airline	20	23	6	7	30 × 2	20 × 12	1,380
r4 airline	10	6	2	3	30 × 2	20 × 12	360
E3 AppWorld	20	29	3	4	60 × 2	168 × 2	3,480

Table 37: Size of the independent evidence per trajectory and domain: rounds, measured candidates, full evaluation's acceptances, incumbent blocks (the units of the block bootstrap), evolve and held-out tasks × trials per candidate, and candidate-evaluation episodes.

early-stopped estimate, which the proposer sees in later rounds. The base, every accepted incumbent and the final incumbent were deployed on the 20 test tasks, 12 trials each. After round 9, every dropped candidate's evaluation was completed to all 60 episodes in a separate job (the shadow evaluation) and decided by RRSI's rule with that round's S^*, noise band and novelty. The reference is the first ten airline rounds of r1–r3. All outcomes are descriptive; the registered consistency criterion asks both trajectories' transfer to lie in r1–r3's range widened by their 90% intervals, $[+0.42, +15.42]$ points.

CL results. Both trajectories meet the criterion, cl1 only at its lower edge (Table 38). Their noise bands, calibrated from three base evaluations, were 12.5 and 7.2 points (r1–r3: 17.8, 5.4 and 9.4). cl1 accepted one change in ten rounds: sequential evaluation dropped 14 of its 16 candidates, and the shadow evaluation agrees with 12 of those drops. The accepted change (round 6) lowered evolve success from 71.7% to 66.7% and was admitted inside the band for a new component; held out it gained $+0.42$ points. Full evaluation would also have accepted round 3's first candidate (evolve -6.7 points, admitted inside the band for a new component) and round 9's ($+1.7$); deployed post hoc, they gained $+3.75$ $[-0.8, +9.2]$ and $+4.17$ $[-0.8, +9.6]$ points over their incumbents (90% intervals), which suggests that cl1's low transfer came partly from what sequential evaluation dropped. Both would have been accepted inside the band, for a new component and for a lower to-

Traj.	Selection	Accepted	Cand. episodes	Loop $	Transfer	90% CI
r1	full	3 (3)	780	$3.42	+8.75	[+2.50, +15.42]
r2	full	3 (2)	900	$3.46	+8.75	[+3.75, +14.17]
r3	full	4 (4)	780	$3.49	+2.92	[+0.42, +5.42]
cl1	seq.	1 (1)	460 (48%)	$3.20	+0.42	[−4.58, +4.58]
cl2	seq.	5 (4)	760 (79%)	$3.68	+12.08	[+5.81, +19.17]

Table 38: Loops on airline, rounds 0–9: r1–r3 driven by full evaluation, cl1 and cl2 by sequential full evaluation. Accepted changes (in parentheses, those inside the noise band), candidate-evaluation episodes (share of the planned episodes), the loop's dollar cost at list prices (Table 30), and transfer at round 10: held-out success of the incumbent after round 9 minus the base, in points, with a 90% task-bootstrap interval.

ken cost, and both had started worse or barely better than the incumbent: sequential evaluation dropped them after 10 episodes at -10 points and after 40 at $+2.5$. cl2 accepted five changes, four inside its band, each of which gained held out ($+0.4$ to $+3.75$ points), and its final harness gained $+12.1$, the most of the five trajectories. Sequential evaluation dropped 8 of its 16 candidates; one of them (round 2) full evaluation would also have admitted, but not chosen. Over both trajectories, full evaluation as the shadow evaluation scores it would have made the sequential loop's decision in 18 of 20 rounds, and the sequential loop made 6 of its 8 acceptances, close to the 93% and 74% measured offline on r2 and r3.

CL cost. Over rounds 0–9 the loops cost $3.20 and $3.68 at list prices, against $3.42–3.49 for r1–r3. Candidate evaluation fell to $0.59 and $1.00 from $0.98–1.16, but cl1 and cl2 measured 16 candidates each, against 13–15, and their proposal-side model calls cost $2.38 and $2.43, against $2.06–2.27. The experiment, deployments and shadow evaluations included, cost about $11.

CL changes after registration. The container restarted during round 7 of both trajectories; both resumed from their checkpoints two hours later, reusing every episode on disk and replaying the stopping order from the same seed. To make up the time, deployments and shadow evaluations ran three at a time. Both changes were made before any held-out result was seen. The deployment of cl1's two forgone acceptances was decided after the registered analysis and is post hoc. The operations log of the registration records each change.

IL design. Registered after CL's results and before any of its episodes (`verify/PREREGISTRATION_IL.md`), eight new airline loops ran rounds 0–9 concurrently, each in its own checkout of one commit: four with sequential full evaluation (s1–s4, as cl1 and cl2) and four with RRSI's own full evaluation (f1–f4); everything else is as for CL. The base harness and the incumbent after round 9 of every loop were deployed on the 20 test tasks, 12 trials each, and the sequential loops' stopped candidates were completed in shadow evaluations. The registered comparison pools them with cl1 and cl2 (sequential) and r2 and r3 (full; their incumbent after round 9 had been deployed for CL), six loops per arm. Its outcomes are transfer at round 10, the loop's dollar cost over rounds 0–9 and its candidate-evaluation episodes, each as sequential minus full with a 90% interval from a bootstrap over loops within arm and, for transfer, over held-out tasks ($B = 2000$), and the one-sided bootstrap probability that the transfer difference is below -5 points. Before any episode we noted that six loops per arm give about 55% power at that margin, so failing it would be uninformative. Sensitivity analyses use the eight concurrent loops only and add r1 to the full arm. No held-out success rate of IL was examined before every deployment had finished. The arm labels were fixed in the registration, not randomised. One script launched the eight loops within one second, in one container, against the same API endpoint with the same parallelism settings; every loop ran the same evolve and held-out tasks, and the order in which the sequential rule runs a candidate's episodes is seeded by domain, round and candidate label, as in every other loop. Each loop calibrated its noise band with the same code before its first selection step, so the bands' imbalance between arms arose from unseeded API sampling, not from the selection step.

IL results. Table 39 and Figure 1. Transfer after ten rounds was $+0.4$ to $+12.1$ points in the sequential loops (mean $+4.2$, standard deviation 4.6) and $+2.9$ to $+12.1$ in the full-evaluation loops (mean $+8.7$, standard deviation 3.1): a difference of -4.4 $[-9.4, -0.1]$ points (Welch standard error 2.25), with probability 0.38 of lying below -5, so non-inferiority at 5 points is not shown. Among the eight concurrent loops the difference was -6.9 $[-11.7, -2.6]$; every full-evaluation loop ended above every sequential one (descriptive). With r1 in the full arm it was -4.5 $[-9.5, -0.3]$. The sequential loops ran 538 candidate-evaluation episodes

Loop	Selection	Accepted	Cand. episodes	Loop $	Transfer
r2	full	3	900	$3.46	+8.75
r3	full	4	780	$3.49	+2.92
f1	full	2	1080	$3.72	+9.58
f2	full	4	600	$3.04	+9.58
f3	full	5	960	$3.71	+12.08
f4	full	5	960	$3.78	+9.17
cl1	seq.	1	460	$3.20	+0.42
cl2	seq.	5	760	$3.68	+12.08
s1	seq.	4	710	$3.17	+5.00
s2	seq.	0	330	$2.83	+0.00
s3	seq.	1	390	$3.07	+6.25
s4	seq.	3	580	$3.17	+1.67
Seq.−full (6 vs 6)			−342	−0.35	−4.44
90% interval			[−483, −203]	[−0.57, −0.10]	[−9.44, −0.14]

Table 39: The registered comparison of independent airline loops, rounds 0–9: accepted changes, candidate-evaluation episodes, the loop's dollar cost at list prices, and transfer at round 10 (held-out success of the incumbent after round 9 minus the base, points). Intervals: bootstrap over loops within arm and, for transfer, held-out tasks. f1–f4 and s1–s4 ran concurrently from one commit.

on average against 880 (-342 [-483, -203]) and cost \$3.19 against \$3.53 ($-$\$0.35 [-0.57, -0.10]). They accepted 0 to 5 changes, 14 in all, and the full-evaluation loops 2 to 5, 23 in all. In the shadow evaluation of the six sequential loops, full evaluation would have made the same decision in 54 of 60 rounds and 20 acceptances, 14 of which the loops made; of their 62 stopped candidates it would have admitted 8 and selected 6. The concurrent loops ran without restart, retry or change after registration, and IL cost \$34.12.

Matched offline comparison (post hoc). The offline analyses of §5.1 pool retail and airline over twenty rounds, whereas the loops measure airline transfer after ten. Restricted to airline rounds 0–9 of r1–r3, the trajectories full evaluation drove, full evaluation's decision value was $+0.68$ points per round [$+0.08$, $+1.59$] and sequential full evaluation's $+0.58$ [$+0.07$, $+1.35$], a difference of -0.10 [-0.35, $+0.07$]; sequential evaluation kept 63% of full evaluation's ten acceptances at 69% of its episodes. In this restricted set the reference gains, unlike in the primary analysis. Because an accepted candidate becomes the next incumbent and every incumbent is deployed, the sum of full evaluation's round values over rounds 0–9 equals the trajectory's transfer at round 10 ($+8.75$, $+8.75$ and $+2.92$ on r1–r3). The same sum for sequential evaluation, whose decisions are scored in full evaluation's states, is $+5.76$, $+9.30$ and $+2.26$: offline it would end 1.0 point lower [-3.7, $+0.8$] (bootstrap of incumbent blocks and tasks). Adding r4 gives -0.9; r2 and r3 alone give -0.1. The live difference, -4.4, exceeds the offline prediction by 3.4 points [-8.3, $+1.8$]. The two analyses share r2, r3 and the held-out tasks, so this interval comes from one bootstrap that draws the 20 tasks once for both and resamples, within strata, the six sequential loops, the four full-evaluation loops that appear only live, r2 and r3 (a drawn trajectory enters both analyses) and r1, with rounds resampled in incumbent blocks as offline (`verify/posthoc_r11.py`); the live loss is larger in 86% of draws, and with the rounds held fixed the interval is [-8.0, $+0.7$]. Two independent bootstraps gave [-8.7, $+1.3$], as the two parts are nearly uncorrelated (0.09). In the live loops' own states the rule kept 14 of full evaluation's 20 acceptances (70%), as offline. The per-decision behaviour thus transferred, while whether IL's loops lost more than their recorded decisions predict remained unresolved; BK's loops, which shared their calibration, lost 0.8 points (Appendix T).

Noise bands (post hoc). Each loop calibrates its noise band δ from three evaluations of the base harness before round 0, which its selection step cannot influence; δ ranged from 2.7 to 12.5 points over the twelve loops (17.8 on r1). By chance the sequential loops' bands were wider, 9.8 points on average against 6.7 (concurrent loops 9.7 against 6.4). A wider band leaves more candidates to RRSI's in-band branch, where admission turns on cost or novelty and the sequential rule stops about a third of admissible candidates (Table 33), and δ correlated with transfer in both arms (-0.57 sequential, -0.63 full). Regressing transfer on arm and centred δ gives an arm difference of -2.1 [-6.2, $+1.9$] for the six loops per arm ($p = 0.35$; slope -0.76 points of transfer per point of δ), -5.8 [-9.4, -2.2] for the eight concurrent loops ($p = 0.02$) and -4.0 [-7.6, -0.3] with r1 (t intervals; a bootstrap over loops and tasks gives [-7.1, $+3.4$], [-10.9, -1.2] and [-8.5, $+1.2$]). The adjustment was not registered, and the band may also moderate the rule's loss rather than only shift transfer; the concurrent comparison, in which both arms ran together, keeps its difference.

What the shadow evaluations separate (post hoc). A sequential loop differs from a full-evaluation loop in admission, since a stopped candidate cannot be accepted, and in history, since a stopped candidate enters the proposer's history as rejected with its early-stopped estimate. In the se-

	Loops	Sequential	Full	Difference [90%]	δ-adjusted [90%]	Accepted	Mean δ
Recorded rounds 0–9, offline	3	+5.8	+6.8	−1.0 [−3.7, +0.8]	–	–	10.9
Live, all (registered)	6+6	+4.2	+8.7	−4.4 [−9.4, −0.1]	−2.1 [−6.2, +1.9]	14/23	9.8/6.7
historical	2+2	+6.2	+5.8	+0.4	–	6/7	9.8/7.4
concurrent	4+4	+3.2	+10.1	−6.9 [−11.7, −2.6]	−5.8 [−9.4, −2.2]	8/16	9.7/6.4
Live minus recorded				−3.4 [−8.3, +1.8]	–	–	–

Table 40: Transfer after ten rounds on airline (points; held-out success after round 9 minus the base). Recorded: full and sequential evaluation's round values summed over rounds 0–9 of r1–r3, in full evaluation's states. Live: the registered comparison IL (sequential: cl1, cl2, s1–s4; full: r2, r3, f1–f4); historical loops ran before the eight concurrent ones. 90% intervals: bootstrap of rounds or loops and of tasks; adjusted for the calibrated noise band δ by least squares (post hoc, t intervals). Live minus recorded: one bootstrap that resamples loops, trajectories and tasks shared by both rows once (post hoc). Accepted changes and mean δ (points), sequential/full.

quential loops' states, full evaluation would have accepted in 20 of 60 rounds, against 23 acceptances in the full-evaluation loops' own 60 rounds; the sequential loops made 14 of the 20. Of the 9 fewer accepted changes, 6 were acceptances the rule stopped and 3 reflect fewer rounds with an admissible candidate. The candidates themselves, scored by complete evaluation against their own incumbents, were similar in the two arms: 86 and 88 evaluated, 27% and 32% admissible, mean change in evolve success -1.5 and -1.1 points, 36% and 35% positive. The 62 stopped candidates reached the proposer after 29 of 60 episodes on average, with estimates of -4.7 points against -2.5 under complete evaluation (correlation 0.53, opposite signs in 18%); 51 of them were stopped in rounds where the decision agreed with full evaluation's, so the history differed even where admission did not. Of the lost acceptances, cl1's two gained $+3.75$ and $+4.17$ points over their incumbents, and the four of IL's sequential loops, deployed for BK, $+3.3$, -0.8, $+2.9$ and $+7.9$. These descriptives do not separate the mechanisms: weaker incumbents make improvements easier to find, which could hide worse proposals, and no loop of IL restored a lost acceptance or fed complete estimates to a sequential loop's proposer; BK's ablations do (Appendix T).

T Blocked Loops (BK)

Design. Registered after IL's results and the post hoc analyses of Appendices R and S, before any of its episodes (`verify/PREREGISTRATION_BK.md`), BK runs four calibration blocks of airline loops, rounds 0–9 with $T = 20$, with IL's models, tasks, prompts and deployment code. Each block calibrates one noise band from three evaluations of the base harness and deploys the base once; its five loops start from that state and differ only in the selection step (`verify/live.py`): f, RRSI's full evaluation; s, sequential full evaluation as in IL; c, the cost-aware rule of Appendix R at $\gamma = 0.05$; h, which evaluates every candidate in full and accepts what full evaluation accepts but records a candidate the sequential rule would have stopped exactly as s records it (early-stopped estimate, the rule's reason, no attribution); and a, which accepts what the sequential rule would accept but records every candidate with its complete evaluation. In h and a the rule is replayed on each candidate's complete episodes in s's order to find where it would stop.

Coupling. To save compute, loops of a block whose rounds add the same thing to the loop's state (the history rows, including their reason text, the attributed candidates and the winner) share those rounds: a block starts one lineage holding all five loops, which drafts once, evaluates every screened candidate in full and computes each loop's round from those episodes as the loop's own rule would record it. Loops whose rounds differ are forked into their own lineage, which receives the state as of the start of the round and the round's episodes, reruns the round under its own rule and continues alone. Each loop therefore has the distribution of an independent loop of its rule, and loops are coupled only while their states are identical. Because the reason texts differ, h, c and the pair (f, a) separate in the first round in which a candidate is evaluated; a stays with f until the rule stops a candidate full evaluation admits, and s stays with h until the rule stops full evaluation's choice. Before registration the coupled code reproduced the records of IL's four full-evaluation loops in all 40 rounds, and every fork in two test blocks rerun from IL's states recorded the state its group predicted, without any new episode (`verify/bk_checks.py`).

Outcomes and analysis. Transfer is the held-out success of a loop's final incumbent minus its block's base, paired over the 20 test tasks (12 trials each). The registered contrasts are s−f, c−f, c−s and the admission and history main effects, $(s+a-f-h)/2$ and $(s+h-f-a)/2$, each a mean over blocks with a 90% interval from a lineage model: an additive block-plus-rule model fitted to f, h and c, which always run in lineages of their own (6 df), plus the coupled differences a−f and s−h, whose covariance with their partners follows from the shared rounds, with Satterthwaite df. A bootstrap over blocks and tasks ($B = 4000$) is reported beside it: $[-3.7, +1.8]$ for s−f, $[-1.9, +3.9]$ for c−f, $[-0.4, +4.1]$ for c−s, $[-3.4, -0.2]$ for admission and $[-2.1, +3.5]$ for history. The registered readings were fixed in advance: the loss is replicated if the interval of s−f lies below 0; the cost-aware rule reduces the loss if that of c−s lies above 0 and loses to full evaluation if that of c−f lies below 0; a mechanism is supported if its main effect lies below 0. By these readings IL's loss was not replicated, the loops cannot tell whether the cost-aware rule reduces the loss, admission is supported and history is not supported, which the registration does not read as the absence of an effect; the interaction, -1.0 $[-3.9, +1.8]$, includes 0. Under a simulated null before the runs, the model's 90% intervals covered 0 in 90–94% of replications, and its expected widths were 9.2 points for a difference of two rules, 3.6 for admission and 10.0 for history at a residual standard deviation of 3.5 points; the observed residual standard deviation was 1.5 points (Table 3).

Sensitivity (post hoc). The registration read each of the five contrasts by its own unadjusted 90% interval, a one-sided test at 5% each, with no adjustment for multiplicity. Table 3 adds 95% intervals from the same lineage model and Holm-adjusted one-sided p values in the direction of each reading (`verify/posthoc_bk.py`). The admission effect has one-sided $p = 0.038$, Holm-adjusted 0.19, a 95% interval of $[-3.4, +0.2]$ and a 95% bootstrap interval over blocks and tasks of $[-3.9, +0.1]$; Bonferroni intervals that hold every reading at a family level of 5% give $[-3.9, +0.8]$ for admission and $[-4.1, +2.4]$ for sequential minus full evaluation. Adjustment therefore changes one reading, admission's, from supported to not supported, and none of the others. With 1.4 df, the history effect's 95% interval is $[-4.2, +5.6]$.

Results. Table 3 gives transfer by block and rule and Table 41 what the loops did. The noise bands were 5.4, 7.2, 5.4 and 8.2 points. Full evaluation's loops gained $+6.7$ to $+11.2$ points, as IL's did ($+2.9$ to $+12.1$). The sequential loops accepted 10 changes and full evaluation's 12; in the sequential loops' own states full evaluation would have accepted in 16 rounds, and in 7 of them the rule had stopped full evaluation's choice. Pooled with IL's twelve loops by least squares with rule, centred noise band and an experiment indicator (registered secondary analysis), sequential minus full is -1.7 $[-4.1, +0.8]$. IL's four lost acceptances (s2 round 5, s3 round 9, s4 rounds 0 and 1), deployed against the incumbents they would have replaced, gained $+3.3$, -0.8, $+2.9$ and $+7.9$ points, and CL's two $+3.75$ and $+4.17$. The cost-aware loops' choices equalled full evaluation's in all 40 of their rounds, with 22 stops, none of a candidate full evaluation would have chosen; they cost 2.5% less than full evaluation's loops.

Changes after registration. The container restarted twice, between 12:36 and 14:42 UTC and between about 16:45 and 19:44 UTC on the day of the runs, once after every round of the first two blocks had finished and once during rounds 7–9 of the last two. Each time the unfinished lineages were resumed with the same scripts, which reuse every recorded episode and skip finished work. No held-out success rate of BK or of IL's lost acceptances was examined before every deployment had finished; the registered analysis then ran unchanged. BK cost $51.98 against a quote of about $60 and a stop line of $85.

U Reproducibility

The code, environments, banks and Study 2's registered pipeline are submitted as anonymised supplementary material. *Study 1*: the environments and fault suites (`agent_exp/env.py`, `agent_exp/tau2_env.py`), every prompt used for attribution, taxonomy induction, labelling, patch generation and judging (`agent_exp/analyst.py`), the agent and analyst settings (DeepSeek-V4-Flash and DeepSeek-V4-Pro through the provider's Anthropic-compatible endpoint, at most 4,000 output tokens per agent call and 8,000 per structured analyst call, provider-default sampling), and the complete intervention banks (`runs/tau2_*/bank.json`: every replay, judge check, null replay and regression run with

Arm	Acc.	Full acc.	Choice stopped	Stopped	Cand. ep.	Cost ($)	Fork round
Full	12	12	1	38	855	3.40	–, –, –, –
History only	12	12	4	39	900	3.60	0, 0, 0, 1
Cost-aware seq.	11	11	0	22	685	3.31	0, 0, 0, 1
Admission only	11	13	2	42	900	3.49	8, 0, –, –
Sequential	10	16	7	37	535	3.00	0, 7, 4, 2

Table 41: Experiment BK, loops summed (or averaged) over the four blocks. Acc.: accepted changes; Full acc.: rounds in which full evaluation would have accepted in the loop's states; Choice stopped: of those, rounds in which the sequential rule stopped full evaluation's choice (for f and h replayed, for c its own cost-aware rule); Stopped: candidates the loop's sequential rule stopped (replayed for f); Cand. ep.: candidate-evaluation episodes per loop that the loop's own rule runs; Cost: the loop's dollar cost as if it had run alone, IL's accounting, calibration included; Fork round: the round in which the lineage holding the loop at the end separated from its parent, per block (–: the block's first lineage). In b1, h and s stayed together; in b3 and b4, a stayed with f.

its token count and source trace). Bank construction is deterministic given the seeds in `agent_exp/bank.py`. Every table and figure of Study 1 is produced offline from the banks and held-out runs by `agent_exp.report_tau2`, from the outputs of `agent_exp.reanalysis`, `reanalysis3`, `reanalysis5` (binomial harm model), `deployment` (deployment gains, $\mathrm{Net}(N)$, transfer) and `judge_configs`. *Study 2*: the vendored RRSI loop with a list of our changes (`rrsi/`, `rrsi/NOTICE.md`), the τ^2 domains (`domains/tau2*`), the harness check against τ^2's orchestrator (`agent_exp/harness_check.py`), and the evidence pipeline, pre-registration and registered analysis (`verify/`); Table 23 is produced by `verify.report_e1` from the output of `verify.analyze`, the analyses with the common error check by `verify.posthoc_e1`, r2 and r3's registered analysis and Table 21 by `verify.e1r` and `verify.report_e1r`, the closed loops by `verify/run_cl.sh`, `verify.cl`, `verify/run_il.sh` and `verify.il` (Tables 38 and 39), BK by `verify/run_bk_all.sh` and `verify.bk` (Table 3) and `verify.report_bk` (Table 41), and the post hoc analyses of Appendices R and S by `verify.posthoc_r9`, `verify.posthoc_r10`, `verify.posthoc_r11` (joint bootstrap, Tables 2 and 40) and `verify.posthoc_r12` (dependence of the cost and success changes). Re-collecting any run needs API access and is not bit-reproducible, because the provider does not offer seeded sampling.

V The CARVE Design Study

Why the simulator was optimistic. Our first design study, CARVE, used a simulator in which it needed half the budget of the strongest baseline (Appendix V). Calibrated to the measured judge accuracy, spurious recovery, analyst accuracy and cost ratio, the simulator still predicts that CARVE leads uncertainty sampling by 14 points on retail at budget 50; on the bank it trails by 15 points at budget 40 (significant harm, $c_{\mathrm{w}} = 0.02$). The simulator shared the method's structure: one b for all categories, value spread over many edges, and a fixed cost c that made every wrong change harmful, an assumption that was never measured and on which the value of verification largely depends.

Our first approach treated judge checks as cheap and replays as expensive, and allocated a budget over both by Bayesian experimental design. We keep it here as the multi-fidelity method compared in §4, together with the simulator in which it was developed. The simulator shares the method's structure. Its results show the method working under its own assumptions, and they are not evidence about real agents.

Calibrated simulation. With the measured judge accuracy, spurious recovery, analyst accuracy and cost ratio, the simulator predicts that CARVE runs no judge checks on ShopDesk and retail and only a few on airline, which matches the break-even analysis in Table 6. For τ^2 retail it predicts net gains at budget 50 of 0.51 for CARVE and 0.38 for uncertainty sampling; for airline, 0.33 and 0.31; for ShopDesk, 0.81 and 0.79.

V.1 Problem Formulation

An agent is built from *components* $j \in \{1, \ldots, J\}$: sections of its system prompt, tool descriptions, tool implementations, orchestration settings. Its failures have been grouped into *categories* $i \in \{1, \ldots, I\}$ by an LLM, with f_i the share of failures in category i. For every cell (i, j) an analyst

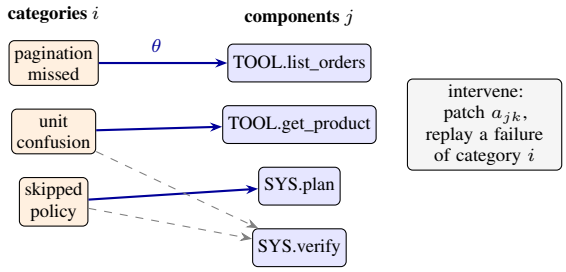


Figure 3: The category–component graph. Solid edges are true causal edges; dashed ones are plausible but wrong attributions (e.g. blaming verification for a unit error). Each intervention patches one component and replays one failure.

LLM can propose K candidate patches a_{ijk} that rewrite component j to address category i.

Interventions. An intervention (i, j, k, ϕ) samples a failed trajectory labelled i, applies patch a_{ijk}, and observes a binary outcome at fidelity ϕ. A *full replay* ($\phi = F$) resumes the episode from the attributed failing step and records whether the task now succeeds. A *single-step check* ($\phi = S$) regenerates only the failing step and asks a judge whether the diagnosed error is gone. They cost c_F and $c_S \ll c_F$.

Edges and patch quality. We write $e_{ij} \in \{0, 1\}$ for whether component j is a cause of category i, meaning some change to j repairs a non-trivial fraction of its failures, and $q_{ijk} \in [0, 1]$ for the repair probability of patch k when $e_{ij} = 1$. This separation matters because a negative replay outcome is ambiguous: either j is the wrong component or a_{ijk} is a bad patch. Testing several patches per cell resolves the ambiguity.

Objective. After spending at most budget B, the method accepts a set of cells $\mathcal{A}$ and, for each, a patch $k^\star_{ij}$. Accepting a true edge repairs a fraction $q_{ijk^\star}$ of category i's failures; accepting a false edge costs a regression c_{fp}, reflecting that unnecessary changes perturb behaviour on tasks that used to succeed (Vasudev et al., 2026). We measure

$$G(\mathcal{A}) = \sum_i f_i \Big(1 - \prod_{j \in \mathcal{A}_i^+} (1 - q_{ijk^\star_{ij}})\Big) - c_{\text{fp}} \big|\{(i,j) \in \mathcal{A} : e_{ij} = 0\}\big|, \tag{5}$$

where $\mathcal{A}_i^+ = \{j : (i, j) \in \mathcal{A},\ e_{ij} = 1\}$. This is the net failure mass repaired; we report it as a fraction of the oracle value obtained by accepting exactly the true edges with their best patches.

V.2 CARVE

Observation model For a trajectory labelled i and patch a_{ijk}, the probability that a full replay succeeds is

$$p_{ijk} = \begin{cases} \lambda\, q_{ijk} + (1 - \lambda)\, b & e_{ij} = 1, \\ b & e_{ij} = 0, \end{cases} \tag{6}$$

where b is the spurious recovery rate (stochastic re-generation sometimes succeeds regardless of the patch) and λ is the accuracy of the category labels used to sample trajectories; a mislabelled trajectory is treated as unaffected by the patch. A single-step check returns $z = 1$ with probability $s\,p + \epsilon\,(1-p)$, where s and ϵ are the sensitivity and false-positive rate of the check relative to the full outcome. All four nuisance parameters can be estimated from a small calibration set on which both fidelities and a human audit are run.

Attribution-informed prior Let n_{ij} be the number of failures in category i that the analyst attributed to component j, and $\hat{r}$ the analyst's attribution accuracy estimated on a small audited sample. With d the expected number of causal components per category and $\rho = d/J$ the base edge density, we set

$$\pi_{ij} = (1 - \hat{r})\,\rho + \hat{r}\,\min\big(1,\ d\,n_{ij}/n_i\big), \tag{7}$$

so that an unreliable analyst ($\hat{r} \to 0$) yields the flat prior ρ and a reliable one concentrates mass on the components it names. Eq. 7 is a heuristic interpolation, not a calibrated model; with a larger audit, π could instead be fitted by logistic regression on audited attributions. Patch quality has prior $q_{ijk} \sim \text{Beta}(\alpha_q, \beta_q)$.

Posterior Cells are independent given the data. For each cell we keep the posterior log-odds of e_{ij} and, for each patch, a posterior over q_{ijk} on a fixed grid of $G = 101$ points. With $\mathcal{D}_{ijk}$ the outcomes observed for patch k,

$$\text{logit}\, P(e_{ij}{=}1 \mid \mathcal{D}) = \text{logit}\, \pi_{ij} + \textstyle\sum_k \log\big(\bar{L}^1_{ijk} / L^0_{ijk}\big), \tag{8}$$

where $\bar{L}^1_{ijk} = \int P(\mathcal{D}_{ijk} \mid q, e_{ij}{=}1)\, dP(q)$ is the marginal likelihood of patch k's outcomes under a true edge and $L^0_{ijk} = P(\mathcal{D}_{ijk} \mid e_{ij}{=}0)$. Every update multiplies one grid vector by a likelihood vector, so updates and look-aheads are vectorised dot products.

Selecting interventions The value of the current decision for cell (i, j), with P_{ij} its edge posterior and $\hat{Q}_{ij} = \max_k \mathbb{E}[q_{ijk} \mid \mathcal{D}]$, is

$$V_{ij} = \max\left(P_{ij}\, f_i\, \hat{Q}_{ij} - (1 - P_{ij})\, c_{\text{fp}},\ 0\right), \quad (9)$$

the expected contribution of the cell to Eq. 5 under the Bayes decision (accept iff the first term is positive), treating cells as additive. Eq. 5 combines components of one category multiplicatively, so V_{ij} slightly overstates the value of a second cause for an already repaired category; we accept this approximation because it keeps cells independent. For an action $a = (i, j, k, \phi)$ repeated n times, let $V'_{ij}(s)$ be the value after observing s successes. The n-step knowledge gradient is

$$\text{KG}_n(a) = \sum_{s=0}^{n} P(s \mid \mathcal{D}, a)\, V'_{ij}(s) - V_{ij}, \quad (10)$$

and CARVE runs the action maximising

$$\text{score}(a) = \max_{n \in \{1,2,4,8\}} \frac{\text{KG}_n(a)}{n\, c_\phi}. \quad (11)$$

Looking ahead several observations matters because a single binary outcome often cannot flip a decision, in which case $\text{KG}_1 = 0$ even for a valuable cell; maximising over n is the standard remedy (Frazier et al., 2008). Because cells are independent, only the observed cell's scores change after each outcome, so each step rescores one cell in $O(KG)$ time after an initial pass over all cells.

Decisions and stopping. At any point the method accepts the cells whose current value is positive and applies, for each, the patch with the highest posterior mean quality. It stops early when no action has a score above a small threshold, i.e. when no remaining experiment is expected to be worth its cost.

V.3 When Do Priors and Cheap Checks Help?

These results analyse an idealised per-cell sequential test, not CARVE's full policy; they explain which quantities govern the method's behaviour. We consider a single cell and a single patch with known success probabilities p_1 (true edge) and $p_0 = b$, under a rule that stops when the posterior log-odds leaves $[-A, A]$, $A = \log \frac{1-\delta}{\delta}$. Let $D_1 = \text{KL}(p_1 \| p_0)$ and $D_0 = \text{KL}(p_0 \| p_1)$ be the expected log-likelihood-ratio drift per observation under each hypothesis. Proofs, which follow Wald's approximations (Wald, 1945), are in Appendix V.5.

Proposition 3 (Value of the LLM prior)**.** *Let $|\ell| < A$ and neglect overshoot and stopping at the wrong boundary (whose probability is approximately $\delta e^{-\ell}$ for a true edge and δe^{ℓ} for a false one). A cell with initial log-odds ℓ then needs $N_1(\ell) = (A - \ell)/D_1$ observations in expectation if it is a true edge and $N_0(\ell) = (A + \ell)/D_0$ if not. Replacing a flat prior ℓ_ρ by LLM-derived log-odds ℓ_{ij} saves, in expectation over cells,*

$$\Delta = \frac{\rho\, \mathbb{E}[\delta\ell \mid e{=}1]}{D_1} - \frac{(1 - \rho)\, \mathbb{E}[\delta\ell \mid e{=}0]}{D_0}, \quad (12)$$

with $\delta\ell = \ell_{ij} - \ell_\rho$, observations per cell, which is positive exactly when the prior raises true edges more, relative to their base rate and drift, than it raises false ones.

Under Eq. 7, $\ell_{ij} \to \ell_\rho$ as $\hat{r} \to 0$, so $\Delta \to 0$: a useless analyst costs nothing beyond the flat prior, whereas trusting its labels outright (LLM-only) makes permanent errors. A miscalibrated analyst can make Δ negative, but only by an amount linear in the log-odds shift, after which the evidence takes over. Appendix V.7 compares these predictions with simulated sequential tests.

Proposition 4 (Cheap checks)**.** *Let D_e^S denote the drifts of single-step outcomes, $z \sim \text{Bern}(s\, p_e + \epsilon(1 - p_e))$. Under the same approximations, a test using only single-step checks reaches the stopping boundary at lower expected cost than one using only full replays if and only if*

$$\frac{c_S}{c_F} < \frac{D_e^S}{D_e}, \quad (13)$$

where e is the true state of the cell.

Since e is unknown, the proposition gives a clear rule only when the cost ratio lies outside the interval spanned by the two critical ratios; inside it, which fidelity is better depends on the posterior, which is what CARVE's look-ahead weighs.

With our default parameters ($p_1 = 0.54$, $b = 0.05$, $s = 0.85$, $\epsilon = 0.15$) the critical ratio is 0.31 for true edges and 0.44 for false ones; with a weak check ($s = 0.65$, $\epsilon = 0.35$) it falls to 0.05–0.08. Section V.4 tests both predictions.

V.4 Controlled Experiments

Real agents do not reveal which component truly caused a failure, so we first evaluate in a testbed where the causal graph is known and the method's model is deliberately misspecified.

Setup

Testbed. Each world has $I = 12$ categories with Zipfian frequencies, $J = 20$ components and $K = 3$ patches per cell. Every category has one or two true components and one *decoy* component that wrong attributions prefer, mimicking symptom-level mislabelling. Patch quality is bimodal: with probability 0.3 a patch is bad ($q \sim \mathrm{Beta}(1, 9)$), otherwise $q \sim \mathrm{Beta}(6, 3)$. A trajectory sampled for category i is mislabelled with probability $1 - \lambda = 0.2$, in which case it comes from another category (weighted by frequency) and is repaired only if *its own* edge to j exists, unlike the model's flat treatment of mislabelled traces. The analyst attributes 20 failures per category with accuracy $r = 0.5$; wrong attributions go to the decoy with probability 0.6. The analyst's accuracy is estimated from an audit of 20 attributions. Costs are $c_F = 1$, $c_S = 0.1$, $c_{\text{fp}} = 0.02$, and $s = 0.85$, $\epsilon = 0.15$, $b = 0.05$. Section V.4 relaxes the assumptions the generator shares with the model. Full details are in Appendix V.6.

Baselines. *LLM-only* accepts, for every category, the most-attributed component and any component holding at least 40% of the attributions, with the first patch, i.e. it acts on an induced taxonomy without verification. *Replay-each* tests attributed hypotheses in order of attribution count with three full replays of the first patch and accepts a hypothesis if at least two succeed, mirroring do-then-verify methods such as DoVer; *Replay-each, 3 patches* gives each hypothesis two replays of each of three patches and accepts if any patch succeeds twice; *Sequential-each* replays each hypothesis, cycling through patches, until CARVE's own posterior leaves $[0.05, 0.95]$. Untested hypotheses fall back to LLM-only, except for Sequential-each, which uses the Bayes decision. *Thompson sampling* samples (e, q) from the posterior and replays the cell and patch with the largest sampled repair. *Uncertainty sampling* replays the cell with the largest frequency-weighted uncertainty $f_i \min(P_{ij}, 1-P_{ij})$, cycling through patches, at full fidelity; *Uncertainty + cheap checks* first runs three single-step checks per (cell, patch) and then confirms with full replays. All baselines except LLM-only and the Replay-each variants share CARVE's posterior and decision rule, so comparisons among them isolate the selection rule.

Method	Budget (full-replay units) 20	50	100	200
LLM attribution only	0.297 ±0.021	0.297 ±0.021	0.297 ±0.021	0.297 ±0.021
Replay-each (do-then-verify)	0.255 ±0.021	0.270 ±0.021	0.466 ±0.027	0.468 ±0.026
Replay-each, 3 patches	0.301 ±0.025	0.317 ±0.026	0.401 ±0.026	0.612 ±0.026
Sequential-each (adaptive stopping)	0.426 ±0.016	0.522 ±0.018	0.643 ±0.016	0.813 ±0.014
Thompson sampling	0.407 ±0.020	0.419 ±0.020	0.500 ±0.017	0.625 ±0.014
Uncertainty sampling	0.455 ±0.016	0.581 ±0.019	0.690 ±0.017	0.804 ±0.016
Uncertainty + cheap checks	0.549 ±0.018	0.662 ±0.016	0.784 ±0.015	0.876 ±0.009
CARVE (all components)	0.666 ±0.019	0.787 ±0.015	0.853 ±0.011	0.902 ±0.009
full replay only	0.514 ±0.016	0.609 ±0.020	0.697 ±0.017	0.825 ±0.012
myopic (n=1 look-ahead)	0.591 ±0.016	0.652 ±0.014	0.706 ±0.016	0.729 ±0.014
uninformed prior	0.347 ±0.019	0.474 ±0.017	0.631 ±0.015	0.800 ±0.012
one patch per cell	0.571 ±0.019	0.636 ±0.020	0.673 ±0.019	0.688 ±0.020
no label-noise term	0.663 ±0.020	0.772 ±0.018	0.864 ±0.012	0.922 ±0.006

Table 42: Net gain as a fraction of the oracle (mean ± s.e., 50 worlds). The lower block ablates CARVE.

Metrics. Net gain (Eq. 5) as a fraction of the oracle, and edge F1. We report means and standard errors over 50 worlds for the main experiment and 30 for the others; all methods see the same worlds, so differences are paired.

Main results Figure 4 and Table 42 show that CARVE is best at every budget. With 50 units it recovers 79% of the oracle gain; the strongest baseline, uncertainty sampling with the same cheap checks, recovers 66% and needs about twice the budget (100 units, 78%) to match it. The gap narrows at large budgets (90% vs. 88% at 200), as every reasonable method eventually resolves the frequent categories.

Where the gain comes from. Two comparisons separate the selection rule from the cheap checks. With full replays only, CARVE beats uncertainty sampling by 1–6 points (e.g. 0.609 vs. 0.581 at budget 50): the knowledge-gradient rule alone helps only modestly. Given cheap checks, its advantage grows to 7–13 points (0.787 vs. 0.662 at 50), because it decides per action whether a check is informative enough for its price and stops checking a cell once checks can no longer change the decision, whereas a fixed check-then-confirm schedule wastes checks on resolved cells. Most of CARVE's advantage is therefore in *how it uses* cheap checks, which also makes their calibration critical (§V.4).

Verification baselines. LLM-only is stuck at 30% because it accepts decoys and bad first patches. Replay-each is initially *worse* than not verifying: with $p_1 \approx 0.54$, requiring two successes in three replays rejects about 44% of true edges even when the patch is good, and its fixed per-hypothesis cost exhausts small budgets on frequent but already-certain hypotheses. Testing three patches (61% at 200) or stopping adaptively (81% at 200) helps, but both remain well below CARVE at small bud-

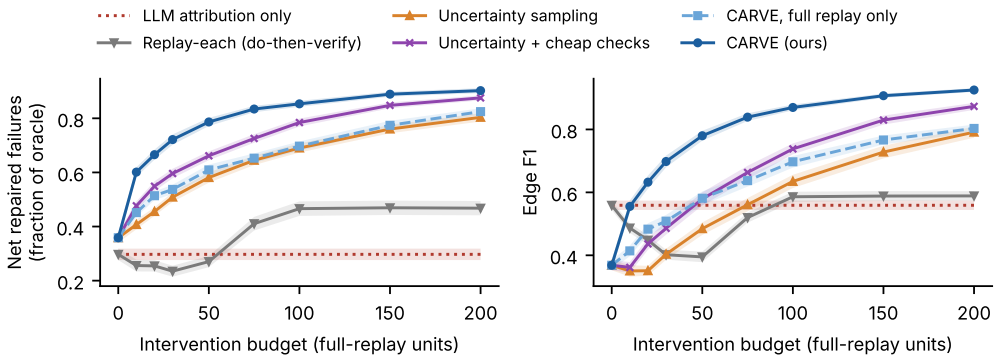


Figure 4: Net repaired failure mass (left) and edge F1 (right) against intervention budget, 50 worlds, shaded ± 1 s.e.

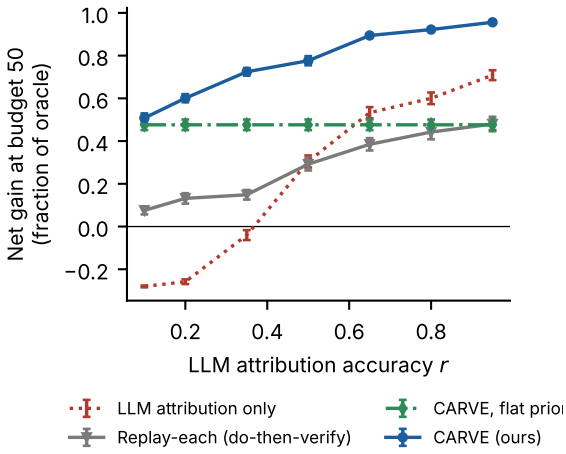


Figure 5: Net gain at budget 50 as the analyst's attribution accuracy r varies (30 worlds, ± 1 s.e.).

gets (32% and 52% at 50). Thompson sampling spends most replays on cells it already believes in and reaches only 63% at 200.

Ablations. Removing single-step checks costs 15–18 points at small budgets. Restricting the look-ahead to one observation is below the full method at every budget and plateaus at 73%, because many cells cannot be flipped by a single outcome and are never revisited. A flat prior costs 32 points at budget 20 and still 10 points at 200. With one patch per cell the method plateaus at 69%: it cannot tell a wrong component from a bad patch and rejects true edges for good. Dropping the label-noise term makes no difference at the default label accuracy $\lambda = 0.8$ (it is even slightly better at large budgets, 0.922 vs. 0.902 at 200), but it matters when labels are worse (§V.4).

When does the LLM prior help? Figure 5 varies the analyst's accuracy r. As Proposition 3 suggests, CARVE improves monotonically with r and is never below the flat-prior variant: at $r = 0.1$ it scores 0.51 ± 0.02 against 0.48 ± 0.03. The prior is not exactly flat there (the decoy still receives about twice the base rate), but its shifts are small, so evidence quickly dominates. Acting on attributions directly is harmful below $r \approx 0.35$, where LLM-only has a *negative* net gain (regressions outweigh repairs), and even at $r = 0.95$ it reaches only 71% of the oracle against 96% for CARVE, since it cannot detect bad patches. Replay-each improves with r and only matches the flat-prior variant of CARVE at $r = 0.95$ (0.48 vs. 0.48).

When do cheap checks help? Figure 7 (Appendix V.8) is qualitatively consistent with Proposition 4. With the default check, the benefit of cheap checks shrinks as their cost grows and vanishes between 0.35 and 0.5 (0.64 ± 0.02 vs. 0.61 ± 0.03 at 0.35, overlapping; 0.59 vs. 0.61 at 0.5), in line with the predicted critical ratios of 0.31 and 0.44. At cost 0.1, a check of quality $s - \epsilon = 0.5$ (critical ratio 0.14–0.22) still helps, while one of quality 0.3 (critical ratio 0.05–0.08) brings no gain. At cost 0.5 CARVE is slightly below its full-replay-only variant although its action set contains every full-replay action, which shows that the greedy rule is not exactly cost-optimal when the two fidelities are nearly equally efficient.

Robustness The generator shares several assumptions with the model: check errors depend only on the full outcome, cells are independent, the analyst's accuracy is estimated without bias, and the regression cost is small. Table 43 and Appendix V.8 relax each in turn.

Worse labels. With label accuracy $\lambda = 0.5$ both methods lose ground, and the label-noise term now

Condition (budget 50)	Unc. + checks	CARVE
Default testbed	0.656 ±0.022	0.776 ±0.021
Label accuracy $\lambda = 0.5$	0.513 ±0.023	0.688 ±0.023
Analyst overconfident ($\hat{r}+0.3$)	0.752 ±0.018	0.857 ±0.014
Analyst underconfident ($\hat{r}-0.3$)	0.495 ±0.023	0.653 ±0.032
Judge fooled by bad patches	0.579 ±0.022	0.377 ±0.027
Regression cost $c_{\text{fp}} = 0.005$	0.711 ±0.022	0.798 ±0.016
Regression cost $c_{\text{fp}} = 0.05$	0.643 ±0.024	0.793 ±0.019
Regression cost $c_{\text{fp}} = 0.1$	0.624 ±0.023	0.823 ±0.018

Table 43: Violations of the model's assumptions (30 worlds). The judge-fooled row is the one setting where CARVE loses.

matters: CARVE scores 0.688, against 0.588 without the term and 0.513 for the baseline.

Biased analyst audit. If the audit overstates the analyst's accuracy by 0.3, CARVE actually improves (0.857), because a stronger prior concentrates the budget and the evidence corrects the decoys. Understating it by 0.3 costs 12 points (0.653), still above the baseline. Under-trusting a decent analyst is thus the more costly error in this testbed.

Regression cost. Raising c_{fp} from 0.005 to 0.1 leaves CARVE between 0.78 and 0.82, while LLM-only falls from 0.39 to -0.13: the more costly a wrong change, the more verification is worth.

Joint causes. When 30% of categories need two components changed together (a single patch then repairs only a fifth as often), violating cell independence, CARVE drops from 0.787 to 0.661 at budget 50 but stays ahead of uncertainty sampling with checks (0.558; Table 46).

Correlated check errors. The one setting where CARVE loses is when the single-step judge is fooled by plausible-looking bad patches (false-positive rate 0.6 for patches with $q < 0.3$). CARVE then trusts checks that systematically pass bad patches and falls to 0.377, below every method that relies on full replays (0.58–0.61). Modelling the dependence (a quality-dependent false-positive rate in the likelihood) restores it to 0.584, on par with full replays, at a cost of 9 points when the judge is not fooled (0.684 vs. 0.776). Whether real judges behave this way is therefore the most important quantity to measure before relying on cheap checks.

Nuisance parameters and scale. Misspecifying b, λ or the patch-quality prior changes the result by at most 5 points (Table 47, Appendix V.8); misstating the check's sensitivity and false-positive rate costs 6–14 points. On a larger graph ($I = 20$, $J = 50$; Table 45) the gap widens: at budget 50 CARVE reaches 78% of the oracle against 60% for uncertainty sampling with checks.

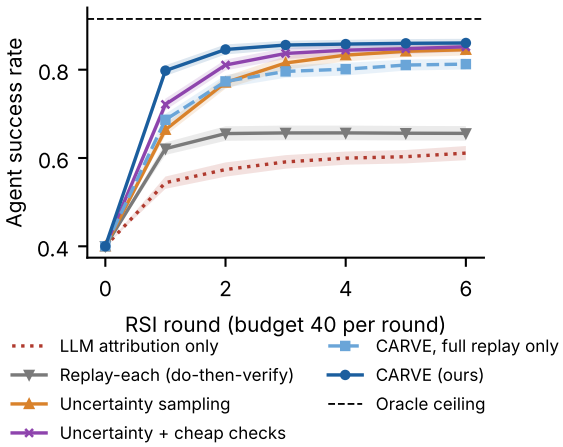


Figure 6: Agent success over six rounds of attribute–verify–patch, with a budget of 40 units per round (30 worlds, ±1 s.e.).

Inside a self-improvement loop Finally we embed each method in a loop. The agent starts at 40% success. In each round the analyst attributes 240 new failures, sampled in proportion to the current failure distribution; the method spends 40 units; accepted patches are applied, which repairs failures (true edges) or adds a regression of c_{fp} times the current failure mass (false edges); and the failure distribution is recomputed, so priorities shift towards categories that remain. Posteriors carry over between rounds.

Figure 6 shows that CARVE reaches 80% success after one round and 85% after two, close to its plateau of 86% (the oracle ceiling is 91.5%). Uncertainty sampling with checks needs four rounds to reach 84% and ends at 85%; without checks it needs six. LLM-only and Replay-each plateau at 61% and 66%: once their accepted false edges and rejected true edges are fixed in place, new attributions do not correct them.

V.5 Proofs for CARVE's propositions

Proposition 3. Let $L_t = \ell + \sum_{u \le t} \log \frac{P(o_u|e=1)}{P(o_u|e=0)}$ be the posterior log-odds after t observations. Under $e = 1$, the increments are i.i.d. with mean $D_1 = \mathrm{KL}(p_1 \| p_0) > 0$. The rule stops at $N = \inf\{t : L_t \notin (-A, A)\}$. By Wald's identity, $\mathbb{E}_1[L_N - \ell] = D_1 \mathbb{E}_1[N]$. Wald's approximations give $P_1(L_N \le -A) \approx e^{-(A+\ell)} \approx \delta e^{-\ell}$

and $P_0(L_N \geq A) \approx \delta e^{\ell}$. Neglecting overshoot and this probability (which requires $\delta e^{|\ell|} \ll 1$), $L_N = A$, hence $\mathbb{E}_1[N] = (A - \ell)/D_1$; if $\ell \geq A$ no observation is taken, which is why we assume $|\ell| < A$. Symmetrically, under $e = 0$ the drift is $-D_0$ and $\mathbb{E}_0[N] = (A + \ell)/D_0$. Taking the difference between ℓ_ρ and ℓ_{ij} and averaging over cells with $P(e{=}1) = \rho$ gives Δ. When the patch quality q is unknown, outcomes are exchangeable rather than i.i.d. under the marginal likelihood, and Wald's identity does not apply directly; conditioning on q recovers the result with $D_1 = D_1(q)$, which approaches zero for a bad patch. This is the formal reason a single bad patch can stall verification and why CARVE keeps several patches per cell.

Proposition 4. By the same argument a single-step test needs $(A \mp \ell)/D_e^S$ observations at cost c_S each, and a full-replay test needs $(A \mp \ell)/D_e$ at cost c_F. The single-step test is cheaper iff $c_S/D_e^S < c_F/D_e$. Since the single-step outcome is a garbling of the full outcome, the data-processing inequality gives $D_e^S \leq D_e$, so the critical ratio is at most one.

V.6 Simulation Details

Category frequencies are $f_i \propto 1/i$. For each category the number of true components is uniform on $\{1, 2\}$ and the decoy is a further distinct component. The audit estimate $\hat{r}$ is a binomial proportion from 20 audited attributions. The prior uses $d = 1.5$. The model's grid has 101 points; look-ahead horizons are $\{1, 2, 4, 8\}$; the stopping threshold is 10^{-6}. Replay-each uses three replays and accepts with at least two successes. Uncertainty sampling switches to an untried patch once the current patch has three full observations. In the loop experiment, 240 attributions per round are allocated to categories by a multinomial draw with the current failure distribution. All randomness is seeded; the code reproduces every number in this paper with `python -m sim.experiments all`.

V.7 Checking Proposition 3

Table 44 compares the predicted expected number of observations with simulated sequential tests ($p_1 = 0.543$, $p_0 = 0.05$, $\delta = 0.02$, 4000 runs per row). The predictions capture the direction and approximate size of the effect of the initial log-odds; they are least accurate when $|\ell|$ is large, where overshoot and wrong-boundary exits (last column) are no longer negligible.

ℓ	e	predicted N	simulated N	wrong exit
-3	1	7.17	5.94	0.242
-3	0	1.55	2.54	0.000
-2	1	6.13	6.28	0.117
-2	0	3.28	3.73	0.001
-1	1	5.09	5.79	0.047
-1	0	5.02	5.21	0.003
$+0$	1	4.05	4.76	0.012
$+0$	0	6.75	7.28	0.009
$+1$	1	3.01	4.27	0.006
$+1$	0	8.48	8.58	0.015
$+2$	1	1.97	2.97	0.001
$+2$	0	10.22	10.23	0.068

Table 44: Proposition 3: predicted vs. simulated observations to a decision.

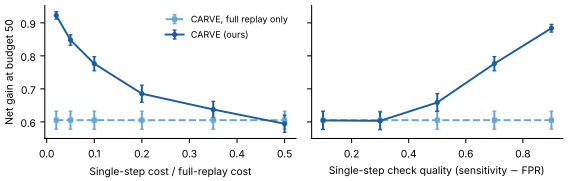


Figure 7: Effect of the single-step check's cost (left, quality fixed at $s{=}0.85$, $\epsilon{=}0.15$) and quality (right, cost fixed at 0.1) on net gain at budget 50.

V.8 Additional Results

Method	50	100	300
LLM attribution only	0.231 ±0.033	0.231 ±0.033	0.231 ±0.033
Replay-each (do-then-verify)	0.224 ±0.034	0.400 ±0.037	0.531 ±0.032
Uncertainty sampling	0.502 ±0.022	0.637 ±0.020	0.822 ±0.015
Uncertainty + cheap checks	0.603 ±0.019	0.746 ±0.018	0.873 ±0.013
CARVE, full replay only	0.613 ±0.013	0.710 ±0.012	0.834 ±0.010
CARVE (ours)	0.780 ±0.016	0.860 ±0.011	0.933 ±0.008

Table 45: Net gain on a larger graph ($I{=}20$, $J{=}50$), 20 worlds.

Method	20	50	100	200
LLM attribution only	0.228 ±0.028	0.228 ±0.028	0.228 ±0.028	0.228 ±0.028
Replay-each	0.202 ±0.027	0.207 ±0.023	0.348 ±0.033	0.362 ±0.036
Thompson sampling	0.325 ±0.031	0.343 ±0.032	0.398 ±0.032	0.484 ±0.028
Uncertainty sampling	0.345 ±0.036	0.465 ±0.035	0.544 ±0.035	0.662 ±0.029
Uncertainty + cheap checks	0.423 ±0.037	0.558 ±0.035	0.615 ±0.034	0.685 ±0.030
CARVE (ours)	0.573 ±0.029	0.661 ±0.029	0.701 ±0.027	0.745 ±0.026

Table 46: Net gain when 30% of categories need two components changed together (30 worlds).

Model assumption (true value)	Net gain
All nuisance parameters as in the generator	0.776 ±0.021
Spurious rate b: 0.01 (0.05)	0.811 ±0.012
Spurious rate b: 0.15 (0.05)	0.826 ±0.016
Label accuracy λ: 1.0 (0.8)	0.768 ±0.022
Label accuracy λ: 0.6 (0.8)	0.794 ±0.017
Check s/ϵ: .95/.05 (.85/.15)	0.717 ±0.016
Check s/ϵ: .70/.30 (.85/.15)	0.636 ±0.021
Patch prior $\mathrm{Beta}(1,1)$ ($\mathrm{Beta}(2,2)$)	0.811 ±0.015
Patch prior $\mathrm{Beta}(5,2)$ ($\mathrm{Beta}(2,2)$)	0.777 ±0.020

Table 47: CARVE with misspecified nuisance parameters, budget 50, 30 worlds. Several "wrong" values do slightly better than the generator's, a reminder that the model is misspecified in other ways too.